

分类号: U469.79

10710-2020222062



长安大学

专业硕士学位论文

面向无人方程式赛车的道路感知和轨迹规划
算法研究

高群森

导师姓名职称	李良敏 副教授		
申请学位类别	机械硕士	专业学位类别 及领域名称	车辆工程
论文提交日期	2023 年 4 月 5 日	论文答辩日期	2023 年 6 月 6 日
学位授予单位	长安大学		

Research on Road Perception and Trajectory Planning Algorithms for FSAC

A Thesis Submitted for the Degree of Master

Candidate: Gao Qunsen

Supervisor: Associate Prof. Li Liangmin

Chang'an University, Xi'an, China

答辩委员会名单

答辩委员会	姓名	职称	工作单位
主席	梅魁志	教授	西安交通大学
委员	王掩刚	教授	西北工业大学
委员	崔红	教授	陕西理工大学
委员	徐志刚	教授	长安大学
委员	郭佳	高级工程师	陕西重型汽车有限公司
委员			
委员			
秘书	李兆凯	副教授	长安大学汽车学院

摘 要

如今,无人驾驶技术在交通运输、物流配送和商业运输等领域展现出巨大的潜力和价值。为确保无人驾驶汽车的安全和高效行驶,提供可靠的环境感知信息和行驶路径至关重要。本文以“中国大学生无人方程式大赛”为平台,针对无人方程式赛车的赛道感知和轨迹规划问题,开展以下研究:

(1) 基于 YOLOv7 的赛道锥桶识别。本文采用 YOLOv7 深度学习框架训练锥桶检测网络,以实现赛车对无人方程式比赛中锥桶赛道的感知。为了训练高性能的检测网络,本文制作各种的锥桶标注数据集。并利用训练模型完成锥桶检测、定位和排序任务,为赛道建图和路径规划提供感知基础。

(2) 基于双目视觉的赛道定位与建图。首先,利用 matlab 对双目相机进行标定,获得相机内外参数。接着,采用 SGM 立体匹配算法从左右相机图像中计算出深度信息。并对组合惯性导航系统进行标定,获取车辆的定位坐标。最后,融合锥桶世界坐标和赛车的定位坐标,建立赛道地图。

(3) 高速循迹赛道的全局参考线生成和轨迹规划。首先,以赛道地图为基础,利用 Delaunay 三角剖分算法求解赛道中心点。然后,利用三次样条插值拟合赛道中心线以及赛车建图时的行驶路径得到全局参考线,并在 Frenet 坐标系中利用参考线进行高速循迹赛道的轨迹规划,算法包括初始状态求解、横纵状态采样和最优轨迹生成等步骤。最后,在 matlab 中进行仿真实验,验证算法的可行性。

(4) 赛道建图和轨迹规划的实车验证。首先,搭建无人方程式赛车实验平台,阐述整车硬件架构和整车控制系。接着,在 ROS 中完成赛道建图和轨迹规划算法的软件设计。最后,通过实车实验,验证无人方程式赛车的赛道感知和轨迹规划系统的可行性。

关键词:无人方程式赛车, YOLOv7, 双目立体视觉, Delaunay 三角剖分, Lattice 轨迹规划

Abstract

Nowadays, autonomous driving technology has shown immense potential and value in fields such as transportation, logistics, and commercial transport. To ensure the safe and efficient operation of autonomous vehicles, reliable environment perception information and driving paths are crucial. This thesis focuses on the issues of track perception and trajectory planning in the unmanned formula racing car competition, using "Formula Student Autonomous China FSAC" as a platform. The main research contents of this thesis are as follows:

(1) Track cone identification based on YOLOv7. This thesis uses the YOLOv7 deep learning framework to train the cone detection network and achieve perception of the cone track in the FSAC. To train a high-performance detection network, a large number of cone labeling datasets were created to train the network model. The trained model is used to complete cone detection, positioning, and sorting tasks, providing a perception basis for track mapping and path planning.

(2) Track positioning and mapping based on binocular vision. Firstly, the binocular camera is calibrated using MATLAB to obtain the camera's intrinsic and extrinsic parameters. Then, the SGM stereo matching algorithm is used to calculate depth information from the left and right camera images. Next, the inertial navigation system is calibrated to obtain the vehicle's positioning coordinates. Finally, the cone world coordinates and the vehicle's positioning coordinates are fused to establish a track map.

(3) Global reference line generation and trajectory planning for high-speed track following. Firstly, based on the track map, the Delaunay triangulation algorithm is used to solve the center points of the track, and then the cubic spline interpolation is used to fit the centerline of the track. The global reference line is obtained by smoothing the driving path of the vehicle using the cubic spline interpolation, and the Frenet coordinate system is used for high-speed track following trajectory planning, including steps such as initial state calculation, lateral and longitudinal state sampling, and optimal trajectory generation. Finally, the feasibility and effectiveness of the algorithm are verified through simulation.

(4) Real car verification of track mapping and trajectory planning. Firstly, an unmanned formula racing car experimental platform is established, describing the overall hardware architecture and the entire vehicle control system. Then, the track mapping and trajectory planning algorithms are designed using ROS. Finally, the feasibility of the unmanned formula racing car's track perception and trajectory planning system is verified through real-car experiments.

Key words: Driverless Formula Car, YOLOv7, Binocular Stereo Vision, Delaunay Triangulation, Lattice Trajectory Planning.

目 录

第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	2
1.2.1 目标检测研究现状.....	2
1.2.2 基于视觉的障碍物定位研究现状.....	4
1.2.3 轨迹规划研究现状.....	5
1.3 主要研究内容.....	8
第二章 基于 YOLOv7 的锥桶识别	10
2.1 目标检测任务分析.....	10
2.2 YOLOv7 算法原理.....	11
2.2.1 骨干网络(Backbone).....	12
2.2.2 检测头(head)	14
2.3 数据集创建与模型验证.....	16
2.3.1 数据集创建与标注.....	17
2.3.2 模型训练与验证.....	18
2.3.3 锥桶检测与排序.....	19
2.4 本章小结.....	21
第三章 基于双目视觉的赛道定位与建图	22
3.1 基于双目相机的锥桶坐标获取	22
3.1.1 双目相机选型.....	22
3.1.2 单目相机成像模型.....	23
3.1.3 双目空间坐标定位原理.....	27
3.1.4 相机标定.....	28
3.2 获取锥桶世界坐标.....	31
3.3 基于组合惯导的自车定位	35
3.3.1 组合惯性导航系统标定.....	36
3.3.2 定位坐标转换.....	37
3.4 赛道地图建立.....	39
3.4.1 锥桶全局坐标转换.....	39
3.4.2 数据同步与边界扩展.....	42
3.5 本章小结.....	43
第四章 高速循迹赛道的轨迹规划算法	44
4.1 基于三次样条插值的路径平滑	44
4.1.1 获取赛道中心线.....	44

4.1.2 基于三次样条插值的平滑算法.....	46
4.1.3 道路中心线与参考线平滑.....	48
4.2 Lattice 动态轨迹规划算法.....	49
4.2.1 Frenet 坐标与直角坐标转换.....	49
4.2.2 轨迹规划初始状态求解.....	52
4.3 横纵向轨迹规划.....	54
4.3.1 横向采样轨迹规划.....	54
4.3.2 纵向采样轨迹规划.....	56
4.3.3 筛选最优轨迹.....	60
4.4 轨迹规划算法仿真验证.....	61
4.4.1 赛道中心线求解算法仿真验证.....	61
4.4.2 Lattice 轨迹规划算法仿真验证.....	62
4.5 本章小结.....	68
第五章 实车数据分析.....	69
5.1 无人方程式赛车实验平台.....	69
5.1.1 整车控制系统.....	70
5.1.2 感知规划算法软件设计.....	70
5.2 赛车轨迹规划实车验证.....	73
5.3 本章小结.....	76
总结与展望.....	77
总结.....	77
展望.....	78
参考文献.....	79

第一章 绪论

1.1 研究背景及意义

随着国家科学技术和经济水平的发展,我国道路基础设施建设不断完善,人们对于交通出行的需求越来越大,促使汽车保有量稳步提高,到 2022 年我国机动车保有量已达到 4.08 亿辆,同时随着城镇化建设的推进,城市道路交通日渐拥堵,交通事故频发^[1]。无人驾驶技术可以更加高效地利用道路空间,这有效解决交通堵塞问题,并减少污染排,同时降低人类驾驶所产生的误操作、减少疲劳驾驶,避免交通事故的发生。此外,无人驾驶技术的应用场景十分广泛,不仅可以应用于结构化道路交通,也可以应用于如图 1.1 所示的物流配送、道路临时作业等领域。



图 1.1 其他场景的无人驾驶应用

目前,国内外各大企业和科研机构纷纷投入无人驾驶技术的研发中,如特斯拉,Waymo, cruise, 百度, 小马智行, 新石器, 驭势科技等。除了公司、企业、研究机构的研发,高校的人才培养也是无人技术发展的重中之重。



图 1.2 2022 届 FSAC 大赛

图 1.2 为由中国汽车工程学会举办的中国大学生无人驾驶方程式大赛 (FSAC)。FSAC 赛事为学生提供了广阔平台来研究和探索无人驾驶技术[2]。大赛要求各大高校车队构想、设计、制造、开发一辆小型无人驾驶方程式赛车参加比赛,旨在能够让大学生

近距离接触新能源汽车技术与智能汽车技术的发展，为无人驾驶产业培养高质量人才。大赛既可以将先进的技术应用到方程式赛车，锻炼学生的实践能力，检验学生的科研成果；又为无人驾驶技术的研究和落地应用提供一定的借鉴。因此本文将以 FSAC 大赛为平台，重点研究基于双目相机的目标检测与定位算法，实现对赛道的实时检测和建图定位，并根据该赛道地图进行轨迹规划。本文对感知规划算法的探索以及工程落地具有积极的借鉴意义。由于赛车行驶在封闭道路，因此本文的感知和规划算法可以迁移应用于诸如机场行李托运，景区观光，工厂搬运等场景，具有一定工程应用价值。

1.2 国内外研究现状

环境的感知是赛车实现无人驾驶的基础，FSAC 比赛的赛道由锥桶搭建而成，所以环境感知任务主要有两个：首先利用目标检测技术识别锥桶，其次利用双目视觉原理定位锥桶的空间坐标，实现赛道地图的建立。建立赛道地图后利用建图时赛车的行驶路径进行轨迹规划，实现赛车在赛道内的无人驾驶。

1.2.1 目标检测研究现状

无人方程式比赛的赛道由不同颜色的锥桶搭建，因此目标检测任务就是要快速准确识别多个锥桶并在图像上定位目标位置。目标的识别和定位是计算机视觉四大基本任务之一。国内外有许多研究人员已经研究出多种技术解决这一问题，大致可以分为传统的目标检测算法和基于深度学习的目标检测算法。

传统的目标检测算法使用不同尺寸的候选框在图像上滑动来确定感兴趣区域，然后设计特征提取算法提取图像的颜色、形状、梯度等特征^[3]，如：Harr 特征法^[4]，HOG 特征法^[5]，SIFT 特征法^[6]。然后利用分类器对特征进行分类，实现目标检测。但传统方法依赖手工特征设计，图像处理步骤复杂，可移植性和环境适应能力差。

近年来基于深度学习的目标检测算法发展迅猛，2012 年 Krizhevsky 提出 AlexNet 模型使用 CNN(Convolutional Neural Networks,卷积神经网络)提取图像特征，将图像的识别、定位、检测融为一体，在图像分类问题上取得优异成绩。自此 CNN 迅速成为图像处理的研究热点^[7]，2014 年 Girshick^[8]等提出首个两阶段的目标检测算法 R-CNN，R-CNN 利用选择性搜索算法在图像上提取 2000 多个候选框 (Region Proposals)，然后将

候选框送入卷积神经网络提取特征，并利用 SVM 支持向量机进行图像分类，最后利用边框回归和非极大值抑制算法生成最终预测框。R-CNN 算法相比传统算法，大幅提高了平均精度，但存在大量的冗余计算，检测速度很慢。为此 He 等^[9]于次年提出 SPP-Net 对 R-CNN 进行改进，该网络遵循 R-CNN 的特征提取步骤，引入空间金字塔池化(SPP)降低卷积的重复运算，提高特征提取速度，但是网络仅对全连接层进行微调，忽略其他层的特征，因此精度有所下降。同年 Girshick 等^[10]提出 Fast R-CNN 使用 VGG16 作为特征提取网络，融合 SPP 使各感兴趣区域(ROI,region of interest)保持相同尺寸，提升了图像处理速度。但是 Fast R-CNN 依旧采用选择性搜索算法，整体检测速度依旧较慢。Ren 等^[11]提出 Faster R-CNN 利用 CNN 来寻找候选区域提高计算速度，但是整个处理过程依然存在冗余计算，同时 ROI Pooling 会降低目标定位精度。为此 Lin 等^[12]提出特征金字塔网络 FPN(Feature Pyramid Networks)利用连续上采样操作将不同层的特征进行融合得到新的多尺度特征图，这使得提取的特征既包含高层语义信息，又包含低层的定位信息，同时提高了检测和定位精度。虽然 FPN 结构的引入提高二阶段网络的检测精度，但其检测思路仍然是生成候选框再进行检测的为二阶段处理思想，网络检测速度一般以秒为单位，难以满足实时检测要求。因此一阶段的检测网络应运而生。

YOLOv1^[13] (You Only Look Once)是首个一阶段的目标检测网络，它将目标检测任务转换为回归问题^[14]，该算法将输入的图片整体划分为 $s \times s$ (如:7x7)的网格，每一个格子预测 B 个边框，物体中心所在网格负责对该目标进行检测，因此多目标识别任务表现良好^[15]。但是该算法采用的网格划分的方式不利于密集目标和中小目标的检测。SSD^[16]算法同属一阶段检测算法，该算法设计多个不同的检测分支来检测多尺度目标，因此小目标的检测精度较好。YOLOv2^[17]使用 DarkNet19 作为特征提取网络，引入 BN 操作^[18]增强特征提取能力，提高网络的召回率和定位精度，但是当检测物体密集分布检测效果仍然不好。YOLOv3^[19]在 DarkNet53 网络中引入残差模块^[20]和 FPN 结构，融合不同维度特征，提升了网络的小目标检测效果，实现精度和速度的平衡。2020 年 Alexey Bochkovskiy 提出 YOLOv4^[21]，该算法利用 CSP 网络结构增强网络特征提取能力，引入 SPP 池化增大网络感受野，同时为 FPN 添加 PAN 结构^[22]进一步提升网络的跨层特征融合能力。随后 UltralyticsLLC 推出 YOLOv5 版本使用数据增强^[23]、自适应锚框^[24]、Focus 切片等操作进一步提升网络精度和速度。Zheng Ge 等提出 YOLOX^[25]实现无锚框检测，添加解耦

操作实现高性能的目标检测。Chuyi Li 等^[26]提出的 YOLOv6 引入更适合 GPU 设备的 RepVGG 结构^[27], 该算法在小型网络中表征能力强, 但随着模型参数的增加, 计算量会急剧上升。Chien-Yao Wang 等提出 YOLOv7^[28], 该网络对模型重参数化和标签分配问题做出改进, 并提出高效的特征聚合网络 ELAN 解决大型网络参数利用率不足等问题, YOLOv7 是目前表现最好的目标检测模型, 它通过增加训练成本的方式提升网络的检测精度, 利用模型重参数化减少网络参数量, 提高参数的利用率, 减少推理时间, 实现对目标高效快速的检测^[29]。

YOLOv7 具备优异的实时检测性能和检测精度, 满足无人方程式赛车的道路感知的实时性需求。因此本文利用 YOLOv7 训练锥桶检测模型, 实现赛车对道路的感知。

1.2.2 基于视觉的障碍物定位研究现状

要完成赛道感知任务需要获取锥桶在三维空间中的位置, 该问题可以归结为障碍物定位问题, 目前自动驾驶领域的障碍物定位方案可分为两类, 一是以雷达为主的定位方案, 二是以视觉为主的定位方案。

以雷达为主的定位方案通常使用激光雷达和毫米波雷达实现障碍物定位, 该方案精度好, 能检测 300m 范围的物体, 但是该方案价格昂贵, 环境中其他物体的点云信息会造成较大干扰, 同时算法难度高, 对于赛车锥桶道路的感知任务来说不是最佳方案。基于视觉的障碍物感知成本低廉, 并且在短距离的精度高, 实时性好, 而且视觉方案可获得丰富的环境信息, 满足锥桶赛道的感知任务要求。如今基于视觉的障碍物检测方案已被广泛应用。

德国奔驰汽车公司于 1990 年开发的“VAMORS-P”无人车^[30]以双目相机、加速计等设备作为道路采集传感器, 实现车辆的无人驾驶。随后国外各大公司和大学争相展开的无人车研究。其中“Navlab-V”的智能车^[31]、“Sandstorm”无人车^[32]、“Stanley”多功能车^[33]、“Boss”无人车^[34]均采用多种视觉传感器实现无人驾驶^[35]。国内红旗“CA7460”采用双目相机感知环境, 完成障碍物距离检测, 于 2011 年实现无人驾驶^[36]。此后各大汽车公司开发的无人车均搭配有摄像头感知方案, 其中特斯拉以摄像头为主要感知方案实现无人驾驶^[37]。美国波士顿公司基于立体视觉研发出了四足仿生机器人“Sot”能够执行各种复杂化境下的任务^[38]。

在锥桶的感知定位方面，HUANG Yong 等^[39]提出融合雷达和视觉传感器的交通锥桶检测方法。王振等^[40]利用单目成像原理定位障碍物并测距。毛先胤等^[41]使用双目相机作为传感器，采用半全局立体匹配算法获取深度信息，实现障碍物的定位。施友宁^[42]利用单目相机 PnP 算法，构建大量锥桶角点数据库实现对锥桶定位。对于基于视觉障碍物感知来说，单目视觉通常需要复杂的标定工作，且需要建立检测目标的数据库，因此移植性差。针对本文的赛道感知任务，选取双目视觉的方案来获取锥桶边界的世界坐标。

为了实现双目定位需要通过立体匹配算法获取深度，双目视觉获得深度的方法主要有传统的立体匹配算法和基于深度学习的立体匹配算法。传统立体匹配算法分为局部立体匹配，全局立体匹配^[43]和半全局立体匹配^[44]。其中局部立体匹配算法利用待匹配点邻域范围内的像素计算视差，该算法匹配速度快，但是误差较大，常见的算法有单步多框检测算法^[45]、归一化互相关算法^[46]等。全局立体匹配算法利用全局最优思想来最小化全局能量函数求解最优视差，其匹配精度高，但算法实时性差，常见算法有图割法^[47]、置信传播法^[48]、动态规划法^[49]等。半全局立体匹配算法 SGM^[50]将二维图像优化问题转换为一维优化，简化匹配难度在精度和速度间取得平衡，具有良好的实时性，目前已被广泛使用^[51]。基于深度学习的立体匹配算法有 MC-CNN^[52]，EdgeStereo^[53]，GCNet^[54]，GANet^[55]，PSMNet^[56]等，但目前基于深度学习的方法计算量大，对 GPU 设备要求高，实时性差，而且目前双目算法的数据集较少，使得网络泛化能力和迁移能力较差。因此本文将采用稳定的 SGM 算法生成锥桶赛道深度图，结合目标检测结果实现锥桶在三维空间中的定位。

1.2.3 轨迹规划研究现状

赛车无人驾驶的轨迹规划算法负责分析周围环境、确定安全可行的行驶路径，并实时调整车辆的运动状态，以保证赛车在复杂场景下安全行驶。无人驾驶汽车的轨迹规划一般分为全局路径规划和局部轨迹规划。全局路径规划对整个运动场景建模，以车辆运动学和环境约束为前提，在预设的起点和终点之间，生成一条能够安全、高效到达终点的路径，该路径仅包含车辆在场景中运动的几何信息。而局部轨迹规划一般需要一条全局路径作为参考线，结合车辆的实时状态和运动学、动力学约束，规划一条包含车辆各

个时刻位置、车速、加速度等信息的时间序列点集^[57]。常见的路径规划算法包括基于图搜索的方法、人工势场法、机器学习方法，基于采样和插值方法等^[58]。

基于图搜索的方法将车辆所处的场景划分为栅格图，并利用各种图搜索算法寻找从起点到终点的最优路径。N Tran 等人^[59]将视图分割为若干空间块，在确定空间块的起始点后，利用 Dijkstra 算法寻找起始点间的最短路径。其中 Dijkstra 算法^[60]采用广度优先搜索思想，以起点为中心向终点层层扩展得到最优路径，该算法搜索范围广，随着地图增大，路径规划的速度和效率低。Li Wenzheng 等人将节点 8 邻域访问次数引入 Dijkstra 算法，提高了算法搜索效率^[61]。Peter Hart 等^[62]对 Dijkstra 算法进行扩展，开发了 A*算法，A*算法利用启发函数引导规划器从起点向终点搜索，提高搜索效率且鲁棒性更强。Maxim Likhachev 等^[63]考虑路径规划的时间限制，提出 ARA*算法，该算法根据可用时间调整边界性能，在有限时间内寻找次优解，提高规划的实时性。Harabor D 等人^[64]将跳点搜索思想应用于 A*算法，优化后继节点的搜索方式，提高算法搜索效率。黄健萌等人^[65]改进跳点搜索方式，结合多项式拟合算法使得规划路径更加平滑。Anthony Stentz^[66]提出 D*算法解决车辆在未知地形行驶时的路径规划问题。目前 A*已广泛应用于路径规划，但是针对实际应用场景，许多学者也作出了改进。张煜等^[67]结合模糊神经网络设计 A*VFF，使车辆在复杂环境下也能规划优质的轨迹。王维等^[68]为当前和过去节点的搜索路径赋予权重，使搜索能快速接近终点，在接近终点时进行精细搜索，提高了 A*路径规划的实时性。Liu 等^[69]根据物流仓库、车间的环境特点提出 DFPA 算法，算法利用 Delaunay 三角剖分处理环境中杂乱分布的障碍物，生成寻路参考节点，并对环境进行栅格化，在提取障碍物信息后，利用改进 A*算法，根据参考节点为移动机器人提供避障路径。总的来说基于图搜索的算法需对车辆的行驶环境进行栅格化建模，然后利用搜索算法搜索始末点间的最优路径，该方法需要完整环境信息，而比赛时不能提前进入赛场，因此该算法不满足本文需求。

人工势场法 APF 假设车辆处于一个虚拟的力场下，为始末点和障碍物赋予不同势场，来引导车辆从起点驶向终点^[70]。这种方法往往会使路径规划陷入局部最优，为此罗乾又等^[71]为势场中的车辆额外添加虚拟目标位置来调节车辆受到的引力，从而避免局部最优解。LiX、Wang Z 等人^[72]应用决策树思想改进人工势场法，避免规划路径陷入局部最优，同时有效解决算法在凹形障碍物间震荡的问题。韩伟等^[73]人提出基于模糊人工势

场法的路径规划算法,利用专家经验调整移动机器人各个时刻的合力,解决机器人在复杂障碍物环境行驶震荡的问题。Zhang J 等人^[74]为了提高人工势场法规划效率,引入 BP 神经网络动态调整势场函数的增益系数。虽然改进人工势场法具有良好的避障能力,但并没考虑车辆的动力学特性,因此该方法并不适用于方程式赛车。

基于机器学习的方法需要大量现有的轨迹规划样本,通过强化学习、增强学习等学习方法寻找输入输出间的映射关系,根据任务需求、行驶环境、车辆状态来映射一条适合车辆的规划轨迹。刘珏^[75]通过对车辆的运动状态进行分析,利用马尔可夫决策算法对车辆建模,使用逆强化学习的方法训练演示轨迹样本,通过学习到的报酬函数生成最优路径。项宏峰等^[76]对神经网络算法进行改进,提出 4 层神经网络结构来规划路径,利用能量函数作为评价函数求解最优路径,解决车辆的局部路径规划问题。基于机器学习的方法相比传统方法更具灵活,但是需要大量的训练样本,这难以在比赛中应用,因此这种方法不适用赛车的路径规划。

基于采样和插值的方法是对无人驾驶车辆的行驶状态空间撒点采样,使得每个采样点均包含车辆的行驶状态,包括位置、速度、加速度等信息。完成采样后连接采样点得到两种状态间的路径轨迹信息。但是为了保证轨迹平滑,一般采用回旋曲线、多项式插值、贝塞尔曲线、高次样条插值等方法连接。根据采样方式的不同可分为随机采样和确定性采样两种。随机采样是在车辆状态空间中随机撒点采样,其中快速随机搜索(rapidly-exploring random tree, RRT)算法^[77]为常用的随机搜索算法,RRT 算法可以快速在半结构化的行驶空间中搜索,但随机采样是在空间中随机撒点,导致 RRT 算法有较多的冗余搜索,同时搜索的路径曲率变化率一般较大。因此宋晓琳等^[78]提出改进的 RRT 算法,在周围空间中采用高斯分布生成采样点,利用 3 次 B 样条曲线对规划路径进行平滑,限制路径的曲率变化率。张兰勇等^[79]提出改进的 RRT*算法在采样时加入偏置函数,并在采样点间用 Dubins 曲线平滑连接,算法还设计了代价函数来评估曲线长度和避障性能,根据代价值选择最优路径。符强等^[80]提出双向目标 RRT 算法,利用目标约束采样和偏执扩展策略引导算法向目标搜索,然后利用 Dijkstra 算法去除可行路径中的冗余点,最后对路径进行平滑,求解最优路径,提高节点利用效率,减少搜索时间。RRT 算法主要用于复杂环境的路径规划,随机搜索采样点会导致规划结果有差异,不适用于赛车固定赛道情况。

确定性采样方法按照事先给定的规则,对车辆行驶空间进行采样,从而获得多个采样点,然后利用曲线插值法对采样点进行平滑得到规划轨迹。确定性采样适用于具有长期连续的轨迹规划情况,一般在车辆的控制空间和状态空间进行采样。控制空间采样是根据车辆运动学模型求解车辆的位置、速度和横摆角,然后对这些控制状态进行采样得到满足车辆性能约束的采样点,然后向前模拟得到车辆的轨迹,这种方法忽略了参考线信息,增加无效采样,轨迹对环境的适应能力较弱。状态空间采样在车辆的前进空间均匀采样,根据起点和目标点的状态求解两个状态间的轨迹,利用代价函数筛选最优轨迹,这种方法以参考线作为状态采样依据进行轨迹规划,更适用结构化道路环境。魏民祥等^[81]提出 Frenet 坐标下采样的轨迹规划算法,使用多项式曲线拟合采样点,生成多条规划轨迹簇,利用代价函数筛选出最优轨迹,经过碰撞检测后输出满足车辆结构化道路的规划轨迹。McNaughton 等^[82]人对道路末状态进行采样,利用高阶多项式求解轨迹,该轨迹满足道路几何特征,具备良好的可执行性。采样点之间的轨迹一般使用多项式插值来描述,一般采用三次多项式^[83]和五次多项式^[84],其中五次多项式描述的轨迹加速度导数连续,生成的轨迹更为平滑,因此使用较广。

赛车在锥桶搭建的道路行驶,属于长时间连续的半结构化道路,因此使用状态空间采样的方法契合轨迹规划需求,由于该方法需要参考线,因此本文首先使用三次样条插值对赛车的行驶路径进行插值求解全局参考线。然后利用全局参考线进行轨迹规划。为了保证规划轨迹能使赛车平滑,稳定行驶,因此采用五次多项式作为规划函数连接采样点生成规划轨迹。由于在 Frenet 坐标系下对于车辆的状态描述更简洁,因此本文采用 Frenet 坐标系下的动态采样轨迹规划方法进行规划赛车行驶轨迹。

1.3 主要研究内容

本文根据无人方程式大赛规则,以高速循迹测试项目为对象,研究无人方程式赛车对锥桶赛道的感知和建图算法,以及高速循迹赛道的轨迹规划算法,本文章节安排如下:

第一章:绪论。首先阐述本文无人驾驶的研究背景和课题研究的实际工程意义。接着分析道路感知中的目标检测、双目立体匹配以及轨迹规划的国内外研究现状。最后说明本文的主要研究内容,章节安排和技术路线。

第二章：基于 YOLOv7 的锥桶识别。首先采集大量的锥桶数据集进行标注，利用数据集训练基于 YOLOv7 的目标检测模型。然后验模型的可靠性，并利用双目相机进行实时目标检测，对输出的锥桶检测结果进行排序，输出锥桶的像素坐标。

第三章：基于双目视觉的赛道定位与建图。首先标定双目相机，获取相机参数。然后利用双目相机的成像原理、SGM 立体匹配算法，结合相机标定参数求解锥桶的世界坐标。最后对组合惯导进行标定，在获取赛车的定位坐标后融合锥桶世界坐标构建赛道地图。

第四章：高速循迹道路的轨迹规划算法。利用 Delaunay 三角剖分算法求解赛道中点，利用三次样条插值计算赛道中心线。然后，用三次样条插值算法平滑赛车建图时的行驶轨迹，求解全局参考线，结合用赛车的定位信息，在 Frenet 坐标系下完成高速循迹赛道的轨迹规划。轨迹规划算法首先对赛车末状态进行采样，利用始末状态求解赛车最优规划轨迹。最后在 matlab 环境做仿真实验，验证道路中心线生成算法和轨迹规划算法的可行性。

第五章：实车验证。首先搭建赛车实验平台，阐述赛车的硬件架构和整车控制系统，并在 ROS 环境下进行感知规划算法的软件设计。然后搭建锥桶赛道进行实车实验，验证本文赛道感知算法和高速循迹的轨迹规划算法的可行性。

总结与展望：对本文现阶段所作的研究工作进行总结，指出当前研究存在的不足，并对本文研究的不足和未来需继续开展的工作进行展望。

本文技术路线如图 1.3 所示：

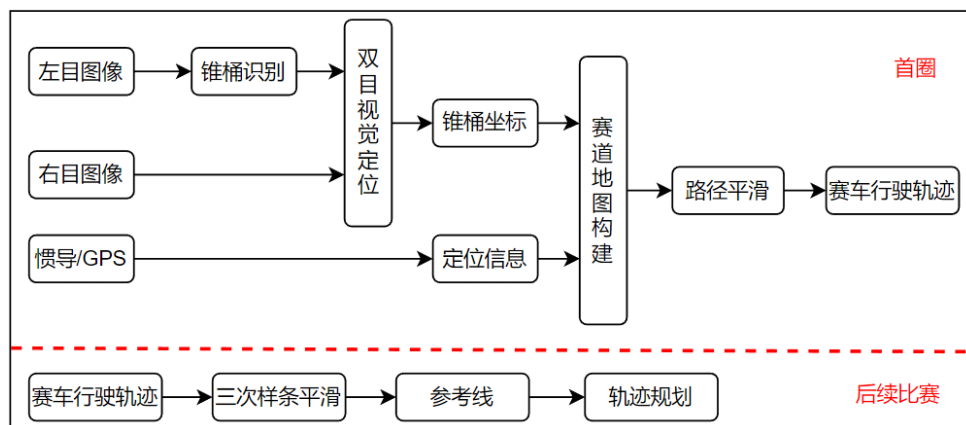


图 1.3 技术路线

第二章 基于 YOLOv7 的锥桶识别

赛车实现无人驾驶的首要任务是实现对赛道的感知，比赛赛道由不同颜色的锥桶按照设计路线构建，因此需要精度高、实时性好和鲁棒性强的目标检测算法检测锥桶。YOLOv7 算法无论在目标检测的精度、速度都优于其他目标检测算法。因此本章分析 YOLOv7 的算法原理，并收集锥桶数据集，训练锥桶检测模型，完成对锥桶赛道的识别与定位。

2.1 目标检测任务分析

无人方程式大赛的动态赛分为直线加速测试，八字绕环测试和高速循迹测试，直线加速和八字绕环并非考验赛车的无人驾驶性能，所以本文主要研究高速循迹测试项目的感知与规划算法。无人方程式比赛规则规定赛车在开赛前不能进入比赛道路测试，正式比赛时，赛车进入赛道后全程不得有人驾驶，但不限制赛车在动态赛项目道路上的跑动圈数，因此本文采取的策略为首圈低速行驶，在感知赛道的同时建立实时地图，并在首圈采集赛车的行驶路径，然后根据该路径计算全局参考线，后续比赛利用参考线循迹行驶。

大赛所有动态赛项目赛道均由三种颜色的锥桶构建，赛道左侧为红色锥桶，右侧为蓝色锥桶，停止点用黄色锥桶标识。图 2.1 中为本文搭建的实验锥桶赛道。



图 2.1 不同道路的锥桶采集

图 2.1 中黄色方框标记的锥桶最小像素仅 10×18 像素，而目前 MS COCO 数据集将分辨率小于 32×32 像素的目标定义为小目标，因此锥桶的检测网络需具备良好的小目标检测性能。同时实际比赛时道路环境复杂，锥桶检测会受到遮挡、阴影、光照等多种外部因素的影响。为了保证赛车对赛道建图的可靠性，目标检测算法需要在复杂环境中快速且准确检测图像上锥桶，避免错检和漏检。目前基于视觉的目标检测方法主要分为传统的计算机视觉法和基于深度学习的目标检测法。

传统计算机视觉法步骤为：图像预处理、特征提取、目标检测和后处理。在图像预处理阶段的滤波降噪、二值化处理等操作会损失图像信息。在复杂环境中采用形态学处理方法，需要根据环境信息设定阈值以便提取目标特征，但是环境干扰项很多，阈值选取难度大。后处理阶段所采用的非极大值抑制和边框位置回归等方法会过滤远处锥桶，导致漏检和误检。因此传统计算机视觉的方法无法满足本文要求。

基于深度学习的目标检测主要分为两种：二阶段的目标检测网络和一阶段目标检测。二阶段目标检测会生成大量的候选框，处理速度一般以秒为单位，难以满足赛道建图实时性要求。而一阶段目标检测算法能兼顾目标检测精度和速度，在无人驾驶的感知任务中广泛应用，目前 YOLOv7 无论在检测精度和速度上都达到最优，而且大量采用 3×3 的卷积进行特征提取，小目标识别精度高，因此本文选用 YOLOv7 作为锥桶检测网络。

2.2 YOLOv7 算法原理

YOLOv7 整体网络结构如图 2.2 所示，网络由输入端(input)、骨干网络(Backbone)、检测头(head)组成。YOLOv7 在输入端采用 Mosaic 数据增强和自适应图片缩放来预处理输入数据集。然后使用 CBS 模块、ELAN 模块、MP 模块组成的骨干网络提取特征，输出三种尺度的特征图并送入检测头进行特征融合与检测。检测头由 SPPCSP 模块，ELAN-W 模块，CBS 模块组成，同时利用 PAFAN 结构融合高层、低层网络的特征，最后输出被检测物体的类别和在图像上的坐标位置。

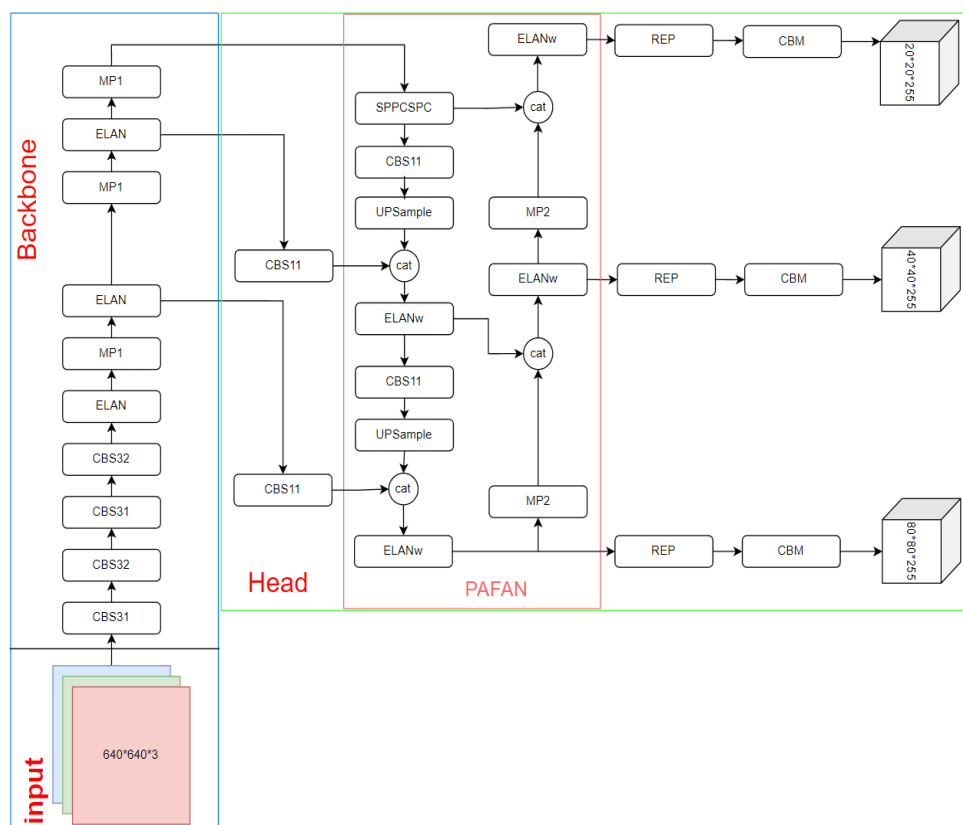


图 2.2 YOLOv7 整体网络架构

2.2.1 骨干网络 (Backbone)

YOLOv7 输入图像经过数据增强后, 将 $640 \times 640 \times 3$ ($u \times v \times c$, u 为图像宽度, v 为图像高度, c 为图像通道数) 的图像输入骨干网络提取特征。骨干网络设计了多种计算块以加快网络的特征提取速度, 增强网络的扩展性和鲁棒性。

1、CBS 计算块

不同大小的卷积核在进行卷积运算时的参数量和感受野不同, 且计算速度差别很大, 例如: 用 1 个 5×5 的卷积核进行卷积操作得到 $25c^2 + c$ 个参数, 但是用 2 个 3×3 的卷积核的参数仅 $9c^2 + c$ 。而两个 3×3 的卷积核具有与 5×5 的卷积核相同大小的感受野, 并且英伟达显卡对 3×3 的卷积核的优化最好, 因此为了加快训练和推理速度、减少计算量, YOLOv7 大量采用 3×3 卷积核进行卷积运算。YOLOv7 还使用 1×1 的卷积核对特征通道进行升维降维操作, 增强网络提取跨层特征的能力。虽然小尺寸的卷积核参数少, 但是随着网络层数增加, 会出现梯度消失现象, 因此网络将卷积运算、BN 运算^[18], 激活函数合并成一个 CBS 计算块, 既防止模型梯度消失, 又增强模型的非线性, 提高模型的特征提取能力。CBS 计算块根据不同的步长(s)和卷积核(k)大小分为三种, 分别为 $k=1$,

$s=1$ 的 CBS11, $k=3$, $s=1$ 的 CBS31, $k=3$, $s=2$ 的 CBS32。三种计算块如图 2.3 所示:

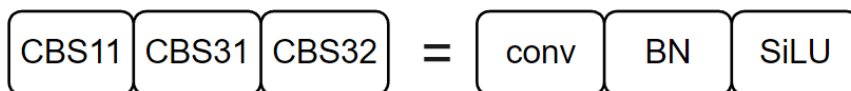


图 2.3 三种 CBS 计算块

模型使用的 CBM 计算块将 CBS 中的 SiLU 激活函数换为了 sigmoid 函数, CBM 结构如图 2.4 所示:

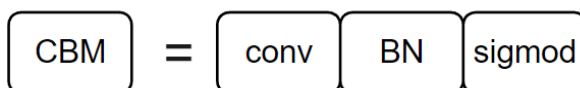


图 2.4 CBM 计算块结构

2、MP 结构

由于 3×3 卷积操作感受野较小, 主要关注局部信息, 因此 YOLOv7 引入了 MP (最大池化) 结构以扩大感受野, 其结构如图 2.5 所示。然而, MP 结构可能会丢失部分图像特征。为解决这一问题, MP 结构采用分组卷积^[85]的方法, 结合 CBS 模块提取特征, 在扩大感受野的同时减少特征损失。

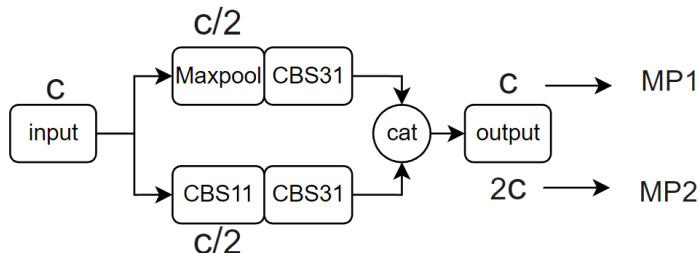


图 2.5 MP 模块结构

若 MP 结构的输入通道与输出通道保持一致则为 MP1, 若输出通道是输入通道的两倍则为 MP2。

3、高效层聚合网络(ELAN)

ELAN 模块由多个 CBS11, CBS31 计算块堆叠而成, ELAN 结构如图 2.6 所示:

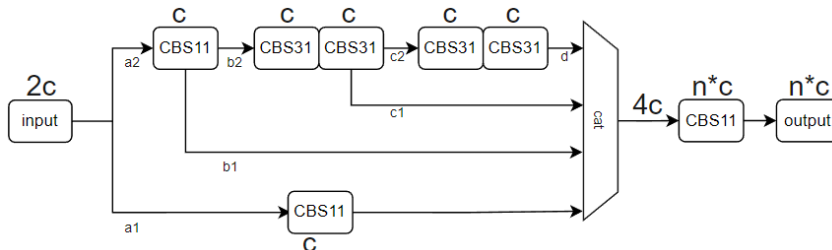


图 2.6 ELAN 模块结构

ELAN 采用分组卷积思想, 将输入特征图分为 a_1 , a_2 两组, 分别与 CBS11 进行卷积, 获得两个相当于输入通道一半(c)的输出特征图, 其中 a_1 用于跨层特征融合。 a_2 则经 CBS11 卷积后得到 b_1 , b_2 两组特征图, 其中 b_1 用于跨层特征融合, b_2 继续与两个 CBS31 卷积得到 c_1 , c_2 两组特征图, 其中 c_1 用于跨层特征融合, c_2 与两个 CBS31 卷积, 得到特征图 d , 最后使用 shuffle 操作打乱 a_1 , b_1 , c_1 , d 顺序后使用 caoncat 融合各层特征, 最终得到 2 倍于输入通道的输出特征图, 再利用 CBS11 调整期望的输出通道。

ELAN 使用分组卷积来扩展计算块的通道和数量, 而各分支的计算块具有相同的分组参数和通道数, 使得网络具备输入、输出通道相等的优势, 可以在不破坏原有梯度路径的情况下, 增强网络的学习能力。并且 ELAN 网络结构按照 a_1 、 a_2 、 b_1 、 b_2 、 c_1 、 c_2 、 d 分支对网络进行扩展, 这样既能加入更多数量的计算块, 又保持了原有的分支结构, 使得基于级联的网络模型在进行深度缩放时, 可以生成各种宽度、深度和分辨率的网络, 以满足不同任务需求。经 ELAN 扩展的网络在结构上“大而整”, 不仅增强网络特征提取能力, 还增强网络的鲁棒性。

图像经过骨干网络的多个 ELAN, MP 模块提取特征后, 输出 20×20 , 40×40 和 80×80 三个维度的特征图, 并传入检测头 head 进行特征融合与目标检测。

2.2.2 检测头 (head)

head 采用 PAFPN 架构融合各维度的特征, 使用 REP 模块将融合后的特征重参数化后输出 20×20 、 40×40 和 80×80 三个尺度的特征图用于目标检测。模型在 FPN 阶段使用 SPPCSPC 模块提取 20×20 的特征图信息, 经上采样后与 40×40 特征图融合, 并利用 ELAN-W 结构提取融合特征, 再通过相同的上采样操作与 80×80 的特征图融合, 得到融合深层语义特征的 40×40 和 80×80 的特征图; 在 PA 阶段使用 MP 结构对 80×80 和 40×40 的特征图进行下采样, 然后分别与 40×40 和 20×20 的特征图融合, 得到融合浅层定位信息的特征图。最后经重参数化模块和 CBM 后, 输出 $20 \times 20 \times 255$, $40 \times 40 \times 255$, $80 \times 80 \times 255$ 的目标检测结果。检测头中的计算块详解结构如下:

(一) SPPCSPC

SPPCSPC 结构如图 2.7 所示:

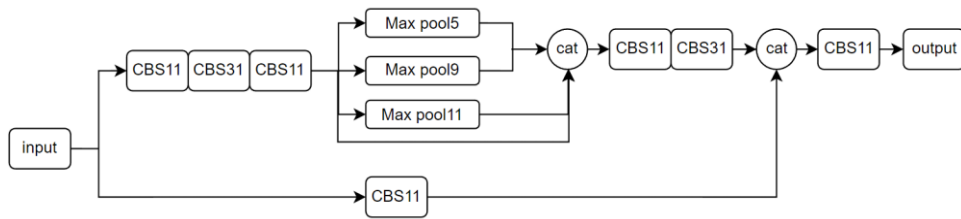


图 2.7 SPPCSPC 模块结构

SPPCSPC 可以有效避免图像区域裁剪、缩放操作导致的图像失真，提高高层特征图识别精度，同时减少网络对特征的重复提取，提高候选框产生的速度。

（二）Rep 模块

Rep 模块在训练时由三个分支的卷积和 BN 模块组成，如图 2.8 所示，但在推理阶段将训练时三个分支的参数重参数化为一个分支的卷积和 BN 计算。模型经过重参数化后，既保留训练的参数特征，又减少了推理时模型的参数量，使模型在保证精度的情况下有效提升目标检测速度。

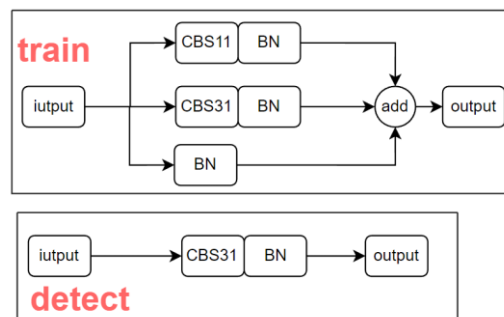


图 2.8 Rep 模块结构

（三）动态标签分配策略

模型输出的特征图有若干特征点，每个特征点会生成多个候选框，但是并非每个特征点都为正样本。YOLOv7 选择距离标签中心位置最近的 2 领域网格和标签中心点所在网格作为训练正样本，如图 2.9 所示。

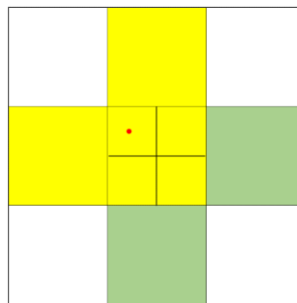


图 2.9 动态标签分配示意图

标签中心为红色点，动态标签分配算法将选择三个黄色网格作为正样本进行训练。该方法策略增加了正样本数量，提高网络的召回率，增加锥桶被检测的概率。

（四）损失函数

完成正样本分配后，需要根据标签与正样本的预测框计算 loss 值，在反向传播阶段利用 loss 值更新网络参数。YOLOv7 的损失函数 $L_{\text{object, Loss}}$ 为：

$$L_{\text{object, Loss}} = L_{\text{loc, Loss}} + L_{\text{conf, Loss}} + L_{\text{class, Loss}} \quad (2.1)$$

其中： $L_{\text{loc, Loss}}$ 为定位损失； $L_{\text{conf, Loss}}$ 为置信度损失； $L_{\text{class, Loss}}$ 为分类损失。置信度损失和分类损失均采用 BCEWithLogits Loss 函数，表达式如下：

$$\text{Loss} = \{l_1, \dots, l_N\}, l_n = -[y_n \cdot \log(\sigma(x_n)) + (1-y_n) \cdot \log(1-\sigma(x_n))] \quad (2.2)$$

其中 N 为 n 个 batch， $\sigma(x_n)$ 为 Sigmoid 函数， x_n 为模型输出的值，y 为样本标签值，y=1 为正样本，y=0 为负样本。

定位损失 L_{Clou} 计算公式为式(2.3)~式(2.5)：

$$L_{\text{Clou}} = 1 - \text{IoU} + \frac{\rho^2(b, b_{\text{gt}})}{c^2} + \alpha v \quad (2.3)$$

$$v = \frac{4}{\pi} \left(\arctan \frac{w_{\text{gt}}}{h_{\text{gt}}} - \arctan \frac{w}{h} \right)^2 \quad (2.4)$$

$$\alpha = \frac{v}{(1 - \text{IoU}) + v} \quad (2.5)$$

其中：IoU 表示预测框与真实框的交并比，反应真实框和预测框的重合度； b 表示预测框中心点， b_{gt} 表示真实框中心点， ρ 代表计算中心点间的欧式距离； c 表示能够同时包含预测框和真实框的最小闭包区域的对角线距离； α 表示平衡参数； v 为梯度， w ， h 表示预测框的宽和高， w_{gt} ， h_{gt} 表示真实框的宽和高。

2.3 数据集创建与模型验证

YOLOv7 的预训练模型是在 MS COCO 等大型数据集上训练的，这些数据集没有锥桶类别，因此为了让 YOLOv7 实现锥桶的检测任务，需要制作锥桶数据集，训练锥桶检测模型。

2.3.1 数据集创建与标注

目标检测网络对锥桶识别效果十分依赖锥桶数据集的质量和数量。因此为了保证锥桶检测效果，需要收集足够多的图像数据和标签数据。本文采集在实验条件下搭建两种光照条件和拍摄角度的赛道录制跑动视频，共得 2132 张锥桶图像，如图 2.10(a)；图 2.10(b) 包含 2020 到 2022 年的国内外大学生方程式比赛实车跑动视频的帧提取图片制作的数据集，其中雨天、晴天各 200 张，柏油路 200 张，水泥地 200 张，广场 200 张。

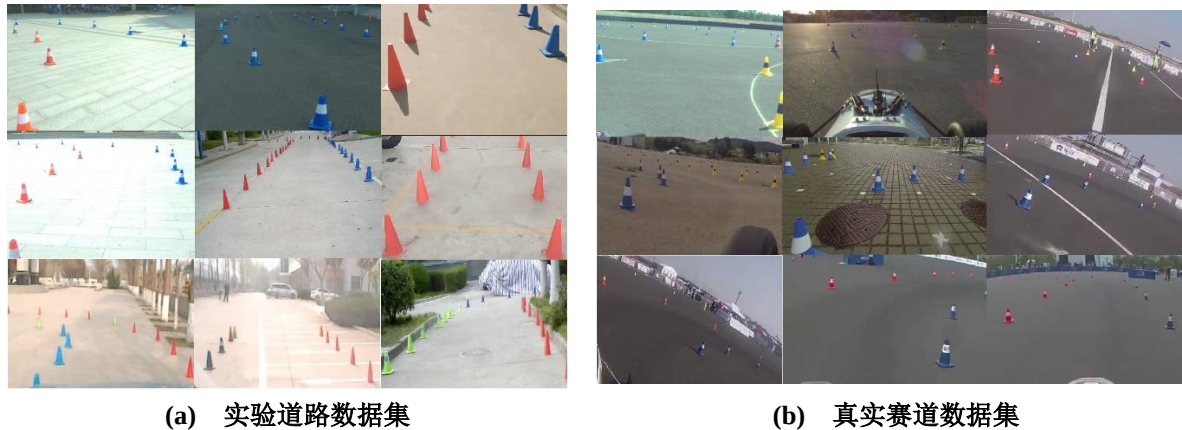


图 2.10 锥桶数据集

在获得锥桶图像数据集后，利用标注软件 labellmg 对图像进行标注，设置 yellow, blue, red 三种标签，由于黄色锥桶数量较少，所以本文使用绿色锥桶代替黄色锥桶。标注界面如图 2.11 为所示：

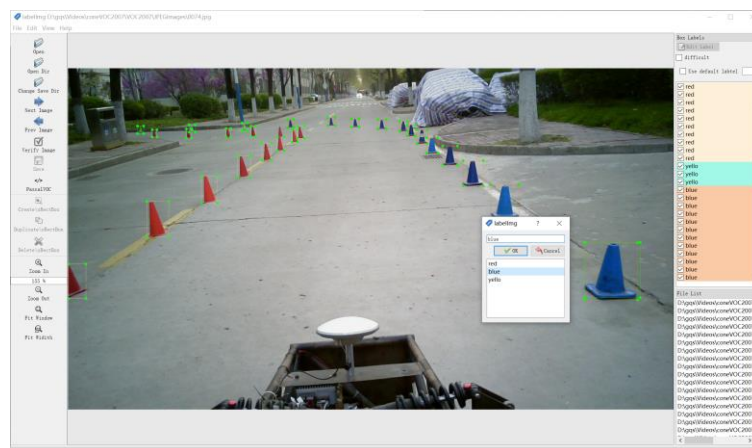


图 2.11 锥桶标注界面

由于赛道的锥桶摆放密度不同，共得 15637 个 voc 格式标注框。通 python 脚本将 voc 格式标注转换为 yolo 格式标注，两种标注格式如图 2.12 所示：

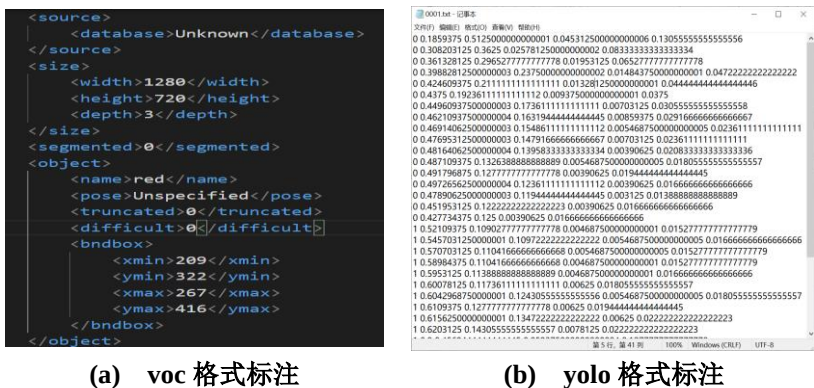


图 2.12 标注格式

2.3.2 模型训练与验证

完成数据集标注工作后，在 windows 操作系统下训练模型，电脑硬件配置如表 2.1 所示：

表 2.1 硬件配置表

操作系统	Ubuntu20.04/windows10
CPU	Gen Intel(R) Core(TM) i5-12500H (16 CPUs)@2.5GHz
GPU	NVIDIA GeForce RTX 3050
RAM	HX432C16FB3/32 DDR4 3600MHz 32GBx2
硬盘	WD40EZRZ SATA3 5400RPM 2TB

首先将锥桶数据集划分为训练集、测试集和验证集，其比例分别为8:1:1。加载标准版本的 YOLOv7 预训练模型 yolov7.pt。根据电脑硬件配置设置训练参数，其中 batch-size 为 4，训练 100 轮，训练设备为 GPU，线程数为 2，启用标签平滑，采用默认超参数配置，采用 adam 优化器。使用迁移学习的方式，将锥桶数据集送入模型进行训练，模型训练结束后选择效果最好的权重文件 best.pt 用于模型验证。图 2.13 为模型在测试集上的检测结果：



图 2.13 测试集检测结果

图 2.13 表明测试集上的锥桶均被检测出来。模型在测试集上的验证结果如表 2.2 所示，其中 P(precision)为精度，R(recall)为召回率，mAP(mean average precision)为平均精度均值，三者值越接近 1 越好。mAP@.5 和 mAP@.5:.95 分别表示 IoU 比值大于 0.5 和比值在 0.5~0.95 范围的平均精度均值。表 2.2 表明模型在验证集上的 P、R、mAP@.5 均达 0.95 以上，证明算法性能优良。

表 2.2 验证集终端输出表

Class	Images	Labels	P	R	mAP@.5	mAP@.5:0.95
all	91	1266	0.99	0.977	0.98	0.746
red	91	639	0.986	0.958	0.962	0.716
blue	91	419	0.988	0.988	0.993	0.757
yellow	91	208	0.995	0.986	0.986	0.765

训练过程中各项损失变化曲线及关键指标变化曲线如图 2.14 所示：

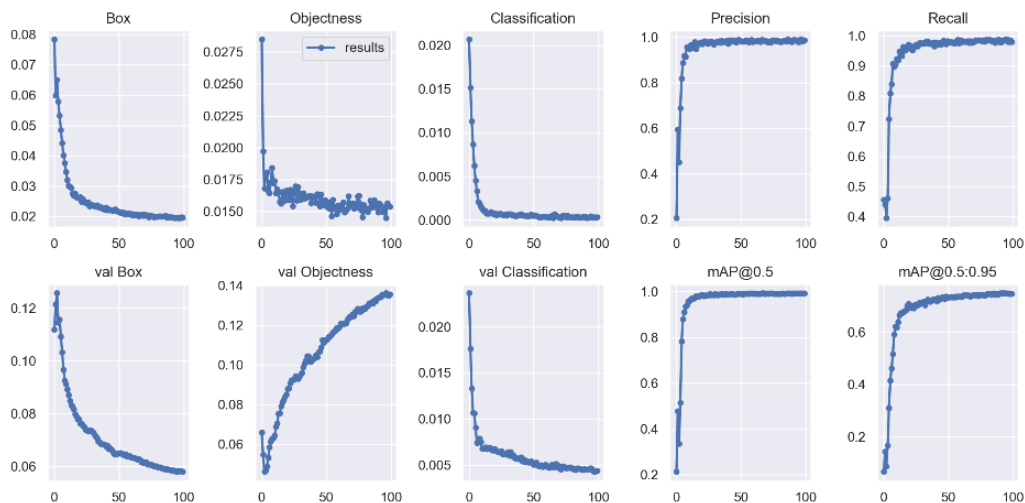


图 2.14 训练参数曲线

其中 Box 表示定位损失函数均值；Objectness 表示目标检测损失函数的均值；Classification 表示分类 loss 的均值；val Box，val Objectness，val Classificationfe 分别表示在验证集上的定位、目标检测、分类损失函数均值，mAP 为平均精度（mean average precision），其曲线围成的面积越大越好。图 2.14 表明，经过 100 轮训练后，模型各项损失均收敛，且精度和召回率均达到稳定状态，其中 mAP 达到 98% 的准确率。

2.3.3 锥桶检测与排序

在 ROS 中部署训练好的锥桶检测模型，对双目相机采集的图像进行实时检测。由于 YOLOv5 速度快于所有二阶段检测模型，在同时期目标检测算法中表现优异^[23]，因

此本文增添 YOLOv5 检测模型对锥桶的识别效果对比。并比较 YOLOv7 与 YOLOv5 检测效果，图 2.15 为 YOLOv7 左目图像的实时检测结果，图 2.16 为 YOLOv5 左目图像的实时检测结果。

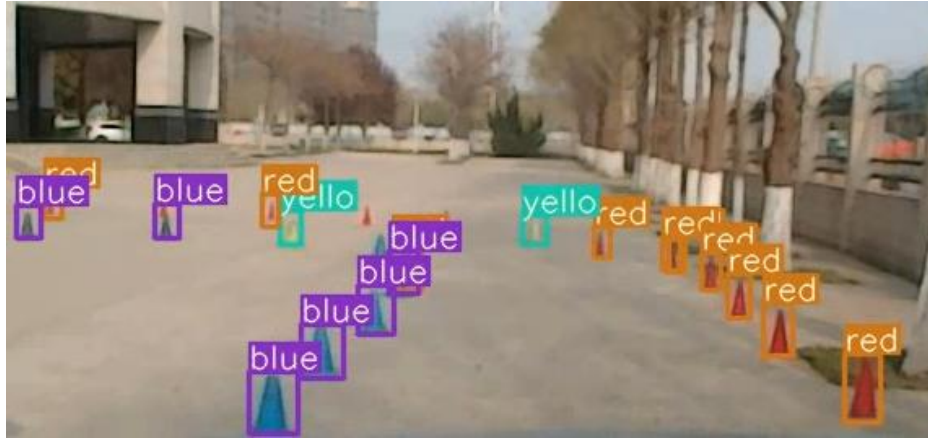


图 2.15 YOLOv7 左目实时检测结果

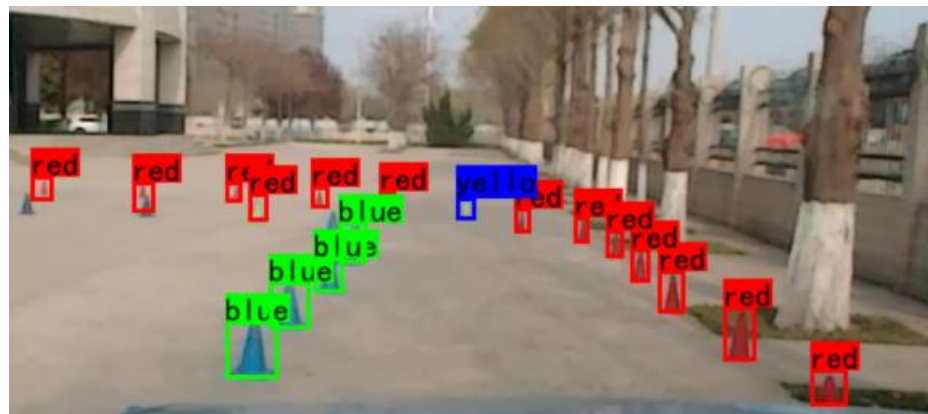


图 2.16 YOLOv5 左目实时检测结果

从图 2.15 与 2.16 的对比可知 YOLOv5 虽然检测远处锥桶效果较好，但是存在将蓝色锥桶识别为红色，黄色锥桶识别为红色的误检，而 YOLOv7 检测结果更加准确。

图 2.17 为图 2.15 中锥桶检测结果的终端输出，图 2.15、图 2.17 表明图中锥桶基本被检测出来，一张图片的检测时间仅 19ms，表明本文的锥桶检测模型目标检测效果好，且实时性好。

```
9 reds, 4 blues, 2 yellos, Done. (15.0ms) Inference, (4.0ms) NMS
[[1, 0, 974, 595], [2, 0, 905, 550], [3, 0, 821, 526], [4, 0, 674, 490],
[[1, 1, 581, 576], [2, 1, 575, 540], [3, 1, 372, 507], [4, 1, 224, 505]]
[[1, 2, 500, 514], [2, 2, 757, 505]]
```

图 2.17 实时检测坐标排序结果

由于模型是随机对图像中的锥桶进行检测，但在路径规划时需要顺序排列的锥桶坐标，因此将锥桶的检测结果按照像素坐标的 v 方向从大到小排序，图 2.15 中部分被检测锥桶坐标的排序结果如表 2.3 所示：

表 2.3 锥桶坐标检测结果排序表

锥桶	序号	类别	像素坐标 u
红色锥桶 red	1	0	(974,595)
红色锥桶 red	2	0	(905, 550)
红色锥桶 red	3	0	(821, 526)
蓝色锥桶 blue	1	1	(581, 576)
蓝色锥桶 blue	2	1	(575, 540)
黄色锥桶 yellow	1	2	(500, 514)
黄色锥桶 yellow	2	2	(757, 505)

2.4 本章小结

本章根据无人方程式大赛规则要求，选择 YOLOv7 作为本文锥桶检测算法，利用自制锥桶标注数据集训练锥桶检测模型，随后在测试集上对模型进行验证，验证结果表明模型具有良好的精度和实时性。最后在 ROS 中部署模型并实时检测双目相机采集图像中的锥桶，对检测结果排序后，输出顺序排列的锥桶像素坐标，为生成赛道建图提供感知信息。

第三章 基于双目视觉的赛道定位与建图

要建立赛道地图，赛车需将锥桶像素坐标转换为全局坐标。因此首先需要将锥桶的像素坐标转换为三维空间中的世界坐标，然后利用赛车自身的定位信息将锥桶的世界坐标转换为全局坐标，实现赛道地图的建立。因此本章将研究基于双目相机的锥桶世界坐标求解算法，分析赛车自身定位原理，然后融合定位坐标和锥桶坐标建立赛道地图。

3.1 基于双目相机的锥桶坐标获取

赛道建图任务首先需要获得锥桶在三维空间中的坐标，因此在第二章获取锥桶像素坐标后，可根据利用双目相机成像原理求解锥桶的世界坐标。本节将研究相机的成像原理以及双目空间坐标定位原理，在完成双目标定后计算锥桶的世界坐标。

3.1.1 双目相机选型

目前基于视觉的物体空间坐标定位方案主要有两种：基于单目的 PnP 估计法和利用深度信息求解物体三维坐标的方法。单目相机利用相机的标定参数，将物体在相机坐标系下的投影信息转换至像素坐标系，然后利用 PnP 算法计算相机在世界坐标系下的位置和姿态信息^[86]。但 PnP 算法需要建立目标特征数据库，通过多次迭代来寻找最小重投影误差，从而获得可靠的坐标信息，该方法实现繁琐扩展性差，不适用于赛车的建图任务。

双目获取目标三维坐标的方法是利用双目相机生成的深度图像，根据双目空间坐标定位原理计算物体的三维坐标。相对于单目相机来说，不用建立数据库，不依赖物体的特征，扩展性强。目前获取物体深度的相机主要分为三种，分别是 TOF 深度相机、结构光深度相机和 RGB 双目相机，其特点如表 3.1 所示：

表 3.1 深度相机性能对比表

相机类型	TOF	RGB 双目	结构光
测距方式	主动式	被动式	主动式
测量精度	1cm	0.01-1cm	0.01-1mm
测量范围	0.1-100m	0.3-25m	0.1-10m
帧率	>100fps	<100fps	几十 fps
分辨率	低于 640×480	可达 2k	可达 1280×720
户外工作	影响较大	无影响	户外影响大

1、TOF 深度相机

TOF 即 Time of flight（飞行时间），TOF 相机获取深度原理是：利用发射器连续向目标物体发射激光脉冲信号，利用传感器接收反射光线，通过激光往返的“飞行时间”计算深度图。但 TOF 相机价格昂贵，相机分辨率较低，易受户外光照影响，因此本文不采用该方案。

2、结构光深度相机

结构光深度相机测量深度的原理是：通过红外激光器将调制后的结构光照射到物体上，再通过红外摄像头采集反射的结构光，相机通过计算反射光的结构变化来获取物体深度信息。该方法运算成本低，分辨率和帧率都较高，但是相机使用红外光作为检测光源，极易受环境光影响，室外效果很差。对于在室外行驶的赛车而言，该方案无法满足需求。

3、RGB 双目相机

RGB 双目相机是采用两颗单目摄像头分别采集同一片区域图像，根据两幅图像之间的微小差距计算视差，结合双目成像原理生成深度图。双目相机成本低廉，且室内外性能良好。虽然相机深度检测范围受基线影响大，远距离精度较低。但是对于本文建图任务而言，首圈以极低速度采集锥桶信息，检测距离要求不高，因此双目相机可以适用于赛车的建图任务。本文使用的双目相机参数如表 3.2 所示：

表 3.2 双目相机参数表

参数	左目摄像头
镜头焦距	4mm
测距范围	1m-6m
基线	12cm
分辨率	1280x720
俯仰角度	70°-90°
动态范围	120dB
图像帧率	30Hz

3.1.2 单目相机成像模型

根据小孔成像原理建立相机成像模型，确定物体世界坐标与图像坐标的间的转换关系如图 3.1 所示：

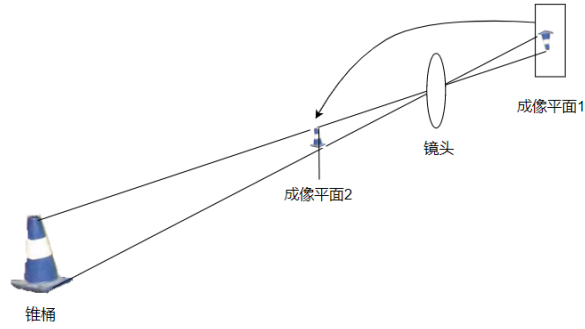


图 3.1 相机成像原理

为了简化分析，将成像平面 1 图像转移到成像平面 2 上，假设锥桶顶点为 P ，将相机坐标系原点固定在镜头光心可建立相机成像模型，其中世界坐标系、相机坐标系、图像坐标系和像素坐标系之间的转换关系如图 3.2 所示。

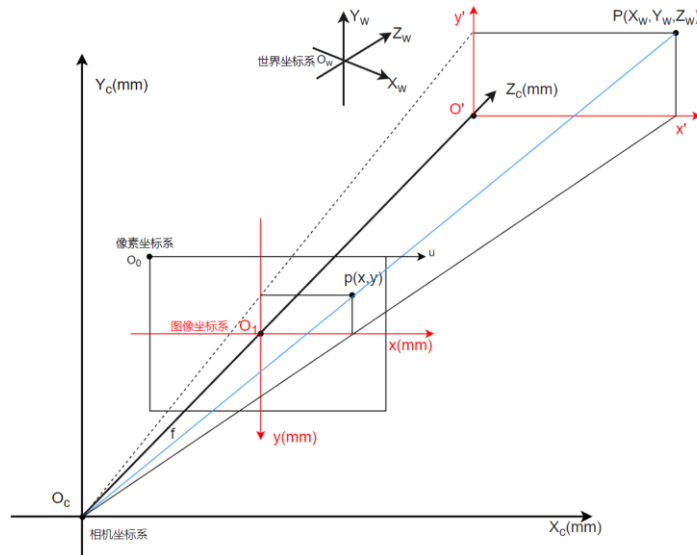


图 3.2 相机成像模型

如图 3.2 所示，坐标系 $O_w-X_wY_wZ_w$ 是锥桶所在空间位置的世界坐标系。 $O_c-X_cY_cZ_c$ 为相机坐标系，坐标系原点为相机的光心， Z_c 轴与相机光轴平行，单位为 mm。在相机的成像平面有两种坐标系分别是图像坐标系 O_1-xy 和像素坐标系 O_0-uv 。图像坐标系以成像平面中心为坐标原点， x ， y 轴分别指向像素坐标的 u ， v 正方向，单位为 mm。像素坐标系以 O_0 为坐标原点，描述物体的像点在数字图像上的坐标，单位为像素。 f 为相机的焦距表示相机光心到成像平面的距离即 O_c 到 O_1 的距离。

建立相机成像模型后可通过四种坐标系间的转换，得到锥桶顶点 P 与相机坐标系的映射关系^[87]。坐标系的转换关系如图 3.3 所示：

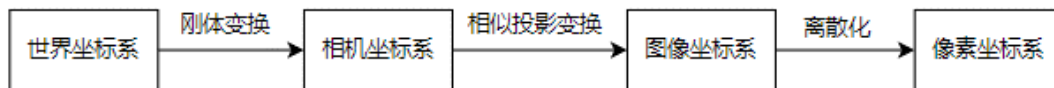


图 3.3 三大坐标转换关系

(1) 世界坐标系与相机坐标系的转换

相机通过刚体变换将世界坐标系下坐标转换到相机坐标系，可以避免物体在成像时发生形变。刚体变换即旋转平移操作，设 P 点在世界坐标系的坐标为 (X_w, Y_w, Z_w) ，在相机坐标系坐标为 (X_c, Y_c, Z_c) ，刚体变化矩阵表达式如下：

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} r_{00} & r_{01} & r_{02} \\ r_{10} & r_{11} & r_{12} \\ r_{20} & r_{21} & r_{22} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \\ T_z \end{bmatrix} \quad (3.1)$$

上式的齐次表达式为：

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}_3^T & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (3.2)$$

其中 $\mathbf{R}$ 为旋转矩阵， $\mathbf{t}$ 为平移向量。

(2) 相机坐标系与图像坐标系的转换

由图 3.2，为了保证远离视点的物体看起来比靠近视点的物体更小，使二维图像更立体有深度，相机根据相似透视投影将 P 点投影到二维图像坐标系的 p 点。设相机坐标系下 P 点齐次坐标为： $[X_c \ Y_c \ Z_c \ 1]^T$ ， p 点的齐次坐标为 $[x \ y \ 1]^T$ ，根据三角形相似原理可得：

$$\begin{cases} x = \frac{fX_c}{Z_c} \\ y = \frac{fY_c}{Z_c} \end{cases} \quad (3.3)$$

转换为矩阵表达式如下式所示：

$$Z_c \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} \quad (3.4)$$

(3) 图像坐标系与像素坐标的转换

由于图像坐标系下物体以毫米(mm)为单位,因此需要将物体按照一定粒度离散化以获得像素坐标。两坐标位于同一平面,设图像坐标的中心位于像素坐标的 (u_0, v_0) 处,如图 3.4 所示:

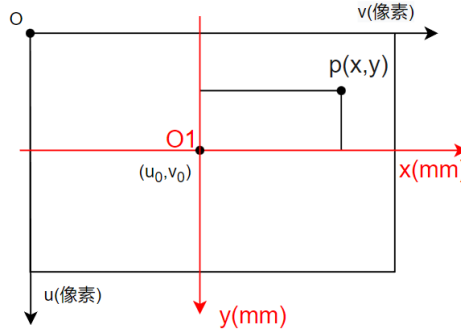


图 3.4 像素坐标与图像坐标关系

根据像素坐标与图像坐标转换关系可得:

$$u = \frac{x}{dx} + u_0 \quad (3.5)$$

$$v = \frac{y}{dy} + v_0 \quad (3.6)$$

其中 dx , dy 由相机厂商决定,单位为(mm / 像素)。式(3.5)、(3.6)的齐次坐标表示为:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{dx} & 0 & u_0 \\ 0 & \frac{1}{dy} & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (3.7)$$

综合(3.1)~式(3.7)可得世界坐标系和像素坐标系的转换关系:

$$Z_c \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & u_0 & 0 \\ 0 & f_y & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0^T & 1 \end{bmatrix} \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = \mathbf{M}_1 \mathbf{M}_2 \mathbf{X}_w = \mathbf{M} \mathbf{X}_w \quad (3.8)$$

其中 $f_x = \frac{f}{dx}$, $f_y = \frac{f}{dy}$ 。 $\mathbf{M}$ 为相机系数矩阵,其中 $\mathbf{M}_1$ 为相机的内参矩阵, $\mathbf{M}_2$ 为相

机外参矩阵,包括 $\mathbf{R}$ 旋转矩阵, $\mathbf{t}$ 平移矩阵, $\mathbf{X}_w$ 为物体世界坐标齐次矩阵。

相机的内外参数矩阵可以通过相机标定获得，同时由于相机的制作工艺的限制，相机采集的图像存在误差和畸变，因此也需要通过相机标定获得的畸变参数校正误差。

3.1.3 双目空间坐标定位原理

根据式(3.8)可以将世界坐标系下的物体转换到图像坐标系下，但系数矩阵 $\mathbf{M}$ 并非方阵，单目相机无法根据图像的像素坐标求解物体的世界坐标。如图 3.5 所示假设左右目相机镜头平行，则空间中一点 P 在左右像素平面的像点将在同一行不同列，即该点在左右图中的成像点在同一个极线上，根据双目空间坐标定位原理即可求得 P 点的世界坐标。

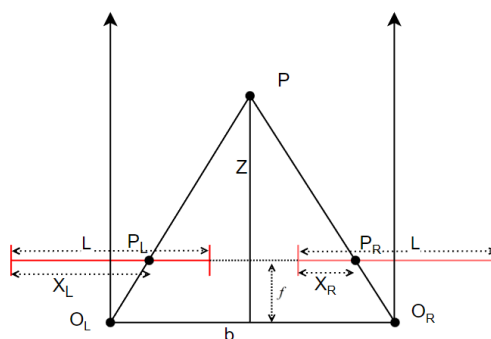


图 3.5 双目空间坐标定位原理

图 3.5 中 O_L ， O_R 分别为左右摄像头的光心，其连线为基线 b 。 f 为相机距离各自成像平面的焦距。 L 为左右相机的成像范围。 P_L 和 P_R 为 P 在左右成像面上的成像点， P_L 和 P_R 距各自成像面的左边缘的距离为 X_L 和 X_R 。 Z 为 P 点与相机基线的距离即深度。本文将双目相机世界坐标固定在左目摄像头光心，与左目相机坐标系重合。其中 $\triangle P_L P_R P$ 相似于 $\triangle O_L O_R P$ ，根据相似原理可得：

$$\frac{P_L P_R}{Z - f} = \frac{O_L O_R}{Z} \quad (3.9)$$

$$\frac{b - [(X_L - \frac{L}{2}) + (\frac{L}{2} - X_R)]}{Z - f} = \frac{b}{Z} \quad (3.10)$$

$$\frac{b + X_R - X_L}{Z_w - f} = \frac{b}{Z_w} \quad (3.11)$$

利用式(3.7)将 X_L ， X_R 的物理坐标转换为像素坐标 u_L ， u_R 可得：

$$\frac{b + (u_R - u_L)dx}{Z_w - f} = \frac{b}{Z_w} \quad (3.12)$$

$$Z_w = \frac{bf}{(u_L - u_R)dx} = \frac{bf_x}{u_L - u_R} \quad (3.13)$$

令视差 $d = u_L - u_R$ ，可得深度计算公式：

$$Z_w = \frac{bf_x}{d} \quad (3.14)$$

根据投影变换过程的相似原理得：

$$\frac{(u - u_0)dx}{f} = \frac{u - u_0}{f_x} = \frac{X_w}{Z_w} \quad (3.15)$$

$$\frac{(v - v_0)dy}{f} = \frac{v - v_0}{f_y} = \frac{Y_w}{Z_w} \quad (3.16)$$

当 $f_x = f_y$ 时，可得 X_w 和 Y_w 坐标：

$$X_w = \frac{u - u_0}{f_x} Z_w = \frac{u - u_0}{f_x} \frac{bf_x}{d} = \frac{b(u - u_0)}{d} \quad (3.17)$$

$$Y_w = \frac{v - v_0}{f_y} Z_w = \frac{v - v_0}{f_y} \frac{bf_x}{d} = \frac{b(v - v_0)}{d} \quad (3.18)$$

联立式(3.14)、式(3.17)、式(3.18)可得：

$$\begin{bmatrix} X_w \\ Y_w \\ Z_w \\ W \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & -u_0 \\ 0 & 1 & 0 & -v_0 \\ 0 & 0 & 0 & f_x \\ 0 & 0 & \frac{1}{b} & 0 \end{bmatrix} \begin{bmatrix} u \\ v \\ d \\ 1 \end{bmatrix} \quad (3.19)$$

记 $\mathbf{Q} = \begin{bmatrix} 1 & 0 & 0 & -u_0 \\ 0 & 1 & 0 & -v_0 \\ 0 & 0 & 0 & f_x \\ 0 & 0 & \frac{1}{b} & 0 \end{bmatrix}$ 为重投影矩阵，由式(3.19)可知，若视差 d 已知，则利用 $\mathbf{Q}$ 即

可计算像素坐标 (u, v) 的世界坐标 $(X_w/W, Y_w/W, Z_w/W)$ 。

3.1.4 相机标定

实际工程中，相机通过透镜把物体投影到成像平面时，会出现径向和切向畸变。同时由于相机内部参数 f_x 、 f_y 未知，因此需要对左右摄像头做单目标定，确定各自畸变参

数和内外参矩阵。

由式(3.19)可知可知，计算物体 Z_w 坐标的关键是视差 d ，可由空间中一点在左右图像上的匹配点计算。但在双目相机成像模型中无法保证左右相机光轴绝对平行，因此左图中像素点 p 在右图中的匹配点可以在二维平面中的任意位置，这将加大寻找匹配点的难度。因此需要利用单目标定参数对双目相机进行立体校正和校准，将左图中的匹配点约束在右图极线范围内。

由于相机标定需要极高精度且步骤复杂，为了简化标定流程，保证标定的可靠性，本文利用 matlab 标定工具 Stereo Camera Calibrator 进行标定。使用黑白格边长 50 mm 的棋盘格作为标定板，如图 3.6 所示。

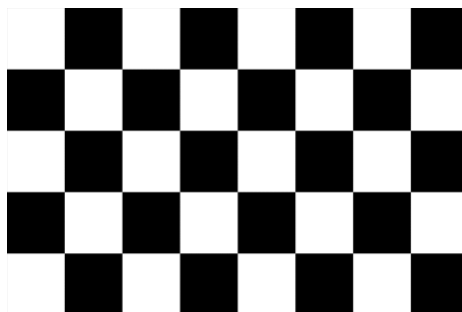


图 3.6 棋盘格标定板

使用双目相机拍摄 168 组包括 1 m~4 m 范围内不同姿态的棋盘格图像作为标定数据集，如图 3.7 所示。

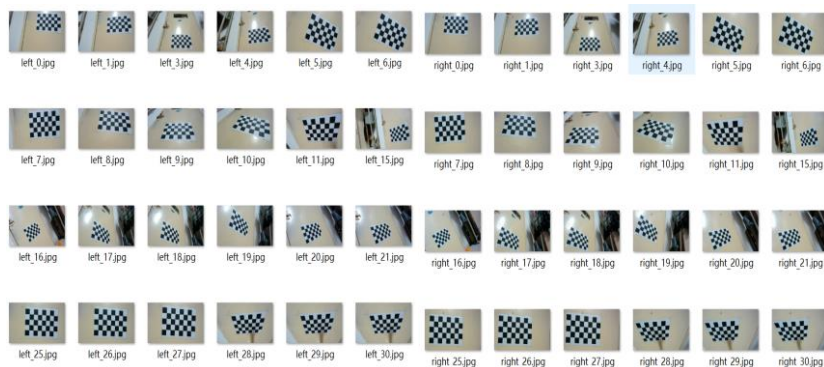


图 3.7 标定数据集

将标定数据集存储路径导入 Stereo Camera Calibrator，设置标定板黑白格尺寸 50mm，按照文件存储的先后顺序寻找图片中标定板的角点，开始单目相机参数标定。如图 3.8 所示，图中绿色为棋盘格的角点，红色点为重投影点。

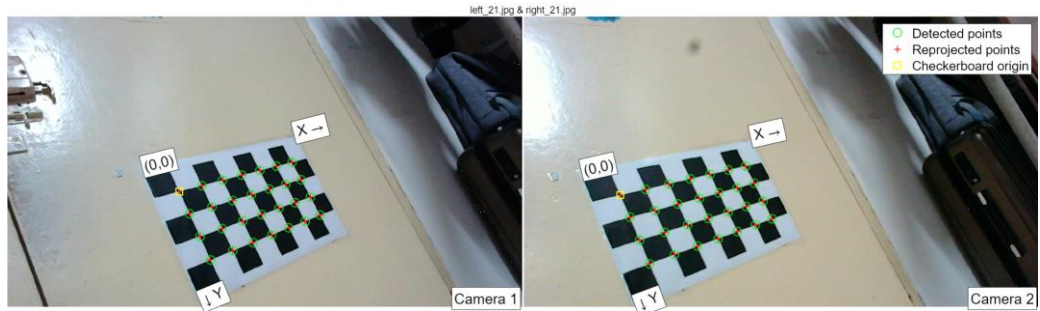


图 3.8 标定图像角点

图 3.9 为双目摄像头标定的重投影误差，其误差越大则标定精度越低，图 3.9 表明中标定误差的最大误差为 0.32 像素，平均误差为 0.20 像素，标定结果良好。

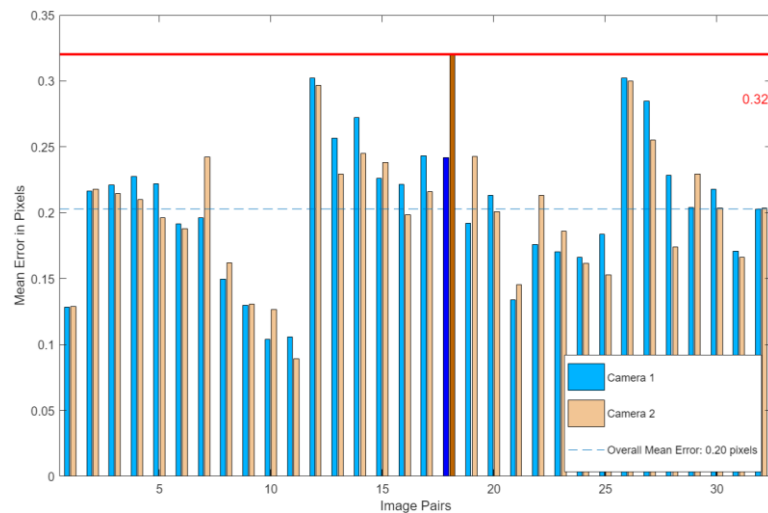


图 3.9 标定重投影误差

在完成左右摄像头标定后，利用标定参数进行极线矫正。图 3.10 为双目拍摄的原始图像，图 3.11 为极线矫正后的图像。图 3.10、图 3.11 表明，经过极线校正后，左右目图像被多条极线约束对齐，此时左图中的角点可以在右图中的同一极线上找到相应的匹配点。因此进行立体匹配计算时，仅需沿极线对图像进行搜索即可得到匹配点。

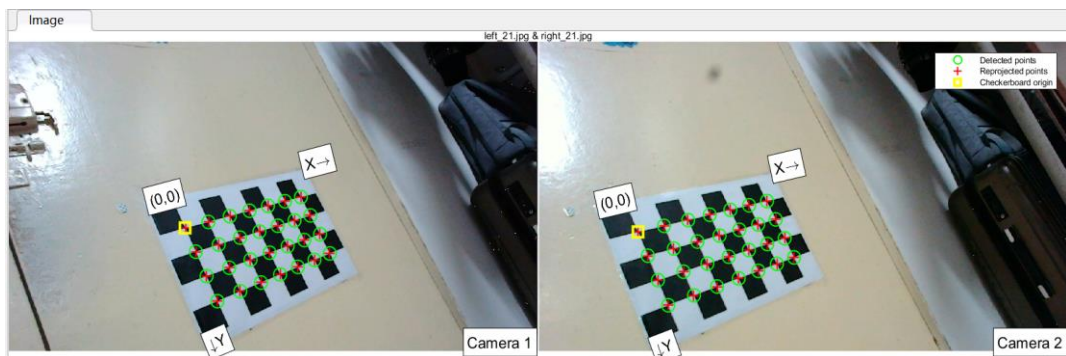


图 3.10 双目原始图像

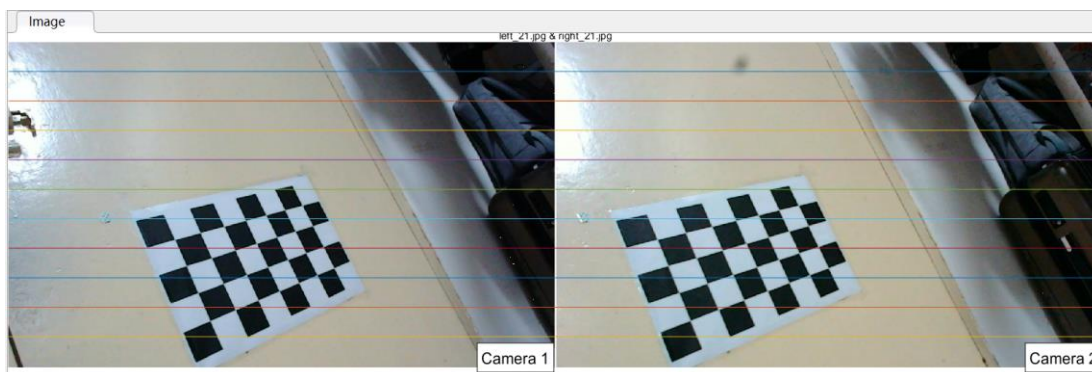


图 3.11 双目极线校正图

完成双目标定得到的双目标定参数如表 3.3 所示：

表 3.3 双目标定参数表

参数	左目摄像头	右目摄像头
内参矩阵	$\begin{bmatrix} 834.6331 & 0 & 0 \\ 0 & 833.1959 & 0 \\ 681.8996 & 500.8749 & 1 \end{bmatrix}$	$\begin{bmatrix} 836.5885 & 0 & 0 \\ 0 & 834.5954 & 0 \\ 648.2737 & 500.5348 & 1 \end{bmatrix}$
径向畸变系数	$[-0.0023 \ 0.0091 \ 0]$	$[-0.0101 \ 0.0182 \ 0]$
切向畸变系数	$[-0.0014 \ 0.0045]$	$[-0.002 \ 0.0021]$
旋转矩阵 $\mathbf{R}$	$\begin{bmatrix} 0.999961108 & -0.000560409 & 0.0088016 \\ 0.000663358 & 0.999931356 & -0.011698027 \\ -0.00879444 & 0.011703411 & 0.999892838 \end{bmatrix}$	
平移矩阵 $\mathbf{t}$	$[-115.2202403 \ -0.102666149 \ -0.620943383]$	

3.2 获取锥桶世界坐标

在完成极线校正后，利用半全局立体匹配算法 SGM 计算双目视差图。根据双目标定参数将视差图转换为深度图，然后根据锥桶目标检测结果在深度图上获取锥桶深度，结合重投影矩阵 $\mathbf{Q}$ 可求得锥桶的世界坐标。

SGM 算法采用全局匹配算法的框架，使用一维路径聚合方法来最优化能量函数得到视差图，具有精度高且运算速度快等特点。SGM 算法分为四步：图像前处理、代价计算、代价聚合、视差计算与优化^[88]。

1、图像前处理

如图 3.12, 首先对左右图像进行 pad=2 的补齐操作, 利用图中黄色的 5×5 滑动窗口, 以步长为 1 在左右图像向上滑动, 对比滑动窗口中心像素和其邻域内像素的灰度值, 求解图像的 Census 变换值^[89]。

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	175	98	74	196	172	152	55
0	0	203	4	222	18	193	71	123
0	0	235	176	20	106	149	234	134
0	0	138	25	109	116	208	114	127
0	0	88	211	199	255	3	4	123
0	0	158	231	69	93	90	67	217

图 3.12 Census 变换计算过程

如图 3.13 所示, 窗口中点像素值为 20, 若其邻域像素灰度值大于中心像素灰度值输出布尔值 1, 反之输出 0。

1	1	1	1	1
1	0	1	0	1
1	1	0	1	1
1	0	1	1	1
1	1	1	1	0

图 3.13 Census 变换输出值

最后输出一串比特值作为该滑动窗口中心像素的 Census 变换结果, Census 变换表达式如下:

$$C_s(u, v) = \bigotimes_{i=n'}^{n'} \bigotimes_{j=-m'}^{m'} \xi(I(u, v), I(u+i, v+j)) \quad (3.20)$$

其中 (u, v) 表示中心点的像素坐标, n' 和 m' 分别为不大于 n 和 m 的一半的最大整数, $\bigotimes$ 为比特位逐位连接运算符, $I(u, v)$ 表示坐标为 (u, v) 像素的灰度值, ξ 运算由式 (3.22) 定义:

$$\xi(x, y) = \begin{cases} 0 & \text{if } x \leq y \\ 1 & \text{if } x > y \end{cases} \quad (3.21)$$

2、代价计算

以图 3.14(a)中滑动窗口的位置为参考点, 计算图 3.14(b)在视差 d 范围内的 Census 值。

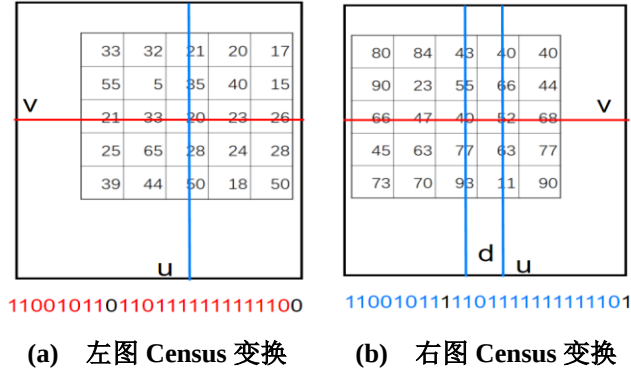


图 3.14 汉明距离计算图

利用异或运算计算左右图比特串对应位的差异得到汉明距离(Hamming)。汉明距离计算式如下：

$$C(u, v, d) = \text{Hamming}(C_{sl}(u, v), C_{sr}(u - d, v)) \quad (3.22)$$

C_{sl} 、 C_{sr} 表示左右图对应像素坐标的 Census 值。图 3.14 中 Census 值不同的位数为 2，因此 Hamming 距离为 2，设视差范围为 d ，根据上述计算即可得 (u, v, d) 的三维代价矩阵。

3、代价聚合

由于 Census 变化仅在滑动窗口内求解像素差值，只考虑了局部相关性，因此有很多噪声。对此定义全局能量函数，以寻找图像中每个像素的最优视差来最小化全局能量函数的方式增强匹配效果。SGM 能量函数表达式如下：

$$E(d) = \sum_p C(p, d_p) + \sum_{q \in N_p} P_1 T[|d_p - d_q| = 1] + \sum_{q \in N_p} P_2 T[|d_p - d_q| > 1] \quad (3.23)$$

其中， C 表示像素 p 的匹配代价， d_p 表示 p 像素的视差图。公式的第一项为所有以 d 为视差的像素匹配代价的总和。第二项表示像素 p 与像素 p 相邻的像素 q 的视差之间差值等于 1 的和。第三项是分别表示像素 p 与像素 p 相邻的像素 q 的视差之间差值大于 1 的和。 T 返回括号内表达式的真值即表达式成立输出 1，否则输出 0。 N_p 表示邻域范围， P_1 、 P_2 为惩罚项，当相邻像素视差变化 1 像素时用 P_1 惩罚，当相邻像素视差变化大于 1 时用 P_2 惩罚。

获得全局能量函数后，为了最小化能量函数，首先计算每个像素在所有视差下的匹配代价，然后沿像素周围的多条路径对每个像素进行一维代价聚合。在每个路径上，采用动态规划方法寻找该路径的最小路径代价值。随后将像素在所有路径上的代价值相加，得到该像素的聚合匹配代价值。像素 p 沿某条路径 r 的路径代价 $L_r(p, d)$ 计算如下：

$$L_r(p, d) = C(p, d) + \min \left\{ \begin{array}{l} L_r(p-r, d) \\ L_r(p-r, d-1) + P_1 \\ L_r(p-r, d+1) + P_1 \\ \min_i L_r(p-r, i) + P_2 \end{array} \right\} - \min_i L_r(p-r, i) \quad (3.24)$$

其中， $C(p, d)$ 为像素 p 在 d 视差范围的代价值； $L_r(p-r, d)$ 为 r 路径上 p 的相邻点在视差为 d 时的代价聚合值。第二项表示 p 在该条路径 r 上的相邻点在不同视差范围内（如： $d, d-1, d+1$ ）的聚合值与惩罚项之和的最小值。第三项为路径 r 上的最小视差用以防止代价聚合值无限增大。

像素 p 所有路径的总路径代价值 $S(p, d)$ ：

$$S(p, d) = \sum_r L_r(p, d) \quad (3.25)$$

代价聚合一般有 4 路径，8 路径，16 路径的聚合，图 3.15 为 3 种路径聚合图。中心像素可沿周围 2, 4, 6, 8 四路径聚合、1~8 八路径聚合和外层十六路径聚合，路径越多精度越高，但是耗时也更长。

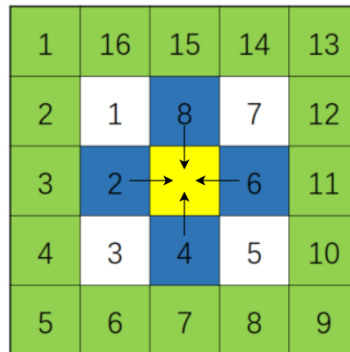


图 3.15 路径聚合图

4、视差计算与优化

在获得像素聚合代价后，使用赢家通吃（WTA）算法^[90]，选择最小聚合代价所对应的视差作为最终视差，为每个像素分配视差值，然后采用子像素拟合、一致性检查、唯一性约束、剔除小连通区、中值滤波等方式进行优化，最终得到整个图像的视差图，结合双目标定参数可得深度图。图 3.16(a) 为左目彩色图像，图中锥桶的深度与图 3.16(b) 深度图中的黄色方框标记一一对应，算法生成的深度图效果良好。

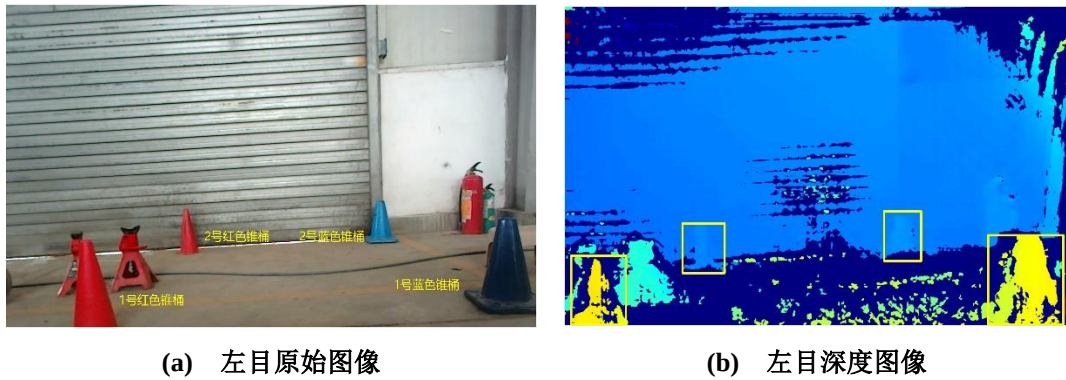


图 3.16 双目深度图像

图 3.16 所示深度图存在噪点,可以根据 YOLO 的锥桶结果在深度图上提取图像块,将图像块中的最小深度值作为锥桶深度,达到去噪目的。在获取目标深度值后,利用重投影矩阵 Q 即可计算出锥桶的世界坐标,锥桶坐标获取流程如图 3.17 所示:



图 3.17 获取锥桶空间坐标流程图

图 3.16 中锥桶世界坐标检测结果如表 3.4 所示,由于世界坐标系与左目摄像头重合,因此建图时仅关注锥桶的 x 轴、 z 轴坐标。图 3.16(a)中锥桶的实际摆放位置与检测结果误差在 0.1m~0.2m 之间波动,在可接受范围,这表明本文的锥桶世界坐标检测算法效果良好。

表 3.4 锥桶世界坐标检测表

相机类型	像素坐标 (u, v)	世界坐标 (x, y, z) (m)	测量距离(m)	真实距离(m)
1 号红色锥桶	[204 712]	[-0.6615 0.2565 1.2215]	1.41m	1.4m
2 号红色锥桶	[440 580]	[-0.7858 0.1523 3.0895]	3.19m	3m
1 号蓝色锥桶	[1179 703]	[0.8143 0.2512 1.2624]	1.52m	1.4m
2 号蓝色锥桶	[895 545]	[0.877 0.0197 2.9293]	3.06m	3m

3.3 基于组合惯导的自行车定位

为了构建赛道的全局地图,需获取赛车的精确定位信息。在获得当前帧内锥桶的空间坐标后,采用坐标转换方法将锥桶的世界坐标映射到全局坐标系下,从而实现单帧赛

道地图的构建。接着，通过拼接各帧赛道地图，得到全局赛道地图。

赛道地图构建过程对定位精度要求较高，传统的全球定位系统(GPS)更新频率低。而惯性导航系统虽然更新频率较快，但其对距离估计涉及积分计算，误差会随时间累积而逐渐增加。组合惯性导航系统(INS)综合 GPS 和惯性导航系统的优点，能提供高频且精确的定位信息，因此本节主要研究基于组合惯性导航系统的赛车定位方法，以及赛道地图的建图过程。

3.3.1 组合惯性导航系统标定

INS 系统获取导航信息，需要对组合惯导进行实车标定，确定传感器之间的相对位置、姿态角和传感器误差等参数。组合惯导的标定分为静态标定和动态标定。静态标定在静止的状态下对 INS 进行测量，确定加速度计的零偏参数和陀螺仪的漂移参数。动态标定则需要将 INS 安装在行驶中的赛车上进行测量，以确定加速度计和陀螺仪的标度因子和误差系数，并对 INS 的误差进行补偿和校正，提高导航精度和稳定性。本文使用的组合惯导是由广州导远电子科技有限公司赞助的 INS570D，如图 3.18 所示：



图 3.18 INS570D 组合惯导

INS570D 由主机、主从卫星接收天线、DTU(4G 模块)组成。主从天线在赛车上的布置如图 3.19 所示：



图 3.19 组合惯导天线布置图

1、静态标定

组合惯导的天线在赛车上一前一后放置，主天线位于后轴中心上方。主从天线安装角度的偏差为 $(x,x,0)$ 。将赛车停靠在空旷水平路面，检查 DTU 联网正常后，将惯导在赛车上的安装位置写入惯导并重新上电，完成静态标定后，保存参数。

2、动态标定

完成静态标定后，等待惯导接收的卫星数大于 17 颗时，启动赛车，按照日字线路（包含左、右转弯和拐弯后需直行），行驶 5-10min，当标定误差稳定后，保存标定参数并将参数写入惯导，重新上电完成动态标定。INS 标定结果如图 3.20 红框所示：

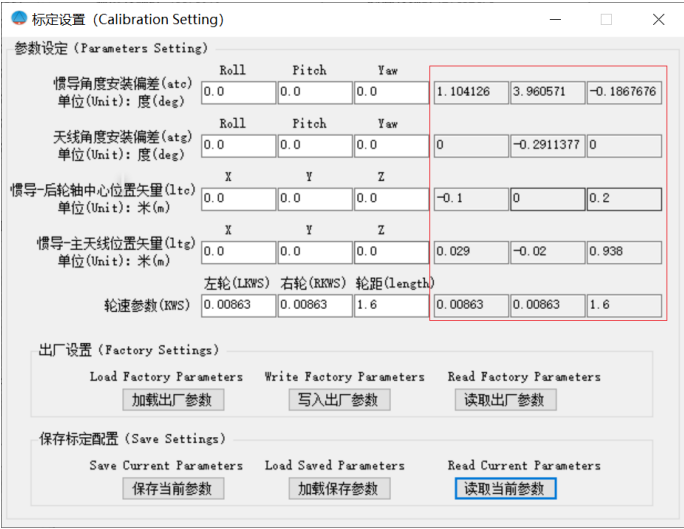


图 3.20 组合惯导标定结果

3.3.2 定位坐标转换

INS 经过标定后输出赛车的实时定位信息，如表 3.5 所示：

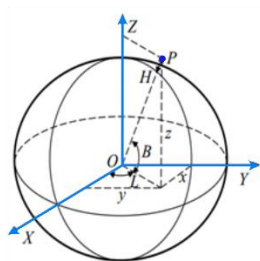
表 3.5 INS570D 信号表

信号名	StartByte	Length	Factor	Offset	Range
横滚角 Roll	3	16	360/32678	-90	-90-90
俯仰角 Pitch	5	16	360/32678	-180	-180-180
横摆角 Yaw	7	16	360/32678	-180	-180-180
X 轴角速度 ω_x	9	16	300/32678	-250	-250-250
Y 轴角速度 ω_y	11	16	300/32678	-250	-250-250
Z 轴角速度 ω_z	13	16	300/32678	-250	-250-250
X 向加速度 a_x	15	16	12/32768	-4	-4-4
Y 向加速度 a_y	17	16	12/32768	-4	-4-4
Z 向加速度 a_z	19	16	12/32768	-4	-4-4

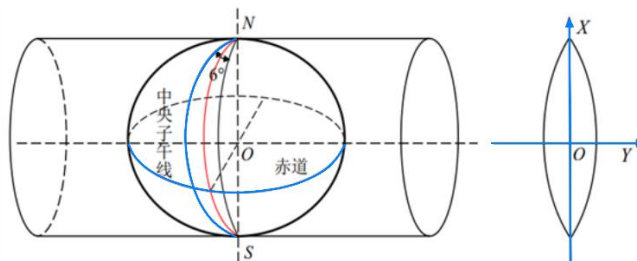
表 3.5 INS570D 信号表 (续)

信号名	StartByte	Length	Factor	Offset	Range
经度 B	21	32	0.0000001	-90	-90-90
纬度 L	25	32	0.0000001	-180	-180-180
高度 H	29	32	0.001	0	-
东向速度 ve	33	16	100/32768	0	-
北向速度 vn	35	16	100/32768	0	-
地向速度 vd	37	16	100/32768	0	-

由表 3.5 可知, 组合惯性导航系统输出的位置坐标是 WGS-84 坐标系下的经纬度坐标, 如图 3.21 所示, 该坐标系原点固定于地球中心, Z 轴指向地球自转轴方向, X 轴延地球中心指向赤道平面与本初子午线相交处点方向, Y 轴位于赤道平面且与 X、Z 轴构成的右手高斯直角坐标系。



图(a) WGS-84 坐标系



图(b) 高斯直角坐标系

图 3.21 WGS-84 坐标系与高斯直角坐标系

由于赛车在进行路径规划时用到的是高斯直角坐标系, 因此需要通过高斯正投影变换将位于 WGS-84 坐标系的定位坐标转换为高斯直角坐标。高斯正投影变换计算如下:

$$X = 111132.9525B - 16038.4201\sin 2B + 16.8325\sin 4B - 0.0220\sin 6B \quad (3.26)$$

式中 X 为子午线弧长, B 为待转换点纬度。由 X 可得 x 坐标为:

$$\begin{aligned} x = & X + l^2 N \sin B \cos B / 2 + l^4 N (5 - t^2 + 9\eta^2 + 4\eta^4) \sin B \cos^3 B / 24 \\ & + l^6 N (61 - 58t^2 + t^4) \sin B \cos^5 B / 720 \end{aligned} \quad (3.27)$$

式中 $l = (L - L_0) / \rho$, L 为待转换点经度, L_0 为投影带中心子午线经度, 西安市地区为 108° , $\rho = 180 \cdot 3600 / \pi$; $N = a / \sqrt{1 - e_1^2 \sin^2 B}$, 其中 a 为地球长半径, 即 $6378137.0(\text{m})$, $e_1 = 2f - f^2$ 为地球第一偏心率, 其中 f 为地球扁率, 即 0.003352810664 ; $t = \tan B$; $\eta = e_2 \cos B$, $e_2 = \sqrt{1 / (1 - f^2)} - 1$ 为地球第二偏心率。

y 坐标为:

$$y = 500000 + lN \cos B + l^3 N \cos^3 B (1 - t^2 + \eta^2) / 6 + l^5 N \cos^5 B (5 - 18t^2 + t^4 + 14\eta^2 - 58\eta^2 t^2) / 120 \quad (3.28)$$

获取组合惯导在直角坐标系中的位置坐标后, 还需要获取车辆的航向角和车身坐标系下的车速, 其中航向角由弧度表示, 转换关系式为:

$$\theta = yaw \cdot \pi / 180 \quad (3.29)$$

车身坐标系下的车速由北东地坐标系下的绝对车速转换得到, 其转换关系为:

$$v = \begin{cases} -v_e \sin(\pi + yaw) - v_n \cos(\pi + yaw), & -\pi \leq yaw < -\pi/2 \\ -v_e \sin(-yaw) + v_n \cos(-yaw), & -\pi/2 \leq yaw < 0 \\ v_e \cos(\pi/2 - yaw) + v_n \sin(\pi/2 - yaw), & 0 \leq yaw < \pi/2 \\ v_e \cos(yaw - \pi/2) - v_n \sin(yaw - \pi/2), & \pi/2 \leq yaw < \pi \end{cases} \quad (3.30)$$

由于组合惯导系统运行频率很高, 为了获得准确的纵向加速度和横摆角速度信息需要利用滑动窗口滤波算法对组合惯导的输出进行滤波处理, 滑动窗口滤波算法为:

$$a = \begin{cases} a_x[i], & i \leq L \\ \sum_{j=i-L}^i a_x[j] / L, & i > L \end{cases} \quad (3.31)$$

$$\varpi = \begin{cases} \varpi_z[i], & i \leq L \\ \sum_{j=i-L}^i \varpi_z[j] / L, & i > L \end{cases} \quad (3.32)$$

式(3.31), (3.32)中, i 为组合惯导输出的信号序号, $L=10$ 为滑动窗口的长度, 表示取 10 个纵向加速度或横摆角速度的平均值。

3.4 赛道地图建立

3.4.1 锥桶全局坐标转换

由于锥桶坐标与定位坐标位于不同坐标系, 因此需要根据相机和组合惯导系统在车身上的安装位置, 利用坐标转换方法将锥桶坐标转换到全局坐标系下。本文的全局坐标系是以赛车起点位置为原点, 车头方向为 y 轴构建的右手坐标系。双目相机和组合惯导在车身安装的相对位置如图 3.22 所示:

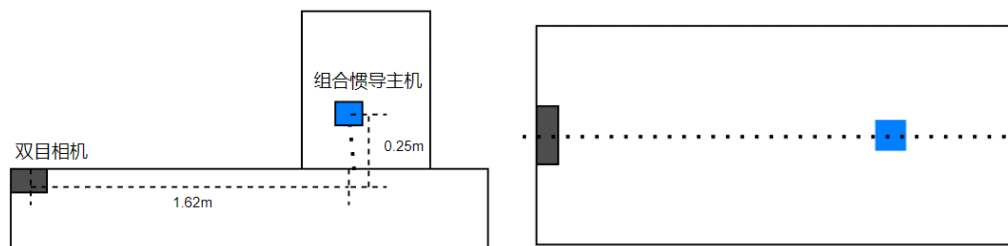


图 3.22 双目相机和组合惯导安装相对位置

根据两种传感器的安装位置，可以利用坐标转换原理完成坐标转换，原理如下：

1、坐标平移变换

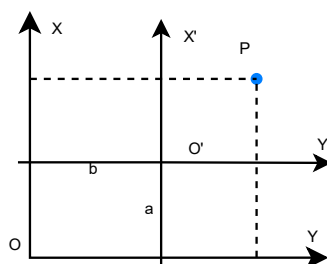


图 3.23 坐标平移变换

图 3.23 中的两个坐标系分别为 XOY ， $X'O'Y'$ ，两者横纵向坐标轴相互平行且方向相同。其中 $X'O'Y'$ 横坐标的偏移量为 b ，纵坐标偏移量为 a 。设 XOY 坐标系下一点 $P(x, y)$ ， P 点在 $X'O'Y'$ 平面坐标为 (x', y') ，根据横纵坐标偏移量可得：

$$\begin{cases} x = x' + a \\ y = y' + b \end{cases} \quad (3.33)$$

2、坐标旋转变换

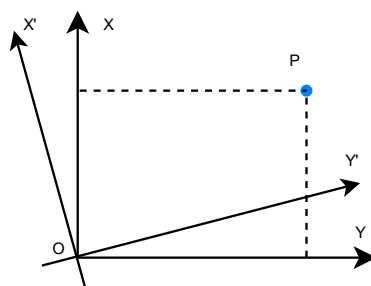


图 3.24 坐标旋转变换

图 3.24 中 XOY 坐标系绕原点 O 逆时针旋转 θ 角得到坐标系 $X'O'Y'$ 。设 P 在 XOY 坐标系下坐标为 (x, y) ，在 $X'O'Y'$ 坐标系下坐标为 (x', y') 。根据旋转角度 θ 可得：

$$\begin{cases} x = x' \cdot \cos \theta - y' \cdot \sin \theta \\ y = x' \cdot \sin \theta + y' \cdot \cos \theta \end{cases} \quad (3.34)$$

转换为矩阵形式为:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x' \\ y' \end{bmatrix} \quad (3.35)$$

或者:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (3.36)$$

3、世界坐标变换

因为相机坐标和车身坐标为三维空间中的坐标, 因此需要进行三维空间中的坐标变换。设空间中有两个坐标系 $O\text{-}XYZ$ 与 $O'\text{-}X'Y'Z'$, 若坐标系 $O\text{-}XYZ$ 仅绕自身的 X 轴逆时针转 θ 之后与坐标系 $O'\text{-}X'Y'Z'$ 重合, 则 P 点在坐标系 $O\text{-}XYZ$ 和 $O'\text{-}X'Y'Z'$ 的关系如式 (3.31) 所示:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & \sin \theta \\ 0 & -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} \quad (3.37)$$

若坐标系 $O\text{-}XYZ$ 仅绕自身 Y 轴旋转 θ 可得对应的坐标关系:

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad (3.38)$$

若坐标系 $O\text{-}XYZ$ 仅绕自身 Z 轴旋转 θ 可得对应的坐标关系:

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ 0 & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad (3.39)$$

对于存在任意旋转变换的两个空间坐标系 $O\text{-}XYZ$ 和 $O'\text{-}X'Y'Z'$, 可以通过依次绕三个坐标轴旋转一定角度, 实现两个坐标系对应坐标轴的重合^[62]。设 $O\text{-}XYZ$ 依次绕自身 X 轴、 Y 轴、 Z 轴分别逆时针转 θ_1 、 θ_2 、 θ_3 , 则 P 在这两个坐标系中的描述关系如下:

$$\begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix} = \begin{bmatrix} \cos \theta_3 & \sin \theta_3 & 0 \\ -\sin \theta_3 & \cos \theta_3 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta_2 & 0 & -\sin \theta_2 \\ 0 & 1 & 0 \\ \sin \theta_2 & 0 & \cos \theta_2 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_1 & \sin \theta_1 \\ 0 & -\sin \theta_1 & \cos \theta_1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (3.40)$$

在整个定位建图过程中涉及多个坐标系的转换, 同时为了提高精度, 需要对坐标转换结果做滤波处理, 为了简化开发, 使用 ROS 系统的 tf 变换工具完成各传感器坐标的

转换和滤波。利用 tf 在 launch 文件中发布相机坐标系和车身坐标系的坐标转换关系，以及车身坐标系与全局坐标系关系，然后利用组合惯导消息发布车身坐标系与全局坐标系的实时坐标转换关系，从而将锥桶在相机坐标系下的坐标转换到全局坐标系，三大坐标系关系如图 3.25 所示。

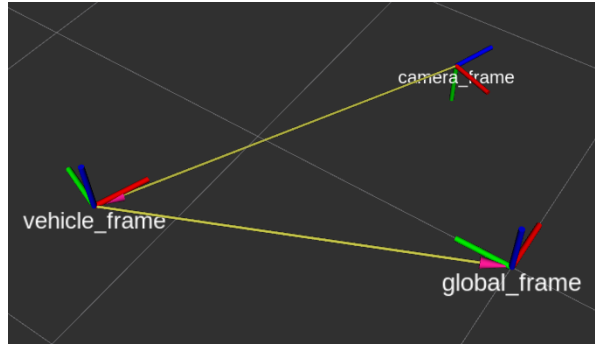


图 3.25 三大坐标系关系

3.4.2 数据同步与边界扩展

由于相机(30Hz)和组合惯导(100Hz)工作频率不同，需要对两者消息进行同步处理。在 ROS 系统中可以使用多传感器时间同步机制 Time Synchronizer 实现，传感器同步机制流程如图 3.26 所示：

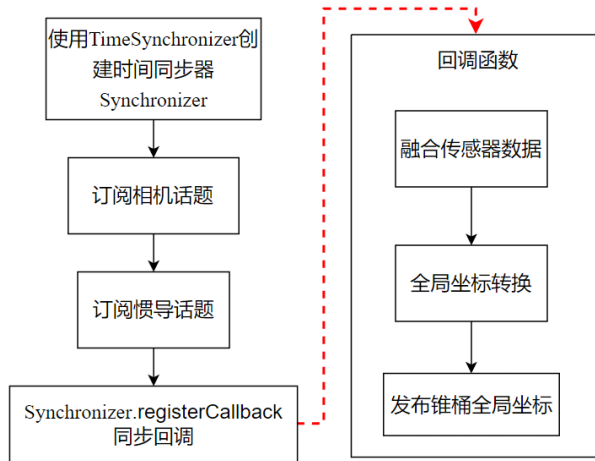


图 3.26 传感器同步流程图

完成传感器同步后即可建立赛道地图，但赛车在弯道行驶的情况下，由于相机拍摄角度受限，仅能看到外侧道路边界，如图 3.27(a)所示，此刻赛车仅对外侧赛道建图，如图 3.76(b)所示。



图 3.27 弯道建图结果

仅有外侧边界的建图结果，车辆难以完成路径规划，因此需要根据融合后的锥桶坐标扩展内侧赛道边界。扩展方法为：以检测到的外侧锥桶为主边界，计算主边界点间的向量并做归一化处理，然后计算归一化向量的法向量，沿法向量向内平移 3m 得到内测边界点，图 3.28 为扩展内测边界后的建图结果，建图结果表明，扩展内测边界后，赛车可在弯道建立完整地图。重复上述建图操作，并拼接各时刻对直道，弯道的建图结果即可得到赛道的全局地图。

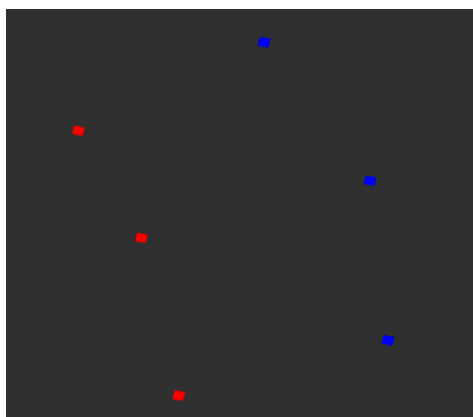


图 3.28 扩展内边界后的弯道建图结果

3.5 本章小结

本章利用双目相机和组合惯导完成锥桶赛道的建图。首先，分析双目视觉原理，以及立体匹配算法原理，在完成相机标定后，结合目标检测结果获取锥桶的世界坐标。然后，对组合惯导进行标定，获取赛车的定位信息。最后，根据赛车的定位信息和锥桶世界坐标实现了锥桶赛道的建图。

第四章 高速循迹赛道的轨迹规划算法

在完成锥桶赛道建图后，首先根据每帧的赛道地图进行局部路径规划。完成首圈行驶后，根据赛车行驶的路径求解全局参考线，再结合赛车的自车定位信息进行高速循迹赛道的轨迹规划，输出一条符合赛车运动学、动力学约束且光滑的轨迹。本章研究道路中心线求解算法、路径平滑算法和高速循迹赛道上的轨迹规划算法，最后在仿真环境下验证算法的可行性。

4.1 基于三次样条插值的路径平滑

赛车进行动态赛项目的首圈，根据赛道地图求解当前帧的赛道中心点，并利用三次样条插值拟合出赛道中线完成局部路径规划。在完成首圈行驶后，利用赛车的行驶路径计算全局参考线，并对参考线做平滑处理，保证高速轨迹规划的有效性。本节将研究锥桶赛道中心线求解算法和基于三次样条插值的路径平滑算法。

4.1.1 获取赛道中心线

高速循迹赛道按照组委会所设计的赛道路线形状和宽度构建，具有很强的随机性，尤其在弯道处，道路曲率不具连续性且感知得到的锥桶坐标为两组非对称分布的离散点，因此需要在随机的二维散点集内找到正确的道路中心点。Delaunay 三角剖分算法能对二维平面的点集进行剖分，生成由端点间的连线构成的封闭三角网，该三角网具备唯一性，如图 4.1 所示，任意三角形的外接圆范围内与 T2 关联的圆周内部不包含任何一点。

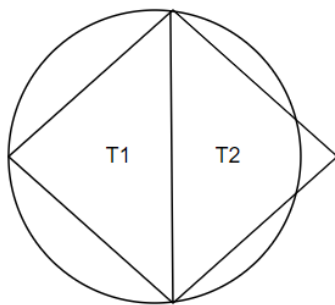


图 4.1 Delaunay 三角

利用 Delaunay 三角剖分的唯一性处理锥桶坐标点集，在生成三角网后寻找内三角边的中点即可求得赛道中心点。生成道路中心线流程如图 4.2 所示：

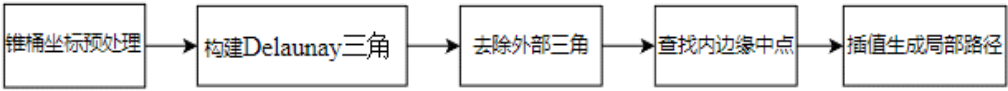


图 4.2 道路中心线求解流程

首先获取已排序的左右边界的锥桶坐标，确定两边的起点坐标，将坐标按如图 4.3 所示数据处理格式排列。

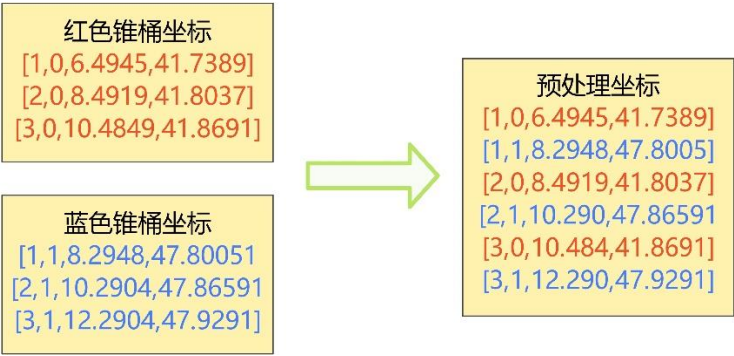


图 4.3 锥桶数据预处理

由于在道路采集时速度慢，为了避免重复采样，因此使用每帧检测的 6 个锥桶坐标创建 Delaunay 三角，定义包含红色与蓝色锥桶起始点的连线为有效三角边，创建 Delaunay 内三角，创建过程示意图如图 4.4 所示。但 Delaunay 三角剖分会在弯道边界外创建图 4.4 中黄色外三角。

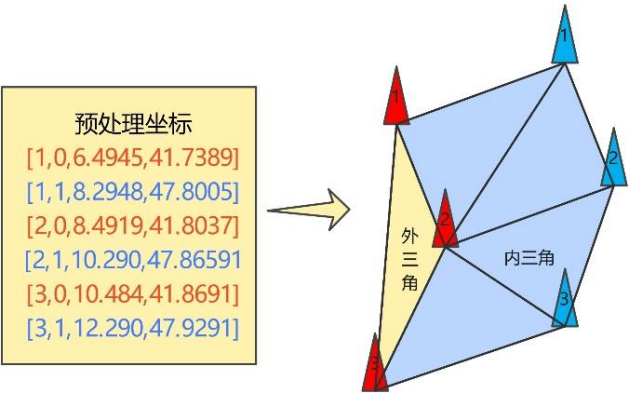


图 4.4 Delaunay 三角创建示意图

外三角不属于道路中心，在建图时会生成错误的路径，因此需要剔除外三角。由图 4.4 可知，外三角的顶点均为同一种颜色，因此在创建完 Delaunay 三角后，剔除三个顶点均为同种类别的三角形，就可以剔除外三角，如图 4.5(a)所示。最后寻找剩余三角形内边中点即可得到图 4.5(b)中红色道路中心点。

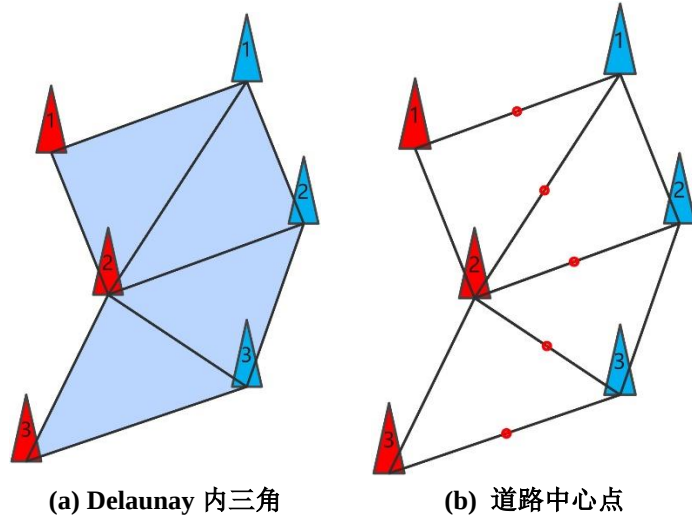


图 4.5 道路中点求解示意图

4.1.2 基于三次样条插值的平滑算法

由图 4.5(b)可知赛道中心点为长距离的离散点，需要拟合离散点求解密集的局部路径。由于本文以赛车建图时的定位信息作为参考线，而组合惯导的输出不平滑，因此也需要对参考线做平滑处理。经过三次样条插值法平滑的曲线具有二阶导数连续的性质，因此本文采用三次样条插值对道路中心线和参考路径做平滑处理^[91]。

假设有 n 个散点，如果仅只用一条高阶多项式曲线拟合多个散点，会发生龙格现象，在端点处出现震荡，如图 4.6 所示，这与路径规划的目标不符。

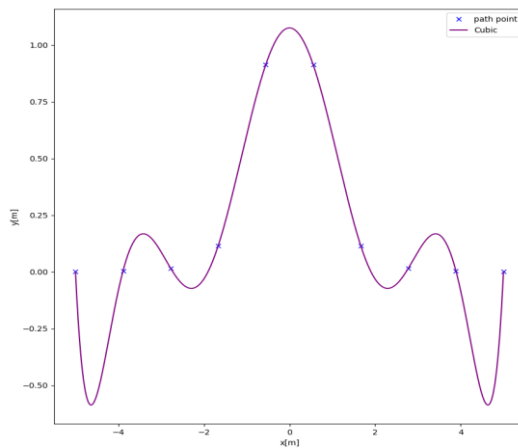


图 4.6 龙格现象

n 个坐标散点对应有 $n-1$ 个区间，因此可以将求解一条连续的高阶多项式转换为求解 $n-1$ 段三次曲线，设某段三次曲线表达式为：

$$y_i = f_i(x) = a_{i,0} + a_{i,1}x + a_{i,2}x^2 + a_{i,3}x^3 \quad (4.1)$$

其中 i 的取值范围为 1 到 $n-1$, $a_{i,0}, a_{i,1}, a_{i,2}, a_{i,3}$ 表示三次多项式各阶次对应的参数值。式中每个多项式 y_i 有 4 个参数, 因此为了求解每一段插值的参数值需要 $4(n-1)$ 个独立方程组, 可通过边界条件来确定。

(1) 由相邻两个区间曲线交点值相等, 可得 $2(n-1)$ 个边界条件, 表达式如下:

$$\begin{cases} f_i(x_i) = a_{i,0} + a_{i,1}x_i + a_{i,2}x_i^2 + a_{i,3}x_i^3 = y_i \\ f_i(x_{i+1}) = a_{i,0} + a_{i,1}x_{i+1} + a_{i,2}x_{i+1}^2 + a_{i,3}x_{i+1}^3 = y_{i+1} \end{cases} \quad (4.2)$$

式中 $f_i(x_i)$ 表示第 i 条插值曲线在 x_i 处的函数值, y_i 表示第 i 个插值点处的 y 坐标值。

(2) 相邻曲线交点导数连续, 保证交点处的一阶导相等, 可得 $(n-2)$ 个边界条件, 表达式如下:

$$\begin{aligned} f'_i(x_{i+1}) &= f'_{i+1}(x_{i+1}) \\ \Rightarrow a_{i,1} + 2a_{i,2}x_{i+1} + 3a_{i,3}x_{i+1}^2 &= a_{i+1,1} + 2a_{i+1,2}x_{i+1} + 3a_{i+1,3}x_{i+1}^2 \end{aligned} \quad (4.3)$$

(3) 曲线交点处二阶导相等, 保证整条曲线曲率的连续性可得 $(n-2)$ 个边界条件, 表达式如下:

$$f''_i(x_{i+1}) = f''_{i+1}(x_{i+1}) \Rightarrow 2a_{i,2} + 6a_{i,3}x_{i+1} = 2a_{i+1,2} + 6a_{i+1,3}x_{i+1}. \quad (4.4)$$

(4) 对于首末两个端点使其端点处的二阶导数为 0, 满足自然边界条件可得 2 个边界条件, 表达式如下:

$$f''_1(x_1) = f''_{n-1}(x_n) = 0 \Rightarrow 2a_{1,2} + 6a_{1,3}x_1 = 2a_{n-1,2} + 6a_{n-1,3}x_n = 0 \quad (4.5)$$

根据这四种约束总共获得 $4(n-1)$ 个边界条件, 联立方程求解, 可以唯一确定各段三次曲线的四个参数 $a_{i,0}, a_{i,1}, a_{i,2}, a_{i,3}$, 在每一段空间内进行插值即可生成光滑的曲线。如图 4.7 所示, 采用三次样条插值法对散点的拟合曲线十分光滑。

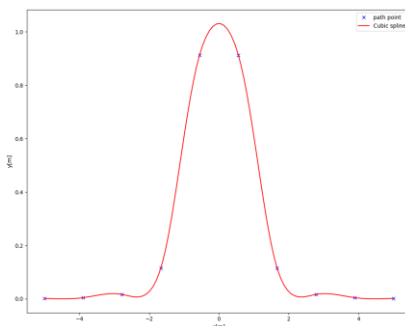


图 4.7 三次样条插值曲线

4.1.3 道路中心线与参考线平滑

赛车在建立赛道地图时，可直接使用三次插值多项式对赛道中心点拟合得到赛道中心线，将多帧赛道中心线拼接后得到全局赛道中心线，赛道中心线仿真结果如图 4.8 所示：

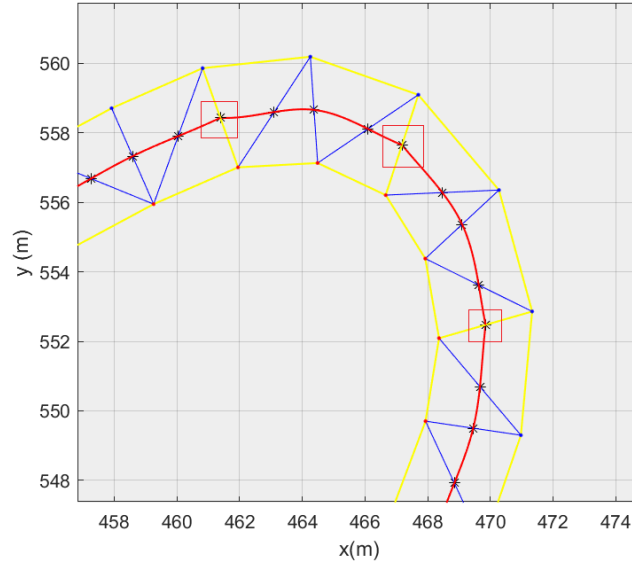


图 4.8 赛道中心线仿真图

图 4.8 所示在某帧内的道路中心线平滑，因此经过离散后就可得到首圈建图时的规划路径。但是在帧与帧拼接处（红色方框位置）存在明显突变，这会导致高速循迹行驶时车辆不稳定。对此，本文对赛车首圈的行驶路径进行采样，利用三次样条插值平滑采样点求解轨迹规划的参考线。在实际工程中使用的参考线为离散点，因此需要对参考线按照等弧长原则进行离散（本文离散距离 $s=0.1\text{m}$ ）。参考线在离散区间内可将两点之间的斜率近似当作参考线的航向角，航向角 θ 计算公式为：

$$\tan(\theta) = \frac{dx}{dy} = \frac{dx/ds}{dy/ds} \quad (4.6)$$

根据散点计算参考线的曲率：

$$\kappa = \frac{|x'y'' - y'x''|}{(x'^2 + y'^2)^{\frac{3}{2}}} \quad (4.7)$$

其中 $x' = dx/ds$ ， $x'' = d^2x/ds^2$ 。由式(4.6)、(4.7)可得参考线离散点 $[x_r, y_r, \theta_r, \kappa_r]$ 。

4.2 Lattice 动态轨迹规划算法

Lattice 轨迹规划算法是在 Frenet 坐标系下进行采样的局部轨迹规划算法。算法将车辆的状态空间栅格化,并根据不同的场景和工况配置采样参数后进行轨迹规划,直接向控制器输出一条光滑且满足车辆运动学和动力学约束的最优规划轨迹,实现对赛车行驶速度、加速度、方向等的精确控制,由于比赛赛道为单车道且不存在其他车辆,不需要进行碰撞检测,因此本文轨迹规划算法流程为:

- 1、算法首先获取经三次插值平滑并离散化后的参考线和自车定位信息,确定赛车各个规划周期内轨迹规划起始点。
- 2、计算赛车行车状态,如果赛车首次启动,那么以赛车起始点作为轨迹规划起点,计算规划起点在 Frenet 坐标系下的投影点,以此获得规划点在 Frenet 坐标系下的初始状态值。若在行车过程中,则需要根据当前位置寻找前瞻点作为轨迹规划起点,并计算其初始状态值。
- 3.分别对横向和纵向的车辆行驶状态采样,获得一系列轨迹规划末状态,利用多组始末状态求解规划函数,从而得到横纵向规划轨迹簇。
- 4.对获得的规划轨迹簇进行运动学和动力学约束检查,剔除不符合规范的轨迹。
- 5.最后根据时间合成横纵向轨迹簇,并计算规划轨迹代价值,将代价值最小的轨迹转换到全局坐标系下输出,该轨迹即为赛车当前规划周期的最优规划轨迹输出。

4.2.1 Frenet 坐标与直角坐标转换

赛车的定位与参考线均在直角坐标系下表示,而赛车的运动轨迹在直角坐标系下的描述却十分复杂,因为大部分赛道并非笔直,难以确定赛车与道路中心的偏移量以及车辆的行驶距离。若赛车在图 4.9 所示的 Frenet 坐标系下做轨迹规划,其纵向位移 s 与横向位移 l 均为线性变化,能够方便的计算车辆相对道路中心和边界的距离,提高轨迹规划效率。

如图 4.9 所示,Frenet 坐标系根据参考线的切向量和法向量建立,原点固定在车身,坐标轴相互垂直,沿参考线方向为纵轴(s),垂直于参考线方向表示横轴(l)。

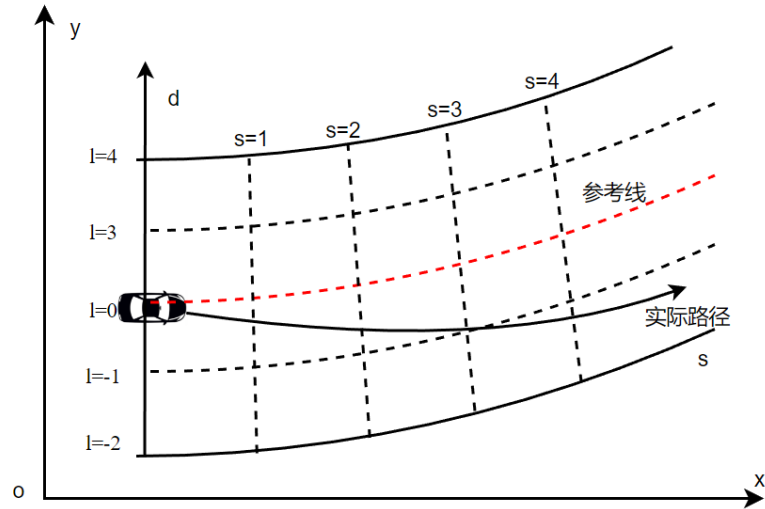


图 4.9 Frenet 坐标系

为了在 Frenet 坐标系下做动态轨迹规划, 需要将直角坐标系下的定位坐标转换为 Frenet 坐标。在完成轨迹规划后, 需要将 Frenet 坐标系下轨迹转换到全局直角坐标系下。

Frenet 坐标与直角坐标间的相互转换如图 4.10 所示, 解析组合惯导信息后得到的车辆定位状态信息为 $[\vec{x}, \theta_x, \kappa_x, v_x, a_x]$, 其中 $\vec{x}$ 为车辆在直角坐标系下的实时坐标 (x, y) 的向量表示, θ_x 为赛车当前的航向角, κ_x 为赛车当前轨迹的曲率, v_x 为当前位置的线速度, a_x 为当前位置的加速度。车辆在 Frenet 坐标系下的状态为 (s, l) , 分别表示车辆当前位置相对于参考线起点的纵向位移以及相对参考线的横向位移。

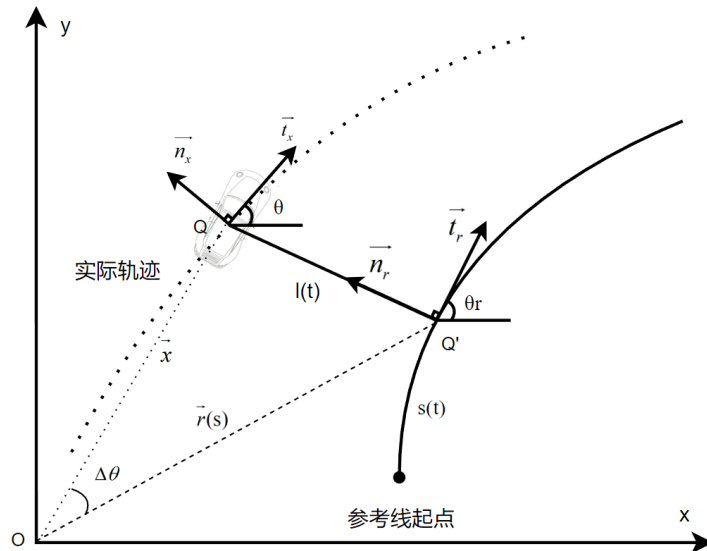


图 4.10 Frenet 坐标系与直角坐标系的转换

图 4.10 中 $\vec{n}_x$ 、 $\vec{t}_x$ 为车辆当前位置 Q 的正交单位向量; $\vec{t}_r$ 、 $\vec{n}_r$ 为参考线上投影点 Q' 处的正交单位向量, $\vec{r}(s)$ 为 Q' 点在全局坐标系下的位置向量, 记 $\vec{r}(s) = (x_r, y_r)$ 。

在 t 时刻将当前位置投影到参考线上可得车辆在 Frenet 坐标系的状态 $[s, \dot{s}, \ddot{s}, l, \dot{l}, \ddot{l}]$ 。其中： s 为纵向位移， $\dot{s}$ 为纵向速度， $\ddot{s}$ 为纵向加速度， l 为横向位移， $\dot{l}$ 为横向速度， $\ddot{l}$ 为横向加速度。此时投影点 Q' 与 Q 点的横向距离为：

$$l = \pm \sqrt{(x - x_r)^2 + (y - y_r)^2} \quad (4.8)$$

若 $(y - y_r) \cos \theta_r - (x - x_r) \sin \theta_r > 0$ ，则 l 取正号，否则 l 取负号。

根据图 4.10 中向量关系可得横向位移 $l = [\bar{x} - \vec{r}(s)]^T \cdot \vec{n}_r$ 。因此横向速度 $\dot{l}$ 为：

$$\dot{l} = [\dot{\bar{x}} - \dot{\vec{r}}(s)]^T \cdot \vec{n}_r + [\bar{x} - \vec{r}(s)]^T \cdot \dot{\vec{n}}_r = v_x \vec{t}_x \vec{n}_r = v_x \sin \Delta \theta \quad (4.9)$$

式中： $-\frac{\pi}{2} \leq \Delta \theta = \theta_x - \theta_r \leq \frac{\pi}{2}$ ， $\dot{\vec{n}}_r = -\kappa_r \vec{t}_r$ 。

由上式可得 $\bar{x}$ 的导数为：

$$\dot{\bar{x}} = \frac{d}{dt}(\vec{r}(s) + l \cdot \vec{n}_r) = \dot{s}(1 - \kappa_r l) \cdot \vec{t}(r) + \dot{l} \cdot \vec{n}_r \quad (4.10)$$

$\dot{\bar{x}}$ 的模即为速度 v_x ，由上式可得：

$$v_x = \|\dot{\bar{x}}\| = \left\| \begin{bmatrix} \vec{t}_r & \vec{n}_r \end{bmatrix} \begin{pmatrix} 1 - \kappa_r l & 0 \\ 0 & 1 \end{pmatrix} \begin{bmatrix} \dot{s} \\ \dot{l} \end{bmatrix} \right\| = \left\| \begin{pmatrix} 1 - \kappa_r l & 0 \\ 0 & 1 \end{pmatrix} \begin{bmatrix} \dot{s} \\ \dot{l} \end{bmatrix} \right\| = \sqrt{[1 - \kappa_r l]^2 \dot{s}^2 + \dot{l}^2} \quad (4.11)$$

综合式(4.9)、式(4.11)可得：

$$\theta_x = \theta_r + \arcsin \left(\frac{\dot{l}}{\sqrt{[1 - \kappa_r l]^2 \dot{s}^2 + \dot{l}^2}} \right) \quad (4.12)$$

相邻两帧速度的变化率表示加速度 a_x ，即：

$$a_x = f[v_x(t) - v_x(t - \frac{1}{f})] \quad (4.13)$$

式中 f 为规划算法执行频率。

取 Frenet 坐标系下三个连续坐标点做相邻坐标点连线的垂直平分线，两线交点即为轨迹的外切圆圆心，如图 4.11 所示。

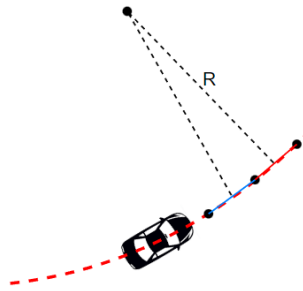


图 4.11 Frenet 坐标系下的曲率计算

根据圆心计算外切圆半径 R ，可得曲率 κ_v ：

$$\kappa_x = \frac{1}{R} \quad (4.14)$$

在实现直角坐标系与 Frenet 坐标系转换后可以将赛车在全球坐标系的二维运动解耦为纵向和横向两个一维方向的运动，然后分别进行动态轨迹规划。

赛车进行动态轨迹规划首要任务是确定当前规划周期的轨迹规划起始点在 Frenet 坐标系下的初始状态。对于刚起步的赛车而言，轨迹规划的起始点就是当前赛车的定位点。而在其他时刻则需要根据规划算法频率寻找前瞻点作为轨迹规划起始点。对于于轨迹的规划频率来说，频率不能过高，过高会使控制器没有足够反应时间完成轨迹跟踪；如果规划频率太低会导致系统对动态环境变化的响应减缓，无法在短时间内完成对行驶路径的调整。本文设置规划更新频率为 $f = 10\text{Hz}$ 。由于轨迹规划需要保证输出轨迹的连续与平滑，因此在寻找下一帧规划起点时需要考虑当前车辆的位置与轨迹信息。图 4.12 为初始状态求解示意图。

图 4.12 规划初始状态求解

向前搜索一周期的轨迹作为赛车在规划算法执行时间内控制器跟踪的轨迹,该轨迹末端点为前瞻点 R ,即当前规划周期的轨迹规划起始点。将 R 点作为轨迹规划点不仅使赛车在轨迹规划期间受控,还增强了规划时帧与帧之间轨迹和参考线更新的连续性。

在找到规划起始点后,通过寻找起始点在参考线上的投影点计算轨迹规划初始状态。但由于参考线是离散点集,因此规划起点的投影点并不一定与参考线点一一对应。为了求解起始点的投影点,首先搜索参考线上距离 R 最短的匹配点 R' ,然后查找前后与 R' 相邻的 R_0, R_1 点,利用向量点积原理求解投影点。根据图 4.12 可得:

$$\begin{cases} \vec{v}_0 = \vec{R}_0 R = (x_{R_0} - x_R, y_{R_0} - y_R) \\ \vec{v}_1 = R_0 R_1 = (x_{R_0} - x_{R_1}, y_{R_0} - y_{R_1}) \end{cases} \quad (4.15)$$

根据向量点积原理可得:

$$\vec{v}_0 \cdot \vec{v}_1 = \|\vec{v}_0\| \cdot \|\vec{v}_1\| \cdot \cos(\vec{v}_0, \vec{v}_1) = (x_p - x_{p_0})(x_{p_1} - x_{p_0}) + (y_p - y_{p_0})(y_{p_1} - y_{p_0}) \quad (4.16)$$

由于对参考线的采样间隔很小,相邻两点的弧长可由弦长代替,则投影点 R'' 到 R_0 的曲线长度 Δs 为:

$$\Delta s = \frac{\vec{v}_0 \cdot \vec{v}_1}{\|\vec{v}_1\|} = \frac{(x_R - x_{R_0})(x_{R_1} - x_{R_0}) + (y_R - y_{R_0})(y_{R_1} - y_{R_0})}{\sqrt{(x_{R_1} - x_{R_0})^2 + (y_{R_1} - y_{R_0})^2}} \quad (4.17)$$

利用式(4.17),在 R_0, R_1 之间做线性插值即可求解投影点 R'' 的轨迹 $[x, y, \theta, \kappa, \kappa', s]$ 。

设线性插值粒度 $w = \Delta s / (s_1 - s_0)$, 则有:

$$\begin{cases} x = (1-w)x_{R_0} + wx_{R_1} \\ y = (1-w)y_{R_0} + wy_{R_1} \\ \theta = (1-w)\theta_{R_0} + w\theta_{R_1} \\ \kappa = (1-w)\kappa_{R_0} + w\kappa_{R_1} \\ \kappa' = (1-w)\kappa'_{R_0} + w\kappa'_{R_1} \\ s = s_{R_0} + \Delta s \end{cases} \quad (4.18)$$

由式(4.18)求得投影点 R'' 的轨迹信息,在根据 4.2.1 节直角坐标系到 Frenet 坐标系的计算方法,即可求得当前轨迹规划周期起始点 R 在 Frenet 坐标系下的初始状态向量 $[s_0, \dot{s}_0, \ddot{s}_0, l_0, \dot{l}_0, \ddot{l}_0]$ 。将该初始状态分解为纵向初始状态 $[s_0, \dot{s}_0, \ddot{s}_0]$ 和横向初始状态 $[l_0, \dot{l}_0, \ddot{l}_0]$ 。

4.3 横纵向轨迹规划

由于在 Frenet 坐标系下将赛车的运动解耦为横向和纵向运动，因此在确定赛车在 Frenet 坐标系下的初始状态后，分别对横向和纵向运动状态采样即可获得轨迹规划末状态。由当前规划周期内的始末状态即可求解多条规划轨迹，然后设计代价函数，计算各条轨迹的代价值，按照代价值大小排序后得到横纵轨迹规划簇。将横纵向轨迹合成后进行赛车运动学、动力学约束检查筛选不满足约束的轨迹，最后将代价值最小的轨迹转换到全局坐标系下，即得到当前规划周期的最优轨迹输出。本节将研究横纵向采样方法以及最优轨迹的选择。

4.3.1 横向采样轨迹规划

赛道不存在障碍物，横向规划的主要任务是过弯转向，保证赛车贴近赛道中心线行驶，因此横向轨迹规划可对赛车的横向位移采样得到横向轨迹规划末状态。设赛车在 t_0 时刻开始轨迹规划，赛车的初始状态已知，即 $[l_{(t_0)}, \dot{l}_{(t_0)}, \ddot{l}_{(t_0)}] = [l_0, \dot{l}_0, \ddot{l}_0]$ ，由于希望赛车贴近参考线行驶，因此采样后的横向速度，横向加速度均为 0，则在 t_1 时刻的横向采样末状态为 $[l_{(t_1)}, \dot{l}_{(t_1)}, \ddot{l}_{(t_1)}] = [l_1, 0, 0]$ ，其中 l_1 由横向位移的采样间隔确定。在获取横向规划始末状态后，利用 5 次多项式作为轨迹规划函数来描述横向位移与时间的关系 $l(t)$ 有：

$$l(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5 \quad (4.19)$$

对规划函数求一阶导可得横向轨迹的速度，表达式为：

$$\dot{l}(t) = a_1 + 2a_2 t + 3a_3 t^2 + 4a_4 t^3 + 5a_5 t^4 \quad (4.20)$$

对规划函数求二阶导可得横向轨迹的加速度，表达式为：

$$\ddot{l}(t) = 2a_2 + 6a_3 t + 12a_4 t^2 + 20a_5 t^3 \quad (4.21)$$

在 $t = t_0$ 时刻的初始状态可表示为：

$$l(t_0) = a_0 + a_1 t_0 + a_2 t_0^2 + a_3 t_0^3 + a_4 t_0^4 + a_5 t_0^5 \quad (4.22)$$

$$\dot{l}(t_0) = a_1 + 2a_2 t_0 + 3a_3 t_0^2 + 4a_4 t_0^3 + 5a_5 t_0^4 \quad (4.23)$$

$$\ddot{l}(t_0) = 2a_2 + 6a_3 t_0 + 12a_4 t_0^2 + 20a_5 t_0^3 \quad (4.24)$$

在 $t = t_1$ 时刻的末状态可表示为：

$$l(t_1) = a_1 + a_1 t_1 + a_2 t_1^2 + a_3 t_1^3 + a_4 t_1^4 + a_5 t_1^5 \quad (4.25)$$

$$\dot{l}(t_1) = a_1 + 2a_2 t_1 + 3a_3 t_1^2 + 4a_4 t_1^3 + 5a_5 t_1^4 \quad (4.26)$$

$$\ddot{l}(t_1) = 2a_2 + 6a_3 t_1 + 12a_4 t_1^2 + 20a_5 t_1^3 \quad (4.27)$$

式(4.22)~式(4.27)写成矩阵形式可得:

$$\begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 & t_0^4 & t_0^5 \\ 0 & 1 & 2t_0 & 3t_0^2 & 4t_0^3 & 5t_0^4 \\ 0 & 0 & 2 & 6t_0 & 12t_0^2 & 20t_0^3 \\ 1 & t_1 & t_1^2 & t_1^3 & t_1^4 & t_1^5 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 & 5t_1^4 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 & 20t_1^3 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \end{bmatrix} = \begin{bmatrix} l_{t_0} \\ \dot{l}_{t_0} \\ \ddot{l}_{t_0} \\ l_{t_1} \\ \dot{l}_{t_1} \\ \ddot{l}_{t_1} \end{bmatrix} \quad (4.28)$$

令 $\mathbf{A} = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \end{bmatrix}$, $\mathbf{T} = \begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 & t_0^4 & t_0^5 \\ 0 & 1 & 2t_0 & 3t_0^2 & 4t_0^3 & 5t_0^4 \\ 0 & 0 & 2 & 6t_0 & 12t_0^2 & 20t_0^3 \\ 1 & t_1 & t_1^2 & t_1^3 & t_1^4 & t_1^5 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 & 5t_1^4 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 & 20t_1^3 \end{bmatrix}$, $\mathbf{L} = \begin{bmatrix} l_{t_0} \\ \dot{l}_{t_0} \\ \ddot{l}_{t_0} \\ l_{t_1} \\ \dot{l}_{t_1} \\ \ddot{l}_{t_1} \end{bmatrix}$, 由于始末状态已知,

因此规划函数系数矩阵 $\mathbf{A}$ 为:

$$\mathbf{A} = \mathbf{T}^{-1}\mathbf{L} \quad (4.29)$$

将系数矩阵 $\mathbf{A}$ 带入式(4.19)即可求得横向规划轨迹。

由于轨迹规划输出规划时间内得轨迹点, 因此需要对规划函数进行离散更新。轨迹更新如图 4.13 所示:

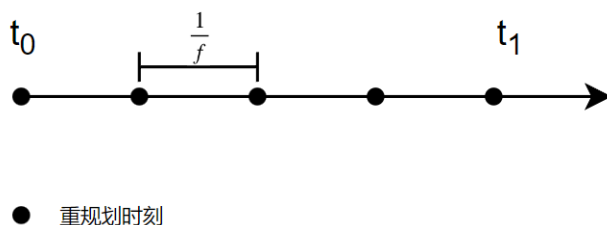


图 4.13 轨迹更新示意图

其中 f 为规划算法更新频率。

由于赛车为无人驾驶，轨迹规划不需要考虑下横向舒适性，仅注重赛车的横向偏移量，即候选横向轨迹偏离参考线的程度，因此横向轨迹的代价函数 cost_{lat} 设计如下：

$$\text{cost}_{lat} = \sum_{t=0}^{t=T} \left[\left(l(t)/l_b \right)^2 \right] / \sum_{t=0}^{t=T} \left[l(t)/l_b \right] \quad (4.30)$$

其中， T 为规划时间 $T = t_1 - t_0$ ； $l(t)$ 表示横向轨迹目标状态到参考线的法向距离， l_b 表示赛道边界到参考线的距离。

计算每条轨迹的代价后排序可得横向规划轨迹簇。如图 4.14 为赛车在赛道中心行驶时，以采样时间间隔 $\Delta t = 1\text{s}$ 和横向位移采样间隔 $\Delta l = 0.35\text{m}$ 时采样得到的横向规划轨迹簇。

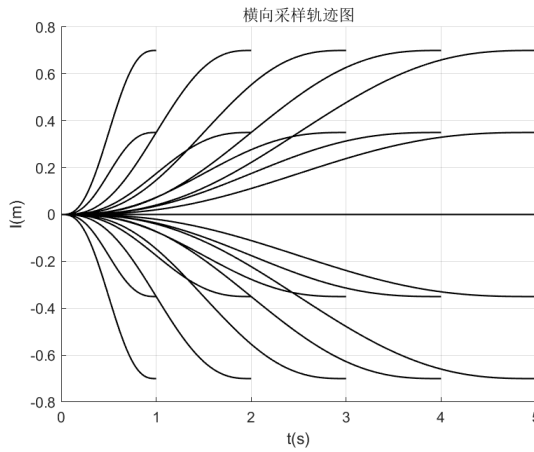


图 4.14 横向规划采样轨迹图

4.3.2 纵向采样轨迹规划

赛车的纵向轨迹规划主要是对赛车的纵向位移，速度和加速度进行规划。赛车的纵向行驶有两个工况，分别是起步加速到目标速度行驶的巡航工况以及从巡航车速减速直至停车的减速停车工况。在不同的工况下，纵向规划的期望配置不同，因此纵向规划需要对巡航工况和减速工况的行车状态进行单独采样。

1. 巡航工况

设赛车在 t_0 时刻纵向初始状态 $[s_{(t_0)}, \dot{s}_{(t_0)}, \ddot{s}_{(t_0)}] = [s_0, \dot{s}_0, \ddot{s}_0]$ 已知。在巡航工况下期望赛车按照目标速度匀速行驶，而纵向位移无限制，因此仅需对纵向速度进行采样即可得到末状态，赛车巡航行驶时的加速度为 0，因此 t_1 时刻纵向采样末状态 $[s_{(t_1)}, \dot{s}_{(t_1)}, \ddot{s}_{(t_1)}] = [s_1, \dot{s}_1, \ddot{s}_1] = [s_1, \dot{s}_1, 0]$ 。由于不关注纵向位移，因此可利用四次多项式作为轨迹规划函数来描述纵向位移与时间的关系 $s(t)$ ，则巡航工况规划函数表示为：

$$s(t) = b_0 + b_1 s + b_2 s^2 + b_3 s^3 + b_4 s^4 \quad (4.31)$$

对纵向规划函数求一阶导可得纵向速度，表达式为：

$$\dot{s}(t) = b_1 + 2b_2 t + 3b_3 t^2 + 4b_4 t^3 \quad (4.32)$$

对纵向规划函数求二阶导可得纵向加速度，表达式为：

$$\ddot{s}(t) = 2b_2 + 6b_3 t + 12b_4 t^2 \quad (4.33)$$

在 $t = t_0$ 时刻的初始状态可表示为：

$$s(t_0) = b_0 + b_1 t_0 + b_2 t_0^2 + b_3 t_0^3 + b_4 t_0^4 \quad (4.34)$$

$$\dot{s}(t_0) = b_1 + 2b_2 t_0 + 3b_3 t_0^2 + 4b_4 t_0^3 \quad (4.35)$$

$$\ddot{s}(t_0) = 2b_2 + 6b_3 t_0 + 12b_4 t_0^2 \quad (4.36)$$

在 $t = t_1$ 时刻的末状态可表示为：

$$\dot{s}(t_1) = b_1 + 2b_2 t_1 + 3b_3 t_1^2 + 4b_4 t_1^3 \quad (4.37)$$

$$\ddot{s}(t_1) = 2b_2 + 6b_3 t_1 + 12b_4 t_1^2 \quad (4.38)$$

式(4.34)~式(4.38)写成矩阵形式可得：

$$\begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 & t_0^4 \\ 0 & 1 & 2t_0 & 3t_0^2 & 4t_0^3 \\ 0 & 0 & 2 & 6t_0 & 12t_0^2 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix} = \begin{bmatrix} s_{t_0} \\ \dot{s}_{t_0} \\ \ddot{s}_{t_0} \\ \dot{s}_{t_1} \\ \ddot{s}_{t_1} \end{bmatrix} \quad (4.39)$$

$$\text{令 } \mathbf{B} = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 & t_0^4 \\ 0 & 1 & 2t_0 & 3t_0^2 & 4t_0^3 \\ 0 & 0 & 2 & 6t_0 & 12t_0^2 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 \end{bmatrix}, \quad \mathbf{S} = \begin{bmatrix} s_{t_0} \\ \dot{s}_{t_0} \\ \ddot{s}_{t_0} \\ \dot{s}_{t_1} \\ \ddot{s}_{t_1} \end{bmatrix}, \quad \text{由于始末状态已知, 可得规}$$

划函数系数矩阵 $\mathbf{B}$ ：

$$\mathbf{B} = \mathbf{T}^{-1} \mathbf{S} \quad (4.40)$$

将系数矩阵 $\mathbf{B}$ 带入式(4.31)即可求得纵向运动轨迹。图 4.15 为赛车起步时，以采样时间间隔 $\Delta t = 1\text{s}$ 、速度采样间隔 $\Delta \dot{s} = 2.8\text{m/s}$ 时采样得到的纵向规划轨迹簇。

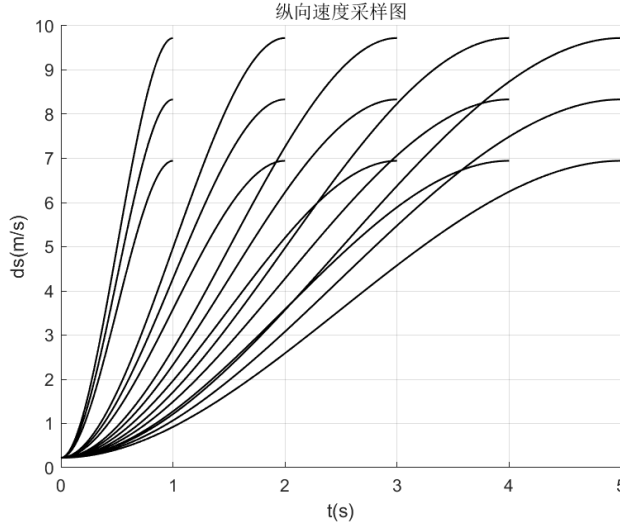


图 4.15 巡航工况纵向速度采样轨迹

2. 减速停车工况

若在 t_0 时刻赛车检测到停止点则需要让赛车在接下来的规划周期内规划一条轨迹控制赛车减速停车。因此相比巡航工况来说，减速工况仅需要考虑纵向位移的约束，在 t_1 时刻对纵向位移上进行采样即可得纵向轨迹规划末状态，对于减速停车工况来说，希望停车时的速度和加速度均为 0，因此 t_1 时刻的末状态 $[s_{(t_1)}, \dot{s}_{(t_1)}, \ddot{s}_{(t_1)}] = [s_1, \dot{s}_1, \ddot{s}_1] = [s_1, 0, 0]$ ，为了保证规划轨迹平滑，采用五次多项式作为规划函数描述纵向位置与时间的关系，则停车工况规划函数 $s(t)$ 为：

$$s(t) = b_0 + b_1 t + b_2 t^2 + b_3 t^3 + b_4 t^4 + b_5 t^5 \quad (4.41)$$

对纵向规划函数求一阶导可得纵向速度，表达式为：

$$\dot{s}(t) = b_1 + 2b_2 t + 3b_3 t^2 + 4b_4 t^3 + 5b_5 t^4 \quad (4.42)$$

对纵向规划函数求二阶导可得纵向加速度，表达式为：

$$\ddot{s}(t) = 2b_2 + 6b_3 t + 12b_4 t^2 + 20b_5 t^3 \quad (4.43)$$

在 $t = t_0$ 时刻的初始状态可表示为：

$$s(t_0) = b_0 + b_1 t_0 + b_2 t_0^2 + b_3 t_0^3 + b_4 t_0^4 + b_5 t_0^5 \quad (4.44)$$

$$\dot{s}(t_0) = b_1 + 2b_2 t_0 + 3b_3 t_0^2 + 4b_4 t_0^3 + 5b_5 t_0^4 \quad (4.45)$$

$$\ddot{s}(t_0) = 2b_2 + 6b_3 t_0 + 12b_4 t_0^2 + 20b_5 t_0^3 \quad (4.46)$$

在 $t=t_1$ 时刻的末状态可表示为:

$$s(t_1) = b_1 + b_1 t_1 + b_2 t_1^2 + b_3 t_1^3 + b_4 t_1^4 + b_5 t_1^5 \quad (4.47)$$

$$\dot{s}(t_1) = b_1 + 2b_2 t_1 + 3b_3 t_1^2 + 4b_4 t_1^3 + 5b_5 t_1^4 \quad (4.48)$$

$$\ddot{s}(t_1) = 2b_2 + 6b_3 t_1 + 12b_4 t_1^2 + 20b_5 t_1^3 \quad (4.49)$$

式(4.44)~式(4.49)写成矩阵形式可得:

$$\begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 & t_0^4 & t_0^5 \\ 0 & 1 & 2t_0 & 3t_0^2 & 4t_0^3 & 5t_0^4 \\ 0 & 0 & 2 & 6t_0 & 12t_0^2 & 20t_0^3 \\ 1 & t_1 & t_1^2 & t_1^3 & t_1^4 & t_1^5 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 & 5t_1^4 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 & 20t_1^3 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \end{bmatrix} = \begin{bmatrix} s_{t_0} \\ \dot{s}_{t_0} \\ \ddot{s}_{t_0} \\ s_{t_1} \\ \dot{s}_{t_1} \\ \ddot{s}_{t_1} \end{bmatrix} \quad (4.50)$$

令 $\mathbf{B} = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \end{bmatrix}$, $\mathbf{T} = \begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 & t_0^4 & t_0^5 \\ 0 & 1 & 2t_0 & 3t_0^2 & 4t_0^3 & 5t_0^4 \\ 0 & 0 & 2 & 6t_0 & 12t_0^2 & 20t_0^3 \\ 1 & t_1 & t_1^2 & t_1^3 & t_1^4 & t_1^5 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 & 5t_1^4 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 & 20t_1^3 \end{bmatrix}$, $\mathbf{S} = \begin{bmatrix} s_{t_0} \\ \dot{s}_{t_0} \\ \ddot{s}_{t_0} \\ s_{t_1} \\ \dot{s}_{t_1} \\ \ddot{s}_{t_1} \end{bmatrix}$, 由于始末状态已知,

可得规划函数系数矩阵 $\mathbf{B}$:

$$\mathbf{B} = \mathbf{T}^{-1}\mathbf{S} \quad (4.51)$$

将系数矩阵 $\mathbf{B}$ 带入式(4.41)可求得停车工况的纵向运动轨迹。设赛车的停车点位于 $s=160$ 处, 仅需对时间进行采样可得到一簇候选纵向轨迹, 如图 4.16 所示:

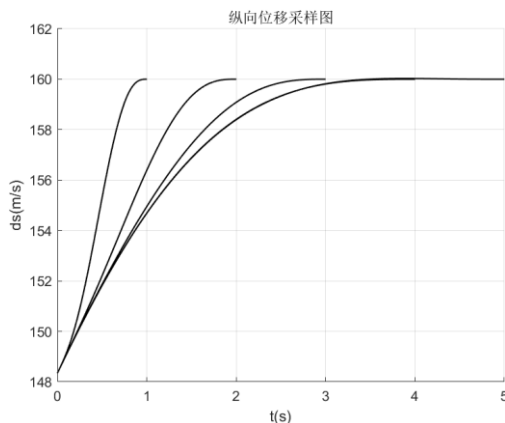


图 4.16 停车工况纵向位移采样轨迹

得到纵向规划轨迹簇后，对轨迹离散处理。赛车在纵向行驶也不需要考虑舒适性，因此对于纵向轨迹代价由纵向行驶距离、速度差和向心加速度决定。其中纵向行驶距离代价函数 cost_s 评价赛车的行驶距离，表达式为：

$$\text{cost}_s = \frac{1}{1 + \text{dist}(T)} \quad (4.52)$$

式中 $T = t_1 - t_0$ 表示规划时间， $\text{dist}(T)$ 为规划时间内纵向行驶的距离。

纵向速度差代价函数 cost_v 评价当前车速与目标车速接近程度，表达式为：

$$\text{cost}_v = \sum_{t=0}^{t=T} (t^2 \cdot |v_{ref} - v(t)|) / \sum_{t=0}^{t=T} t^2 \quad (4.53)$$

式中 v_{ref} 为赛车的巡航速度， $v(t)$ 为规划轨迹的纵向车速。

向心加速度代价函数 cost_a 评价赛车转弯的效率，表达式为：

$$\text{cost}_a = \sum_{t=0}^{t=T} (v^2(t) \cdot \kappa_{ref})^2 / \sum_{t=0}^{t=T} |v^2(t) \cdot \kappa_{ref}| \quad (4.54)$$

式中 κ_{ref} 表示规划轨迹的曲率。

为三个评价指标赋予不同的权重即可得纵向轨迹的总代价函数 cost_{lon} 为：

$$\text{cost}_{lon} = w_s \cdot \text{cost}_s + w_v \cdot \text{cost}_v + w_a \cdot \text{cost}_a \quad (4.55)$$

比赛为了取得好成绩，希望赛车遵守以下规则：首先要保证赛车平稳过弯，在弯道出减速行驶。其次巡航速度与规划速度相差越小越好；最后希望在一个规划周期内通过距离越长越好。由多次实验得到各项权重取值 $w_a = 3$ ， $w_v = 2$ ， $w_s = 1$ 。根据各项权重计算每条候选轨迹的代价值并排序，可得纵向轨迹规划候选轨迹簇。

4.3.3 筛选最优轨迹

因为横向和纵向轨迹规划出的轨迹簇在时间上一一对应，因此可以根据时间 t 将横纵向轨迹合成。并根据横纵向轨迹代价函数 cost_{lat} ， cost_{lon} 计算每条合成轨迹的代价值。合成轨迹的代价函数 cost_{all} 为：

$$\text{cost}_{all} = \text{cost}_{lat} + \text{cost}_{lon} \quad (4.56)$$

根据合成轨迹的代价函数计算合成轨迹的代价值，并按照代价值从小到大排序。然后依次进行赛车运动学和辆动力学约束检查。

赛车的设计参数如下：赛车固有曲率值为 0.4m^{-1} ，纵向速度在实际比赛时设置 30km/h 的巡航速度，纵向加速度最大值为 0.5g m/s^2 ，赛车横向加速度最大值为 0.6m/s^2 。对于任何一条轨迹的所有离散轨迹点来说，其曲率，纵向速度，纵向加速度，横向加速度都不能大于赛车固有参数。完成运动学、动力学约束检查即可输出代价最小的合成轨迹，根据式(4.8)~式(4.14)即可得到全局坐标系下的轨迹。

4.4 轨迹规划算法仿真验证

为了验证本章赛道中心线及轨迹规划的有效性，本节将在 `matlab` 环境中验证道路中心线生成算法，在 `python` 环境中仿真验证轨迹规划算法。

4.4.1 赛道中心线求解算法仿真验证

首先设计如图 4.17 所示锥桶赛道边界的坐标点，如图 4.17 所示：

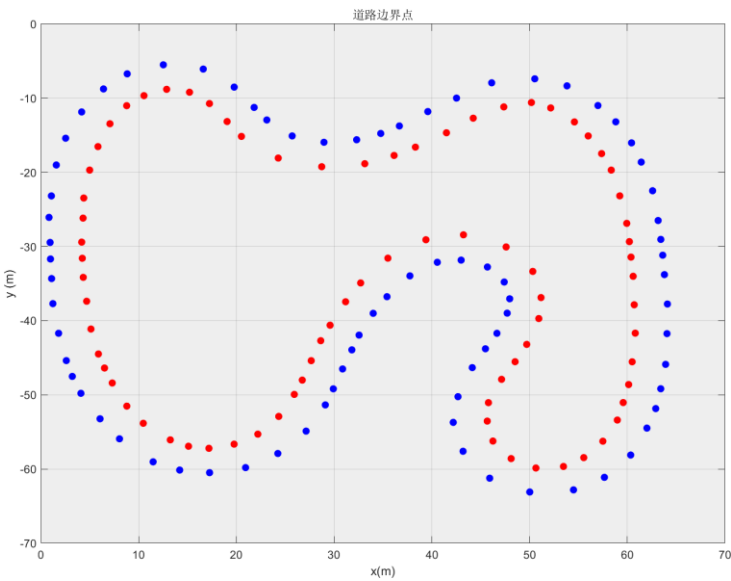


图 4.17 锥桶赛道边界

图 4.18 为 Delaunay 三角剖分算法求解的道路中心线。图中黄色边框为一帧的计算结果，蓝色为 Delaunay 三角的内边缘，黑色点为内边缘中点，即道路中心点，红色线条为各帧道路中心点的拟合曲线拼接而成的完整道路中心线。图 4.18 的求解结果表明本文的 Delaunay 三角剖分算法能够求解锥桶赛道的道路中心线。

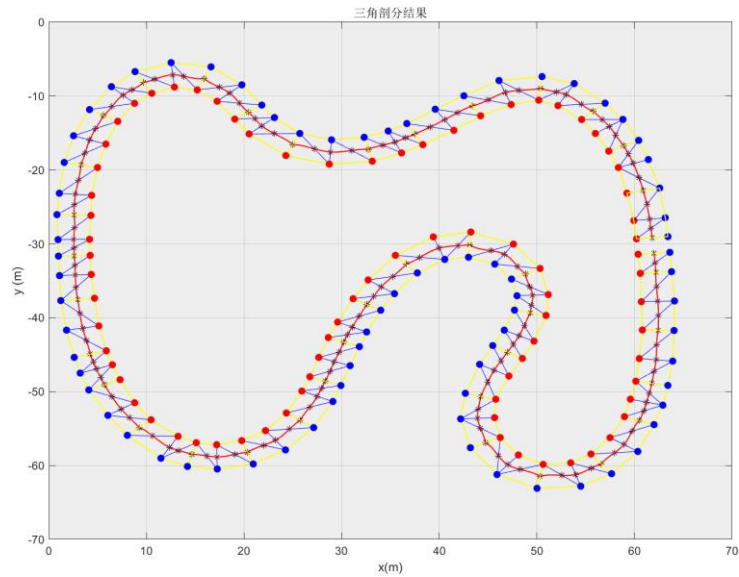


图 4.18 道路中心线

4.4.2 Lattice 轨迹规划算法仿真验证

在仿真环境中，利用三次样条插值算法平滑图 4.18 中的道路中心线，利用该曲线模拟参考线进行轨迹规划仿真，道路中心线拟合的参考线如图 4.19 所示。

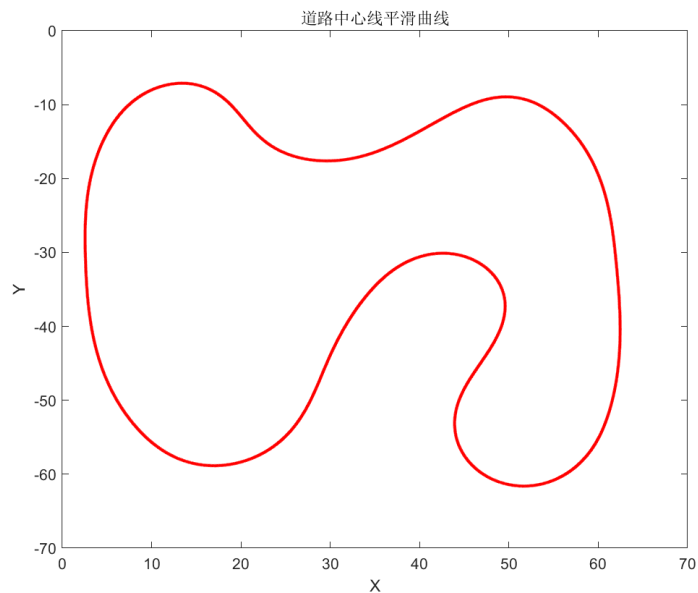


图 4.19 轨迹规划参考线

本文轨迹规划的参数配置如表 4.1 所示：

表 4.1 轨迹规划参数配置表

参数名称	数值
道路宽度 L	3m
道路横向采样间隔 Δl	0.35m
道路横向采样点(m)	$[-0.7,-0.35,0,0.35,0.7]$

表 4.1 轨迹规划参数配置表（续）

时间采样间隔 Δt	1s
规划周期 dt	0.1s
规划时间 T	5s
纵向速度采样间隔 Δv	1.4m/s
停车距离	20m
纵向速度采样数量 N	3

1、巡航工况轨迹规划仿真

赛车在巡航工况下距离参考线法向 0.85m 处启动，规划输出全局坐标系下最优合成轨迹如图 4.20 所示，蓝色点为当前轨迹的离散轨迹点。

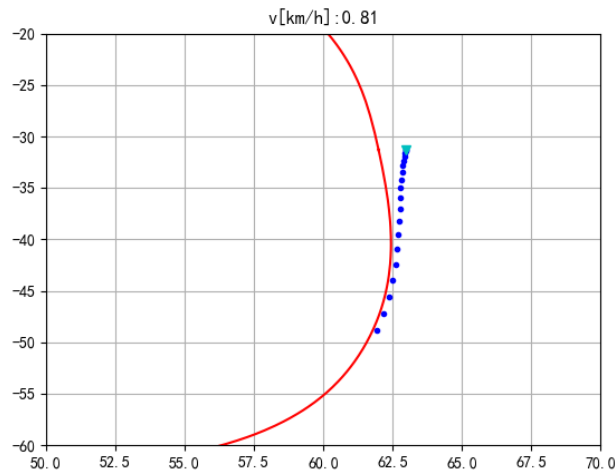


图 4.20 巡航工况合成轨迹输出

如图 4.21 所示，在巡航工况下，根据规划参数对横向终端状态采样，共得 25 条横向轨迹，图 4.21 红色轨迹为经过赛车运动学，动力学约束检查后，代价值最小的横向最优轨迹。

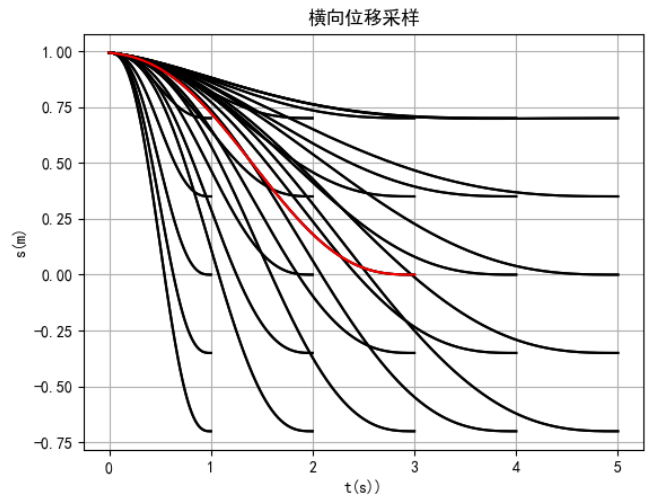


图 4.21 巡航工况横向最优采样轨迹

如图 4.22 所示，横向最优轨迹的速度、加速度曲线连续且平滑，且末状态的速度、加速度均为 0，满足规划要求。横向速度的绝对值最大值为 0.59m/s ，横向加速度最大值 0.58m/s^2 ，满足车辆约束要求。

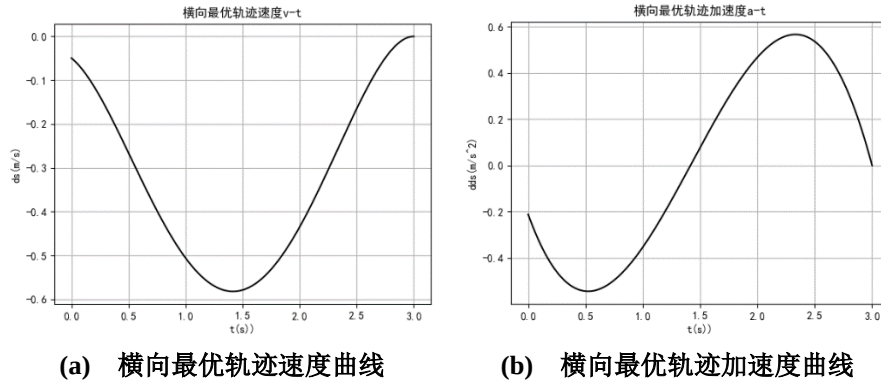


图 4.22 巡航工况横向最优轨迹速度和加速度

如图 4.23 所示，在巡航工况下，根据规划参数对纵向终端状态采样，共得 15 条纵向轨迹，图 4.23 红色轨迹为经过赛车运动学，动力学约束检查后，代价值最小的纵向最优轨迹。

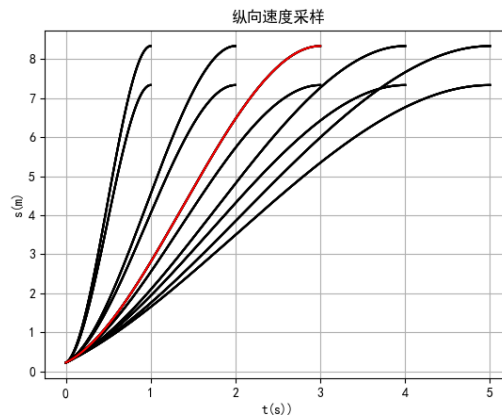


图 4.23 巡航工况纵向最优采样轨迹

如图 4.24 所示，纵向最优轨迹的加速度线从 0 增大到最大值 3.8m/s^2 后减小为 0，满足巡航时加速度为 0 的要求，且曲线连续、平滑，加速度满足车辆约束要求。

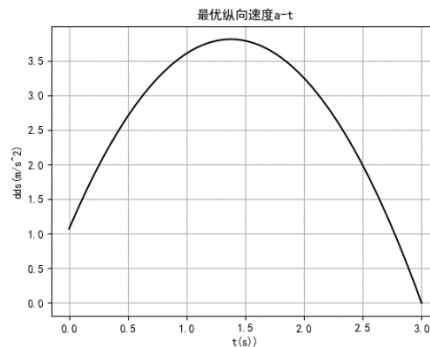


图 4.24 巡航工况纵向最优轨迹加速度

2、停车工况轨迹规划仿真

设置赛道停车点在 $s=248.3$ 处，赛车行驶到 $s=228$ 处时切换巡航工况到减速停车工况，此时规划输出的全局坐标系下最优合成轨迹如图 4.25 所示。

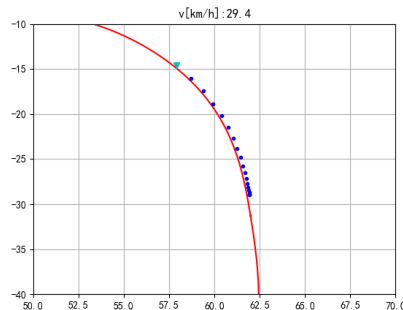


图 4.25 减速停车工况合成轨迹输出

如图 4.26 所示，在停车工况下，根据规划参数对横向终端状态采样，得 25 条横向采样轨迹。其中红色轨迹为代价值最小的横向最优轨迹。

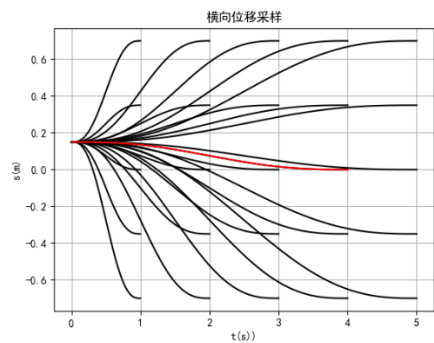


图 4.26 减速停车工况横向最优采样轨迹

如图 4.27 所示，横向最优轨迹的横向速度、加速度曲线连续且平滑，末状态的速度和加速度均为 0，而在整个规划周期内轨迹横向速度、加速度几乎为 0，这表明赛车在停车工况时缓慢调整车身以贴近参考线行驶，表明赛车在停车工况下的横向轨迹规划表现良好。

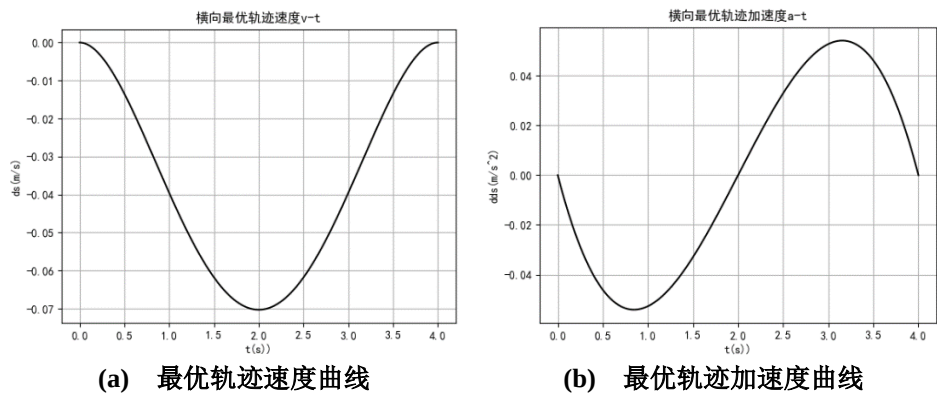


图 4.27 减速停车工况横向最优轨迹速度与加速度

如图 4.28 所示,在停车工况下,根据已知停车点,仅在时间上采样得到 5 条候选轨迹。其中红色轨迹为代价值最小的纵向最优轨迹。

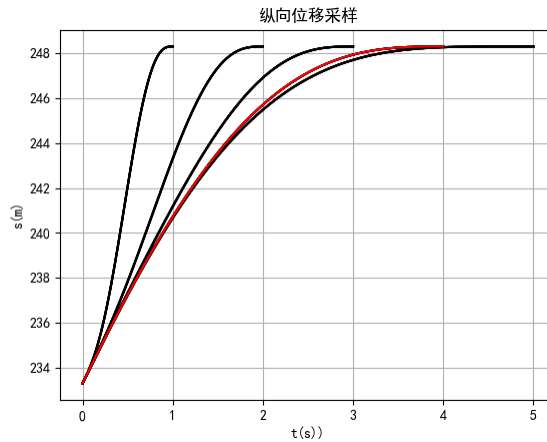


图 4.28 减速停车工况纵向最优采样轨迹

如图 4.29(a)所示,纵向最优轨迹速度曲线的初速度为 8.1m/s ,此时赛车减速行驶,直至停车时速度为 0。如图 4.29(b),加速度曲线先减小到 -2.9m/s^2 ,直到停车时加速度变为 0。由图 4.29 纵向最优轨迹的速度,加速度末状态均为 0,满足轨迹规划要求。

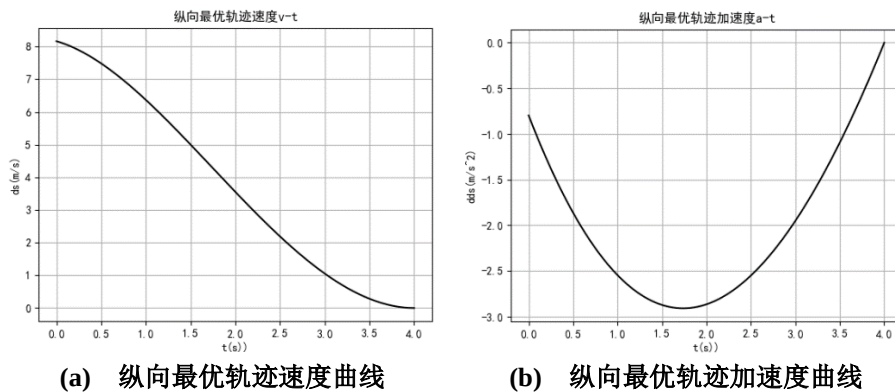


图 4.29 减速停车工况纵向最优轨迹速度与加速度

图 4.30 为仿真环境下赛车的行驶路径,其中红色为全局参考线,蓝色为赛车规划轨迹。如图 4.30 所示,赛车从起步后逐渐靠近参考线行驶,后续规划路径与参考线基本重合。图 4.31 为仿真的速度规划结果,由图可知赛车的规划速度从 0 开始平稳增加,达到巡航速度后保持该速度行驶,当检测到停车点后,开始减速行驶,当赛车到达终点时,速度降为 0。图 4.32 为规划的加速度规划结果,由图可知赛车起步时,加速度逐渐增加,到达峰值 2.3m/s^2 后逐渐降低,当速度达到巡航速度时,加速度减为 0。当赛车检测到停车时,开始增加减速度,到达峰值 -2.9m/s^2 后开始减小减速度,当赛车到达终点时加速度变为 0。

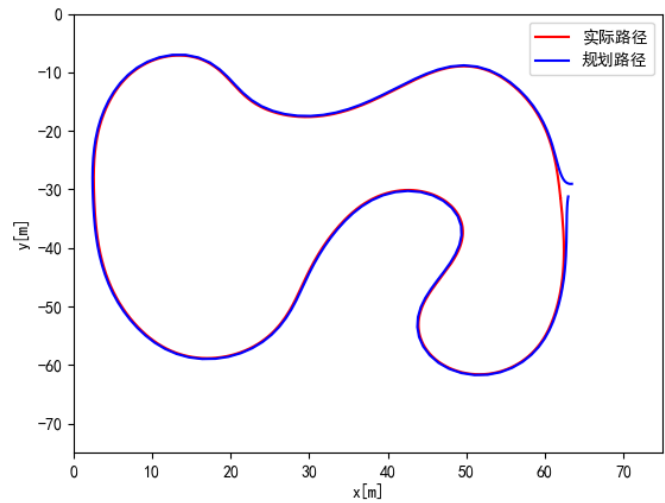


图 4.30 最优轨迹路径曲线

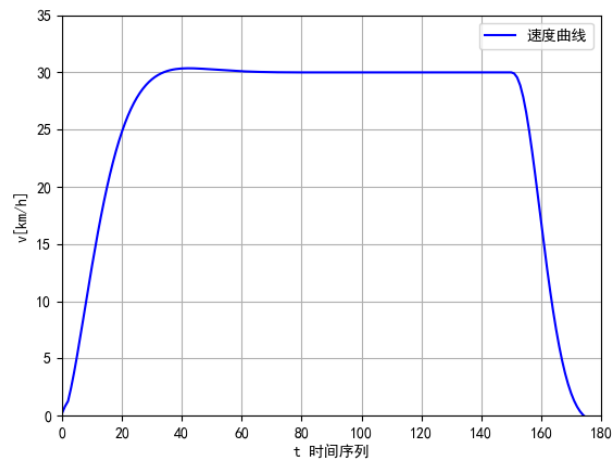


图 4.31 规划轨迹速度曲线

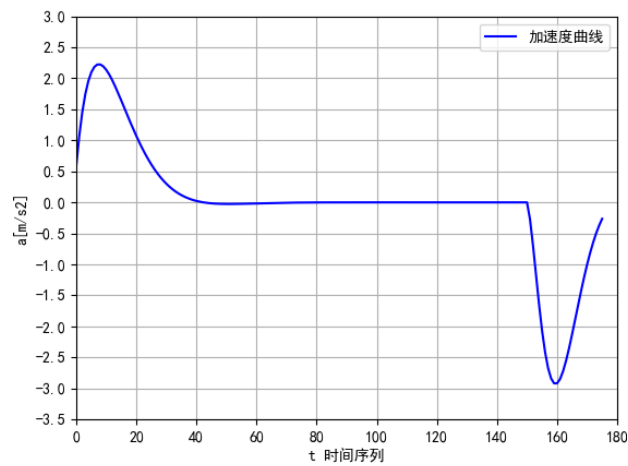


图 4.32 规划轨迹加速度曲线

综合图 4.20~图 4.32 可知出本文轨迹规划算法能够满足赛车在巡航工况和停车工况的轨迹规划要求，且输出轨迹连续且光滑，满足车辆运动学和动力学约束。

4.5 本章小结

本章主要研究用于锥桶赛道的道路中心线生成算法，以及高速循迹赛道的轨迹规划算法。首先利用 Delaunay 三角剖分求解道路中心点，然后利用三次样条插值分别对道路中心线和全局参考线做平滑处理。利用全局参考线在 Frenet 坐标系下进行高速循迹轨迹规划。在仿真环境下搭建锥桶赛道，验证道路中心线求解算法的正确性。并利用道路中心线近似作为参考线，对 Lattice 轨迹规划算法进行仿真验证，实验结果表明该算法能够满足赛车高速循迹赛道的轨迹规划要求。

第五章 实车数据分析

本文按照无人方程式大赛的赛事规则，设计赛车的无人系统，完成相应的环境感知和路径规划任务，但要实现赛车无人驾驶还需要各种硬件，软件支撑。本章将搭建无人方程式赛车实验平台，然后基于 ROS 系统开发感知规划算法，最后搭建实验锥桶赛道验证赛车的建图效果以及高速循迹的轨迹规划性能。

5.1 无人方程式赛车实验平台

本文的无人方程式赛车实验平台如图 5.1 所示。赛车的硬件组成为：双目相机、组合惯性导航系统、高压电池箱、电机控制器、线控转向系统、线控制动系统、无线遥控系统及系统状态指示灯。

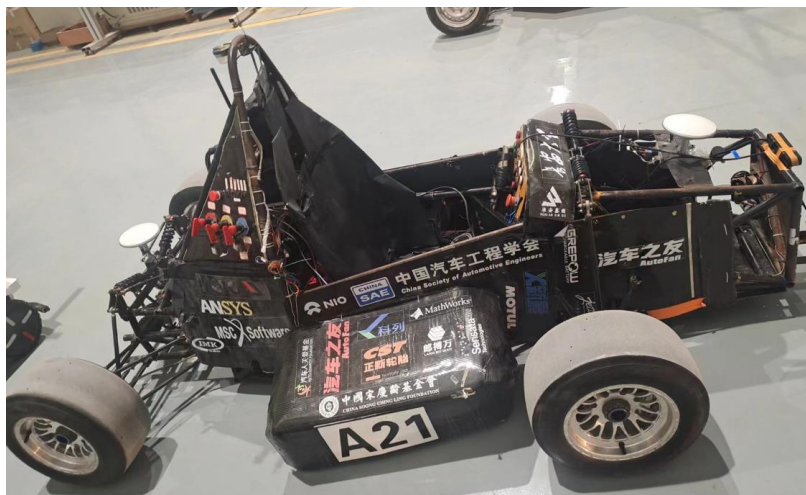


图 5.1 实验平台车

整车参数如表 5.1 所示：

表 5.1 赛车参数表

参数名称	参数值
质量	261kg
Z 轴转动惯量	403kg · m ²
轮距	1610mm
轴距	1620mm
质心到前轴距离	740mm
质心到后轴距离	880mm

5.1.1 整车控制系统

整车实验平台的车控系统如图 5.2 所示，使用笔记本电脑作为车载上位机，STM32F407 为下位机主控芯片。

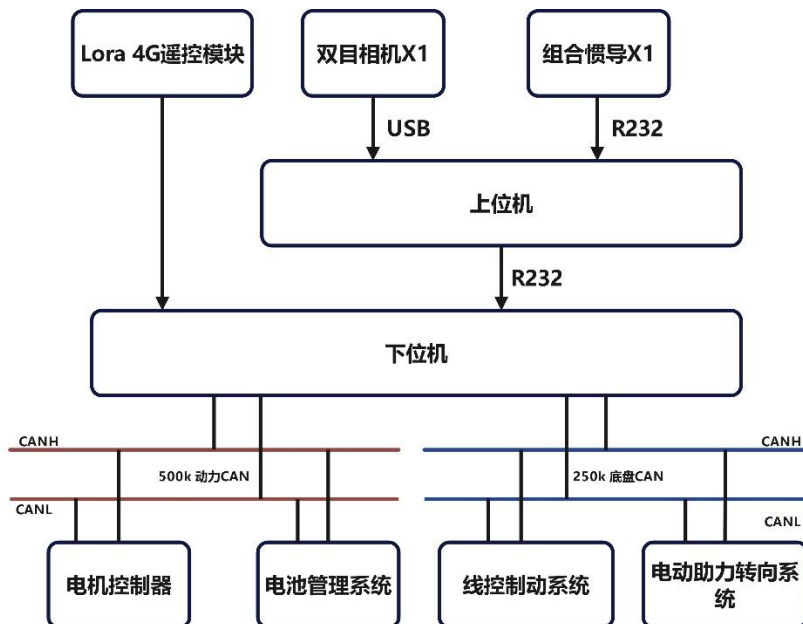


图 5.2 整车车控系统

上位机运行无人系统算法，负责收集感知系统的传感器信息，利用感知，规划和控制算法输出整车的制动力大小、油门开度和方向盘转角控制量。下位机根据上位机指令控制电机控制器、线控转向系统和制动系统输出无人算法所期望的电机扭矩，刹车，方向盘转角，实现赛车的无人行驶。双目相机，组合惯导分别以 USB 通讯和 RS232 将图像和定位信息传入给上位机。上位机以 RS232 与下位机进行通讯。下位机使用速率 500k 的 CAN 线与电机控制器和电池管理系统通讯，使用 250k 速率 CAN 与电机控制器，线控制动系统和电动助力转向系统通讯。遥控系统通过 Lora 4G 模块实现，用于在紧急状态下控制赛车停车。

5.1.2 感知规划算法软件设计

由于锥桶坐标和自车定位坐标来自不同的传感器，因此上位机需要具备数据同步和通信能力，而机器人操作系统（ROS）是一个灵活的框架，提供了大量的集成工具、库和协议能极大简化复杂机器人的开发。本文基于 Ubuntu 20.04/ROS Noetic 环境开发感知规划软件。

根据本文算法内容，共设计 4 个 ROS 节点，分别为 `vehicle_state_provide_node`，`cones_detect_node`，`mapping_node`，`planning_node`。其中 `vehicle_state_provide_node` 获取并解析组合惯导消息为车辆提供实时定位信息并记录赛车行驶轨迹。`cones_detect_node` 使用 YOLOv7 模型检测锥桶，并利用双目深度信息求解锥桶的世界坐标。`mapping_node` 负责锥桶空间坐标和定位信息的同步，并建立赛道地图。`planning_node` 负责高速循迹的轨迹规划。各个节点间的通讯机制如图 5.3 所示。

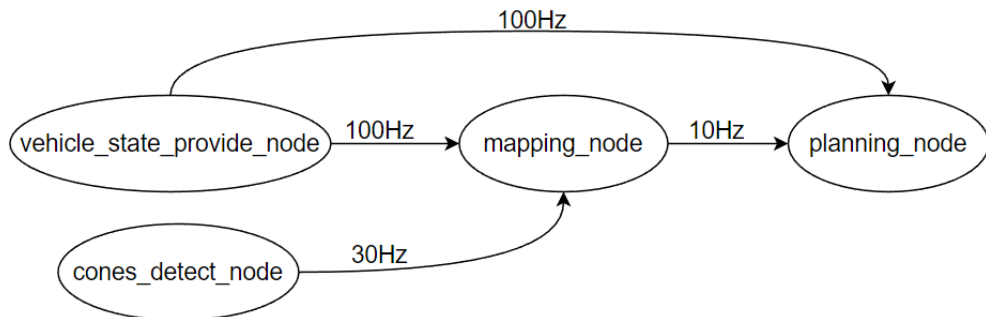


图 5.3 ROS 节点通讯调度

1、车辆状态解析节点 `vehicle_state_provide_node`

赛车的建图和轨迹规划算法都依赖实时的定位信息，因此 `vehicle_state_provide_node` 需要解析组合惯导的通讯数据包，并录制赛车建图时的行驶轨迹，软件流程如图 5.4 所示：

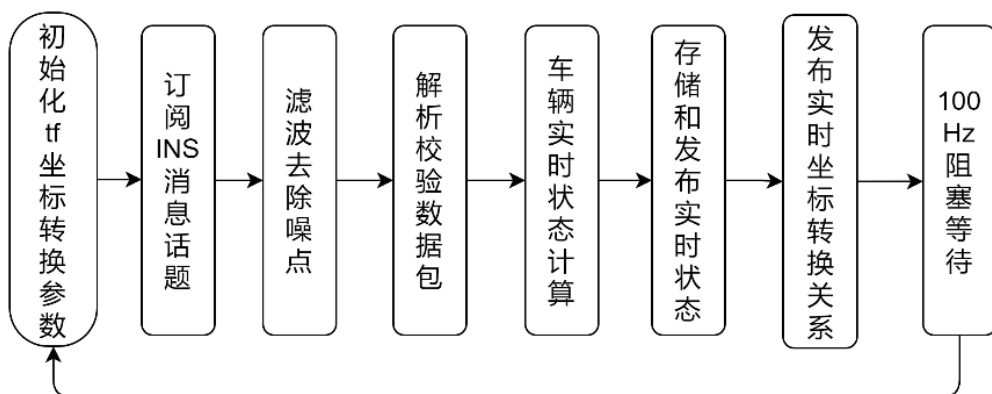


图 5.4 车辆状态解析软件流程

2、锥桶检测与空间坐标转换节点 `cones_detect_node`

锥桶检测与空间坐标转换节点负责读取双目相机的图像，利用 YOLOv7 锥桶检测模型检测锥桶，并结合双目视觉求解锥桶相对相机坐标系的世界坐标，该节点软件流程如图 5.5 所示：

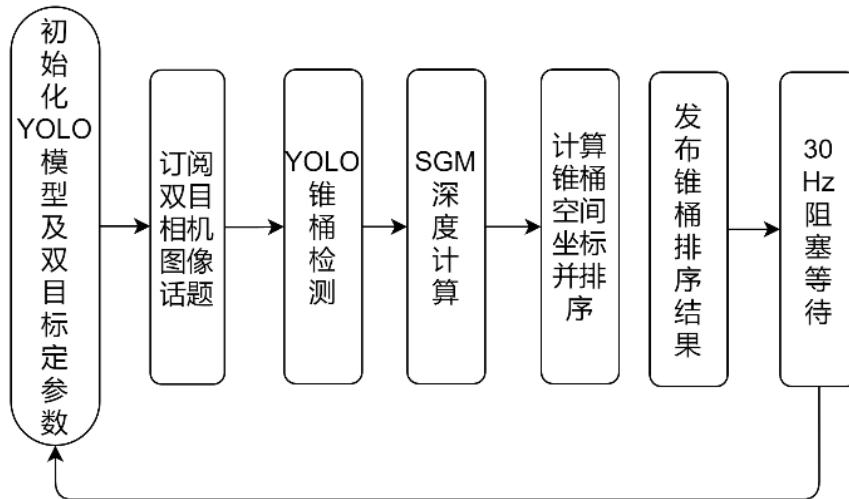


图 5.5 锥桶检测与空间坐标转换软件流程

(三)建图节点 mapping_node

该节点的功能包括同步组合惯导的定位信息和锥桶坐标信息，扩展弯道内边界坐标，求解平滑的道路中心线，软件流程如图 5.6 所示：

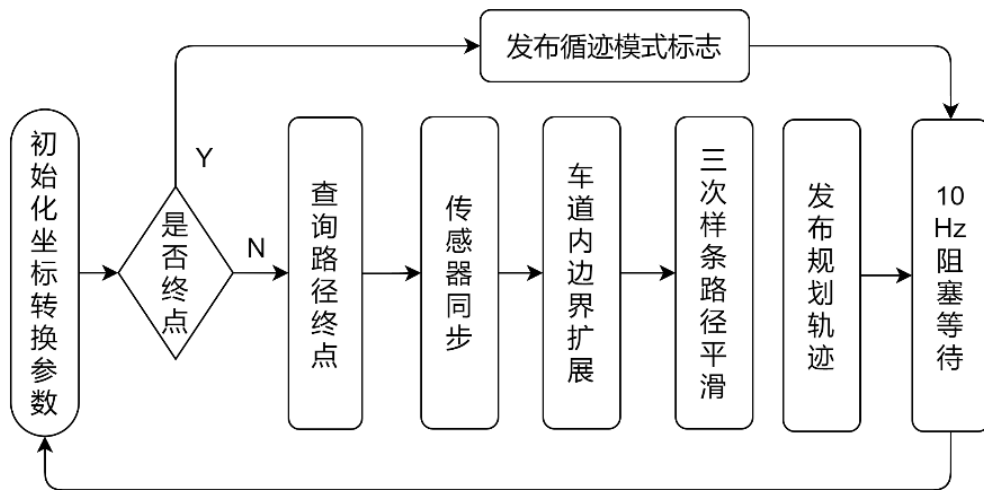


图 5.6 锥桶赛道建图软件流程

(四)轨迹规划节点 planning_node

当完成建图后需要对行驶路径做平滑处理，生成轨迹规划的参考线，然后根据轨迹规划算法完成轨迹规划，并发布最优轨迹(时间戳，全局坐标，距离，航向角，曲率，速度，加速度)，软件流程如图 5.7 所示：

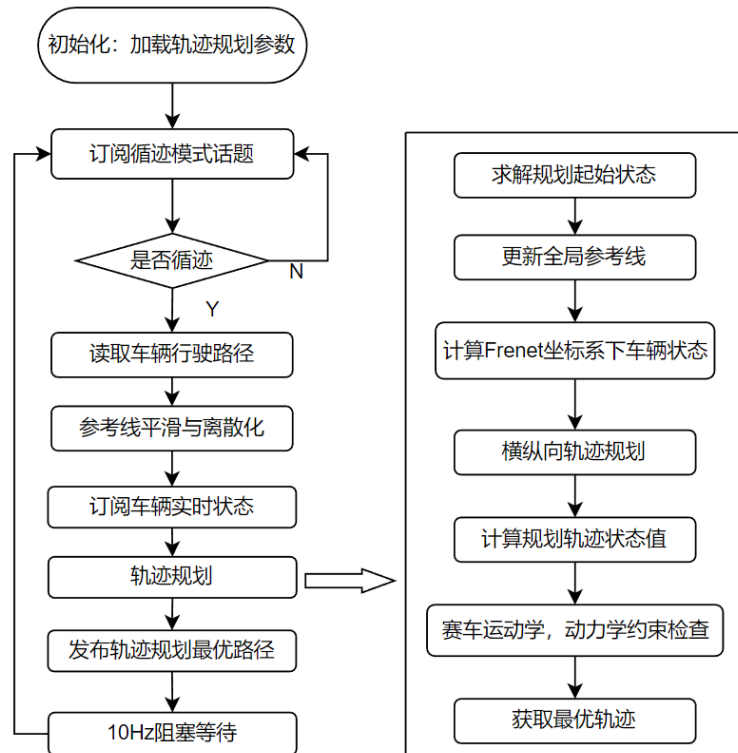


图 5.7 轨迹规划软件流程

5.2 赛车轨迹规划实车验证

图 5.8 为在长安大学校内汽车试验场背后小广场用锥桶搭建的实验赛道，赛道左侧为红色锥桶，赛道右侧为蓝色锥桶，锥桶间距 2-3m，道路宽度为 3m。



图 5.8 实验锥桶赛道

赛车利用纯追踪算法来跟踪规划轨迹，控制车辆行驶。纯追踪算法计算车辆与期望路径上某一参考点之间的误差，并根据误差调整车辆的行驶状态以跟踪规划路径。算法首先在规划路径上确定前视距离，然后计算车辆当前位置与参考点间的横纵向误差，根据误差计算车辆需要调整的转向角，从而控制车辆跟踪规划轨迹。赛车轨迹跟踪算法由车队其他成员开发。

图 5.9 为赛车行驶在直线道路时的锥桶道路识别过程。图 5.9(a)为双目相机左右目图像及深度图，可以看出无论在水泥路还是柏油路都能准确识别锥桶。图 5.9(b)中路线坐标系与实际道路坐标系 y 轴指向相反，因此实际道路为左边红色锥桶和右边红色锥桶，对应建图界面中左侧蓝色锥桶边界和右侧红色锥桶边界。

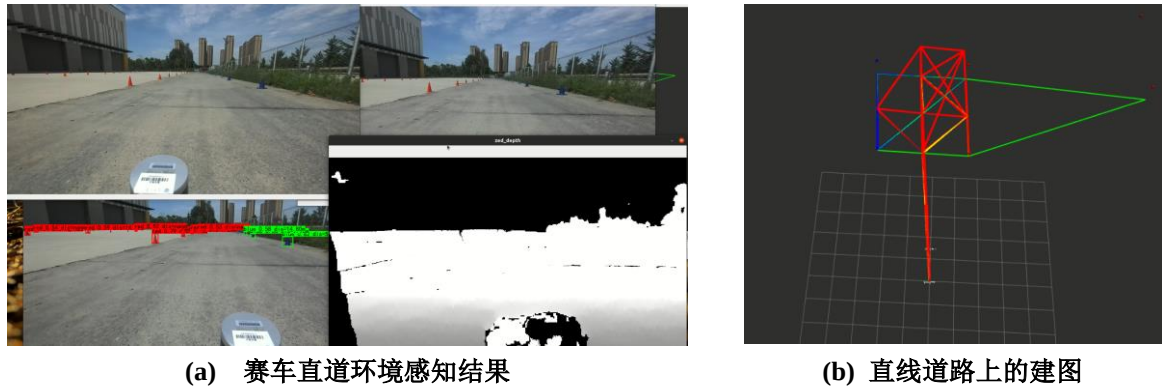


图 5.9 赛车直道环境感知与建图过程

图 5.10 为赛车行驶在弯道时的建图过程，图 5.10(b)中两边红色、蓝色三角边界，中间红色为候选规划路径，绿色为三角剖分算法生成的三角，由于坐标系 y 轴指向相反，因此实际车辆左转，对应图 5.10(b)黄色右转道路中心线。

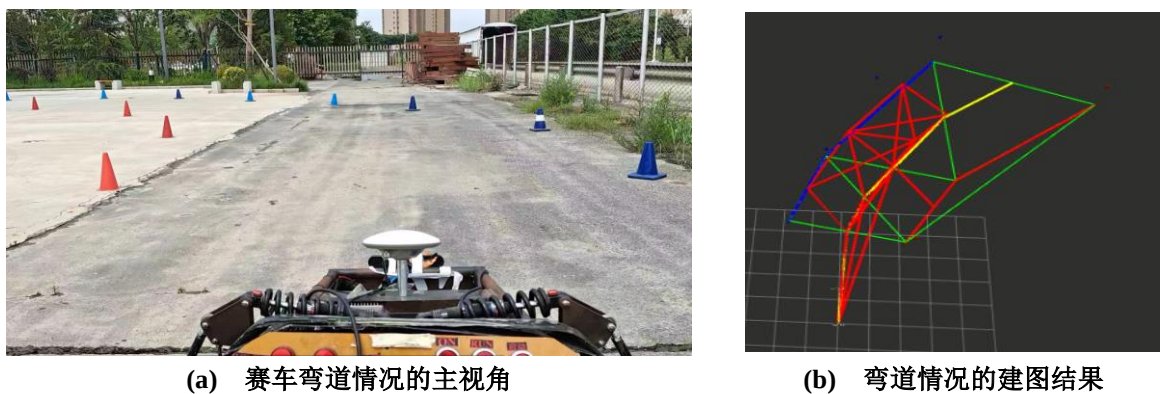


图 5.10 首圈赛道实时建图结果

图 5.11 中绿色曲线为赛车建图时的行驶路径点。

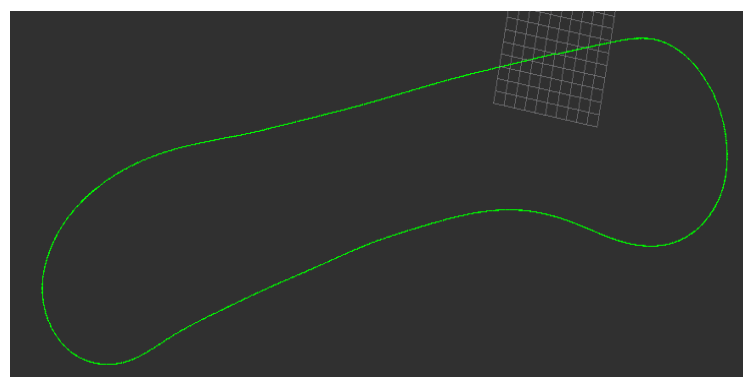


图 5.11 赛车行驶轨迹

图 5.12 为赛车循迹行驶时的轨迹规划结果。其中红色散点为赛车以规划轨迹为参考的实际行驶路径，用以表征轨迹规划效果。其中黄色曲线表示参考线与实际轨迹的重合度，黄色越深，重合度越高。

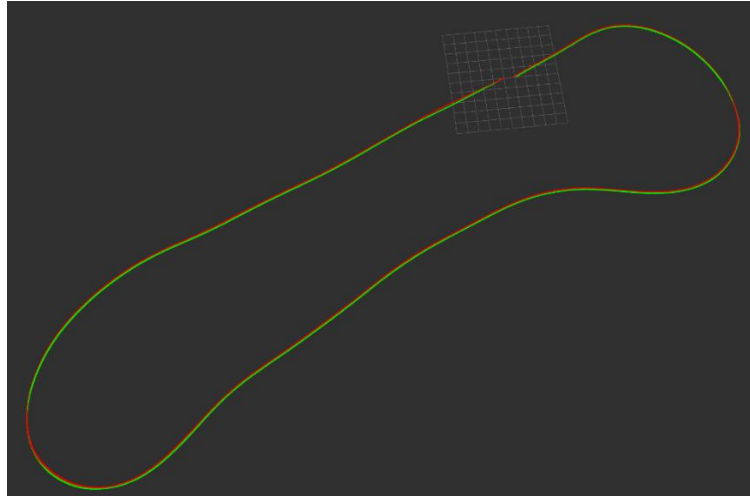


图 5.12 轨迹规划结果

由图 5.12 可知赛车直线行驶时，规划轨迹与参考线基本重合。但在弯道行驶时，规划轨迹与参考线略有偏离。在脱离弯道后，赛车的规划轨迹逐渐靠近参考线，随后沿参考线行驶至终点。

图 5.13 为赛道轨迹规划在全局坐标系下的航向角平滑后的曲线。结果表明航向角连续，但在局部位置出现跳变，这是因为全局坐标系下，赛车完成多次转向后，赛车向起点方向行驶，等同于车辆掉头，因此航向角的跳变属正常现象，但是车辆在其他位置行驶时航向角均平滑。

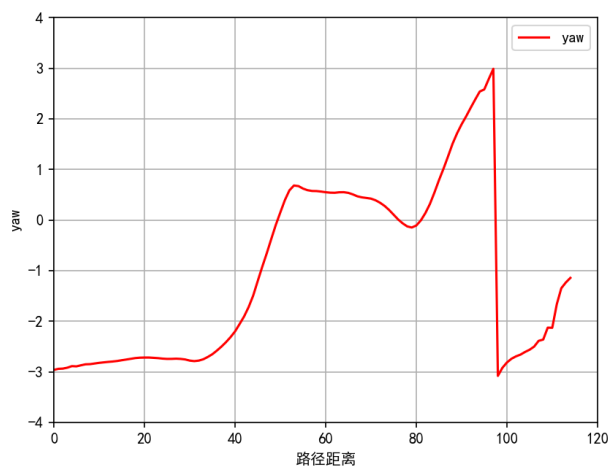


图 5.13 轨迹规划航向角

由图 5.9~图 5.13 表明，本文道路感知算法和轨迹规划算法的实车表现良好。

5.3 本章小结

本章首先搭建实验平台，分析赛车的硬件结构、整车控制系统和整车参数。然后基于 ROS 系统开发无人驾驶赛车的感知和规划系统。最后在校园内搭建赛道，进行实车验证，并对建图结果和轨迹规划进行分析，实验结果表明本文的道路感知和轨迹规划算法能够满足赛车比赛要求。

总结与展望

总结

本文针对无人驾驶赛车的道路感知以及高速循迹赛道的轨迹规划做了以下研究：

(1) 训练基于 YOLOv7 锥桶检测模型，实现锥桶赛道边界的识别。首先，采集了大量不同视角和道路场景下的锥桶数据集并对数据集进行标注，利用标注数据集训练锥桶检测模型，并在验证集上对模型进行验证，结果表明模型对各种场景下的锥桶的识别精确度达 98%，达到目标检测精度要求。然后，经双目实时检测效果可知，模型能将图像中的锥桶全部识别出来，且一张图片的检测时间仅 20ms 左右，证明模型具有良好的实时性、精确性和鲁棒性。最后对检测道的锥桶坐标进行排序，为后续的定位建图提供有序的锥桶坐标序列。

(2) 实现了赛道的定位与建图，首先，对双目相机进行标定获取相机的内外参数，并结合 SGM 算法得到的深度图和双目相机成像原理求解锥桶的世界坐标。实验结果表明双目视觉算法对锥桶坐标检测精度在 0.2m 以内，满足低速情况下的道路采集需求。然后，对组合惯性导航系统进行标定，融合锥桶的世界坐标和赛车定位信息完成赛道地图的建立，然后扩展赛道外边界，解决弯道情况下相机对赛道的识别不完整问题，由弯道情况的建图结果表明本文感知建图算法能够完成赛道的建图工作。

(3) 根据赛道建图结果完成赛车的轨迹规划任务。首先利用 Delaunay 三角剖分算法求解道路中点，并利用三次样条插值算法求解道路中心线。将赛车建图时的行驶轨迹经平滑和离散后得到轨迹规划所需的全局参考线。然后在 Frenet 坐标系下设计 Lattice 轨迹规划算法完成动态轨迹规划。由仿真实验结果表明本文的 Delaunay 三角剖分算法能完成锥桶赛道中心线的求解。在 python 环境下对轨迹规划算法进行仿真，实验结果表明，本文轨迹规划算法满足赛车巡航工况和减速停车工况的轨迹要求，且规划轨迹满足赛车运动学、动力学约束。

(4) 搭建实验平台，利用 ROS 系统开发感知规划算法程序。以实验车为平台进行实车验证。实验结果表明本文的道路感知算法能够建立锥桶赛道地图，同时本文轨迹规划算法规划的轨迹在赛道中心波动，波动在可接受范围内，满足赛车行驶要求。

展望

本文对于无人方程式赛车的道路感知和规划算法研究达到预期效果，但是由于时间和实验条件等多方面的限制，本文仍存在许多不足之处，未来将针对不足进一步开展以下研究：

（1）本文锥桶道路是在实验场景下搭建，场景较为简单，但是真实赛车的比赛环境更为复杂，因此需要采集更多场景的锥桶数据集，训练适用于更加复杂场景的检测模型，提高算法的实用性能。

（2）本文目标检测算法会对同一个锥桶进行多次重复检测，未来将展开 DeepSORT 多目标追踪模型研究，提升算法性能。同时展开多目标检测研究，拓展算法的应用场景。

（3）本文根据实际项目需求选择的双目相机，由于相机帧率较低，仅能满足低速情况的建图需求，其检测距离较短，同时算法仍然存在噪声。未来将会进一步研究基于深度学习的双目实时检测算法，提高检测精度和实时性。

（4）本文双目传感器和惯导位置仅通过简单的坐标变化进行融合后建立赛道地图，还存在较大误差，为了获取更加精确的建图信息，对双目相机和惯导进行联合标定，开展传感器的融合研究，提高检测精度。保证赛车行驶时建图的可靠性和稳定性。

（5）实际赛车电机控制效果不够理想，因此实际使用的道路仅为轨迹规划的直角坐标，会导致部分道路上效果较差，未来将展开无人赛车下位机的控制算法研究。以进一步完善轨迹规划算法。

参考文献

- [1] 陈秀娟.全国汽车保有量达 2.81 亿辆[J].汽车观察,2021(01):7.
- [2] 大学生方程式系列赛事 [EB/OL]. 中国汽车工程学会 :
<http://www.formulastudent.com.cn/>
- [3] Liu L, Ouyang W, Wang X, et al. Deep learning for generic object detection: A survey[J]. International journal of computer vision, 2020, 128: 261-318.
- [4] 宋晓琳, 邬紫阳, 张伟伟. 基于阴影和类 Haar 特征的动态车辆检测[J]. 电子测量与仪器学报, 2015, 29(9): 1340-1347.
- [5] 徐渊, 许晓亮, 李才年, 等. 结合 SVM 分类器与 HOG 特征提取的行人检测[J]. 计算机工程, 2016, 42(1): 56-60.
- [6] Lowe D G. Distinctive image features from scale-invariant keypoints[J]. International journal of computer vision, 2004, 60: 91-110.
- [7] 陈鑫. 基于 YOLOv5 和脉冲神经网络的智能抓取系统[D].中国科学院大学(中国科学院深圳先进技术研究院),2022.DOI:10.27822/d.cnki.gszxj.2022.000153.
- [8] Girshick R, Donahue J, Darrell T, et al. Rich feature hierarchies for accurate object detection and semantic segmentation[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2014: 580-587.
- [9] He K, Zhang X, Ren S, et al. Spatial pyramid pooling in deep convolutional networks for visual recognition[J]. IEEE transactions on pattern analysis and machine intelligence, 2015, 37(9): 1904-1916.
- [10] Girshick R. Fast r-cnn[C]//Proceedings of the IEEE international conference on computer vision. 2015: 1440-1448.
- [11] Ren S, He K, Girshick R, et al. Faster r-cnn: Towards real-time object detection with region proposal networks[J]. Advances in neural information processing systems, 2015, 28.
- [12] Lin T Y, Dollár P, Girshick R, et al. Feature pyramid networks for object detection[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2017: 2117-2125.

- [13]Redmon J, Divvala S, Girshick R, et al. You only look once: Unified, real-time object detection[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 779-788.
- [14]李科岑, 王晓强, 林浩, 等. 深度学习中的单阶段小目标检测方法综述[J]. 计算机科学与探索, 2022, 16(1): 41.
- [15]李兆冬, 陶进, 安旭阳. 移动机器人目标识别与定位算法发展综述[J]. 车辆与动力技术, 2020 (1): 43-48.
- [16]Liu W, Anguelov D, Erhan D, et al. Ssd: Single shot multibox detector[C]//Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14. Springer International Publishing, 2016: 21-37.
- [17]Redmon J, Farhadi A. YOLO9000: better, faster, stronger[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2017: 7263-7271.
- [18]Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift[C]//International conference on machine learning. pmlr, 2015: 448-456.
- [19]Redmon J, Farhadi A. YOLOv3: An incremental improvement // IEEE Conference on Computer Vision and Pattern Recognition[J]. Honolulu, USA: IEEE, 2017: 6517-6525.
- [20]He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778.
- [21]Bochkovskiy A, Wang C Y, Liao H Y M. Yolov4: Optimal speed and accuracy of object detection[J]. arXiv preprint arXiv:2004.10934, 2020.
- [22]Liu S, Qi L, Qin H, et al. Path aggregation network for instance segmentation[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 8759-8768.
- [23]Cubuk E D, Zoph B, Mane D, et al. Autoaugment: Learning augmentation strategies from data[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2019: 113-123.

- [24]Zhang S, Chi C, Yao Y, et al. Bridging the gap between anchor-based and anchor-free detection via adaptive training sample selection[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2020: 9759-9768.
- [25]Ge Z, Liu S, Wang F, et al. YOLOx: Exceeding yolo series in 2021[J]. arXiv preprint arXiv:2107.08430, 2021.
- [26]Li C, Li L, Jiang H, et al. YOLOv6: A single-stage object detection framework for industrial applications[J]. arXiv preprint arXiv:2209.02976, 2022.
- [27]Ding X, Zhang X, Ma N, et al. RepVGG: Making vgg-style convnets great again[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2021: 13733-13742.
- [28]Wang C Y, Bochkovskiy A, Liao H Y M. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors[J]. arXiv preprint arXiv:2207.02696, 2022.
- [29]戚玲珑,高建瓴.基于改进 YOLOv7 的小目标检测[J].计算机工程,2023,49(01):41-48.DOI:10.19678/j.issn.1000-3428.0065942.
- [30]刘铁,郑南宁,程洪,等.基于变形模板和遗传算法的道路检测方法[J].模式识别与人工智能,2004,17(2):156.
- [31]Thrun S, Montemerlo M, Dahlkamp H, et al. Stanley: The robot that won the DARPA Grand Challenge[J]. Journal of field Robotics, 2006, 23(9): 661-692.
- [32]王会,罗涛,陆培源.激光雷达在无人车辆中的应用及关键技术分析[J].激光与红外,2018,48(12):1458-1467.
- [33]熊璐,杨兴,卓桂荣,等.无人驾驶车辆的运动控制发展现状综述[J].机械工程学报,2020,56(10):127-143.
- [34]Montemerlo M, Becker J, Bhat S, et al. The stanford entry in the urban challenge[J]. Journal of field Robotics, 2008, 7(9): 468-492.
- [35]陈佳佳.城市环境下无人驾驶车辆决策系统研究[D].中国科学技术大学,2014.
- [36]贺汉根,孙振平,徐昕.智能交通条件下车辆自动驾驶技术展望[J].中国科学基金,2016,30(2):106-111.
- [37]陈宇鹏.基于深度学习的自动驾驶单目视觉目标识别技术研究[D].吉林大学,2019.

- [38] Ho T, Lee S. Piezoelectrically actuated biomimetic self-contained quadruped bounding robot[J]. Journal of Bionic Engineering, 2009, 6(1): 29-36.
- [39]Yong H, Jianru X. Real-time traffic cone detection for autonomous vehicle[C]//2015 34th Chinese Control Conference (CCC). IEEE, 2015: 3718-3722.
- [40]王振, 王化明. 基于单目视觉的障碍物定位和测量[J]. 机械制造与自动化, 2015, 44(1): 198-201.
- [41]毛先胤, 吕黔苏, 马晓红, 等. 基于双目视觉的电力线巡检机器人障碍物定位测距[J]. 自动化与仪器仪表, 2021.
- [42]施友宁. 锥桶环境下的智能车辆感知与路径规划控制 [D]. 重庆理工大学, 2022. DOI:10.27753/d.cnki.gcqgx.2022.000489.
- [43]陈炎, 杨丽丽, 王振鹏. 双目视觉的匹配算法综述[J]. 图学学报, 2020, 41(5): 702.
- [44]Hirschmuller H. Stereo processing by semiglobal matching and mutual information[J]. IEEE Transactions on pattern analysis and machine intelligence, 2007, 30(2): 328-341.
- [45]顾征, 苏显渝. 采用色彩相似性约束的立体匹配[J]光电工程, 2007(01):95-99.
- [46]Dinh V Q, Pham C C, Jeon J W. Robust adaptive normalized cross-correlation for stereo matching cost computation[J]. IEEE Transactions on Circuits and Systems for Video Technology, 2016, 27(7): 1421-1434.
- [47]张令涛, 曲道奎, 徐方. 一种基于图割的改进立体匹配算法[J]. 机器人, 2010 (1): 104-108.
- [48]Pan C, Liu Y, Huang D. Novel belief propagation algorithm for stereo matching with a robust cost computation[J]. IEEE Access, 2019, 7: 29699-29708.
- [49]郭大波, 卢朝阳, 焦卫东, 等. 遮挡检测/立体匹配中的分段动态规划法[J]. 电子学报, 2009, 37(7): 1516.
- [50]Zhang X, Dai H, Sun H, et al. Algorithm and VLSI architecture co-design on efficient semi-global stereo matching[J]. IEEE Transactions on Circuits and Systems for Video Technology, 2019, 30(11): 4390-4403.
- [51]尹晨阳, 职恒辉, 李慧斌. 基于深度学习的双目立体匹配方法综述[J]. 计算机工程, 2022, 48(10):1-12. DOI:10.19678/j.issn.1000-3428.0064294.

- [52]Zbontar J, LeCun Y. Stereo matching by training a convolutional neural network to compare image patches[J]. J. Mach. Learn. Res., 2016, 17(1): 2287-2318.
- [53]Song X, Zhao X, Hu H, et al. Edgestereo: A context integrated residual pyramid network for stereo matching[C]//Computer Vision—ACCV 2018: 14th Asian Conference on Computer Vision, Perth, Australia, December 2–6, 2018, Revised Selected Papers, Part V 14. Springer International Publishing, 2019: 20-35.
- [54]Kendall A, Martirosyan H, Dasgupta S, et al. End-to-end learning of geometry and context for deep stereo regression[C]//Proceedings of the IEEE international conference on computer vision. 2017: 66-75.
- [55]Zhang F, Prisacariu V, Yang R, et al. Ga-net: Guided aggregation net for end-to-end stereo matching[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 185-194.
- [56]Chang J R, Chen Y S. Pyramid stereo matching network[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 5410-5418.
- [57]Paden B, Čáp M, Yong S Z, et al. A survey of motion planning and control techniques for self-driving urban vehicles[J]. IEEE Transactions on intelligent vehicles, 2016, 1(1): 33-55.
- [58]李柏,张友民,邵之江.自动驾驶车辆运动规划方法综述[J].控制与信息技术,2018(06):1-6.
- [59]Tran N, Nguyen D T, Vu D L, et al. Global path planning for autonomous robots using modified visibility-graph[C]//2013 International Conference on Control, Automation and Information Sciences (ICCAIS). IEEE, 2013: 317-321.
- [60]Fusic S J, Ramkumar P, Hariharan K. Path planning of robot using modified dijkstra Algorithm[C]//2018 National Power Engineering Conference (NPEC). IEEE, 2018: 1-5.
- [61]Wenzheng L, Junjun L, Shunli Y. An improved Dijkstra's algorithm for shortest path planning on 2D grid maps[C]//2019 IEEE 9th International Conference on Electronics Information and Emergency Communication (ICEIEC). IEEE, 2019: 438-441.
- [62]王鹤静,王丽娜.机器人路径规划算法综述[J/OL].桂林理工大学学报:1-15[2023-04-06].<http://kns.cnki.net/kcms/detail/45.1375.N.20221213.1104.001.html>.

- [63]Likhachev M, Gordon G J, Thrun S. ARA*: Anytime A* with provable bounds on sub-optimality[J]. Advances in neural information processing systems, 2003, 16.
- [64]Harabor D, Grastien A. Improving jump point search[C]//Proceedings of the International Conference on Automated Planning and Scheduling. 2014, 24: 128-135.
- [65]黄健萌,吴宇雄,林谢昭.移动机器人平滑 JPS 路径规划与轨迹优化方法[J].农业机械学报,2021,52(02):21-29+121.
- [66]Stentz A. Optimal and efficient path planning for partially known environments[J]. Intelligent Unmanned Ground Vehicles: Autonomous Navigation Research at Carnegie Mellon, 1997: 203-220.
- [67]张煜. 基于多传感器数据融合的无人车避障导航研究[D]. 西安工业大学, 2015.
- [68]王维, 裴东, 冯璋. 改进 A* 算法的移动机器人最短路径规划[J]. 计算机应用, 2018, 38(5): 1523-1526.
- [69]Liu Z, Liu H, Lu Z, et al. A dynamic fusion pathfinding algorithm using delaunay triangulation and improved a-star for mobile robots[J]. Ieee Access, 2021, 9: 20602-20621.
- [70]Huang Y, Ding H, Zhang Y, et al. A motion planning and tracking framework for autonomous vehicles based on artificial potential field elaborated resistance network approach[J]. IEEE Transactions on Industrial Electronics, 2019, 67(2): 1376-1386.
- [71]罗乾又, 张华, 王姮, 等. 改进人工势场法在机器人路径规划中的应用[J]. 计算机工程与设计, 2011, 32(4): 1411-1413.
- [72]Lin X, Wang Z Q, Chen X Y. Path planning with improved artificial potential field method based on decision tree[C]//2020 27th Saint Petersburg International Conference on Integrated Navigation Systems (ICINS). IEEE, 2020: 1-5.
- [73]韩伟, 孙凯彪. 基于模糊人工势场法的智能全向车路径规划[J]. 计算机工程与应用, 2018, 54(6): 105-109.
- [74]Zhang J, He Y, Peng Y N. Neural network and artificial potential field based cooperative and adversarial path planning[J]. Acta Aeronautica et Astronautical Sonica, 2019, 3: 228-238.
- [75]刘珏. 基于逆强化学习的舰载机牵引车路径规划研究[D]. 哈尔滨工程大学, 2017.

- [76]项宏峰, 曹少中, 徐长波, 等. 基于神经网络的 AGV 智能车路径规划的仿真研究[J]. 北京印刷学院学报, 2017, 25(7): 128-130.
- [77]余卓平, 李奕姗, 熊璐. 无人车运动规划算法综述[J]. 同济大学学报 (自然科学版), 2017, 45(8): 1150-1159.
- [78]宋晓琳, 周南, 黄正瑜, 等. 改进 RRT 在汽车避障局部路径规划中的应用[J]. 湖南大学学报: 自然科学版, 2017, 44(4): 30-37.
- [79]张兰勇, 韩宇. 基于改进的 RRT* 算法的 AUV 集群路径规划研究[J]. 中国舰船研究, 2022, 18: 1-9.
- [80]符强, 蓝星辉, 任风华等. 一种双向目标 RRT 路径规划算法研究[J]. 计算机仿真, 2023, 40(01): 447-454+461.
- [81]向金林, 王鸿东, 欧阳子路等. 基于改进双向 RRT 的无人艇局部路径规划算法研究[J]. 中国造船, 2020, 61(01): 157-166.
- [82]魏民祥, 滕德成, 吴树凡. 基于 Frenet 坐标系的自动驾驶轨迹规划与优化算法[J]. 控制与决策, 2021, 36(4): 815-824.
- [83]McNaughton M. Parallel algorithms for real-time motion planning[M]. Carnegie Mellon University, 2011.
- [84]符爱周, 韩震宇, 葛伊伦. 三次多项式加减速算法的应用研究[J]. 制造技术与机床, 2022 (1): 119-123.
- [85]Zhang Z, Li J, Shao W, et al. Differentiable learning-to-group channels via groupable convolutional neural networks[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2019: 3542-3551.
- [86]陆原超, 周俊荣, 雷焱焱, 等. 6R 机器人过路径点的轨迹规划与 MATLAB 仿真[J]. 机械工程师, 2020.
- [87]廖云荣. 基于双目视觉感知的无人车路径规划[D]. 西安工业大学, 2022. DOI:10.27391/d.cnki.gxagu.2022.000224.
- [88]李迎松. 摄影测量影像快速立体匹配关键技术研究. 武汉大学, 2018.
- [89]张杰, 王增才, 闫明. 基于改进 Census 变换和自适应权重的立体匹配算法[J]. 电子测量技术, 2022, 45(23): 45-52.

- [90]Makhzani A, Frey B J. Winner-take-all autoencoders[J]. Advances in neural information processing systems, 2015, 28.
- [91]刘宝, 狄鑫, 韩丽华. 改进三次样条插值在机械臂轨迹规划中的应用[J]. 机械科学与技术, 2021.